



数字集成电路设计前沿技术与挑战

刘雷波

清华大学集成电路学院

2025年11月12日





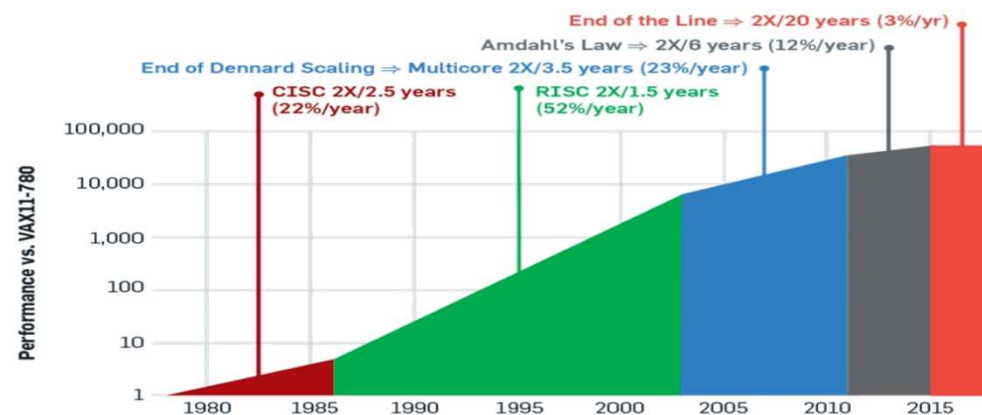
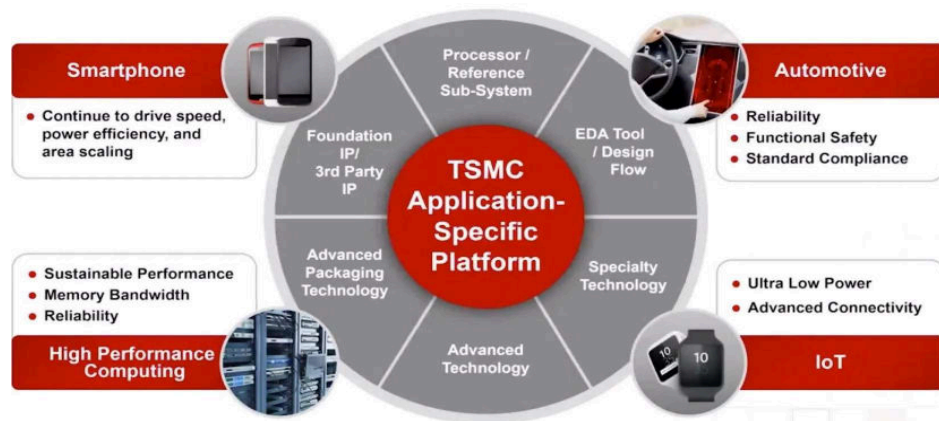
领域定制是集成电路的未来发展方向



台积电：应用驱动，集成电路设计需要新范式^[1]



Hennessy & Patterson：后摩尔时代的计算架构是领域定制^[3]



- 通用处理器体系结构的发展遭遇瓶颈^[3]

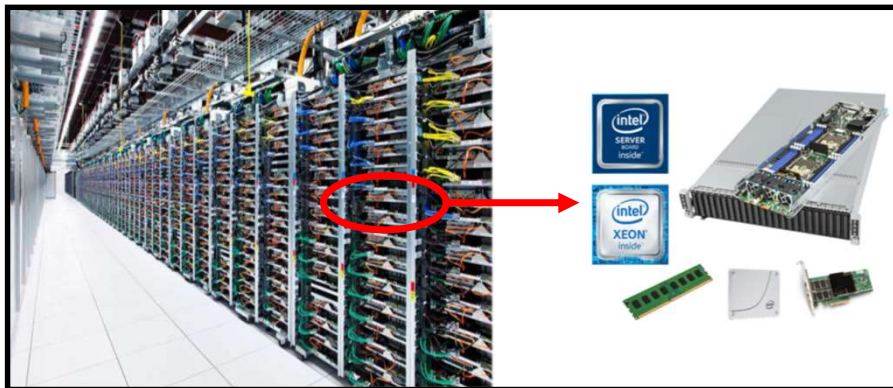
- 2017年，台积电提出要面向需求进行定制化开发^[2]
- 2020年，台积电提出application-specific platform^[1]

[1] Cliff Hou. OIP Ecosystem Forum 2020.

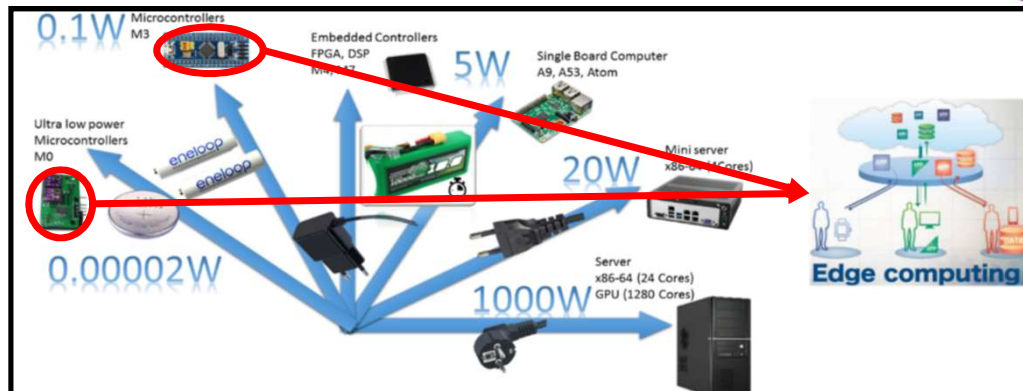
[2] Cliff Hou. "A smart design paradigm for smart chips," ISSCC 2017 (Plenary Talk).

[3] John Hennessy, David Patterson, "A New Golden Age Computer Architecture: Domain-Specific Hardware/Software Co-Design ...," ISCA 2018 (Plenary Talk).

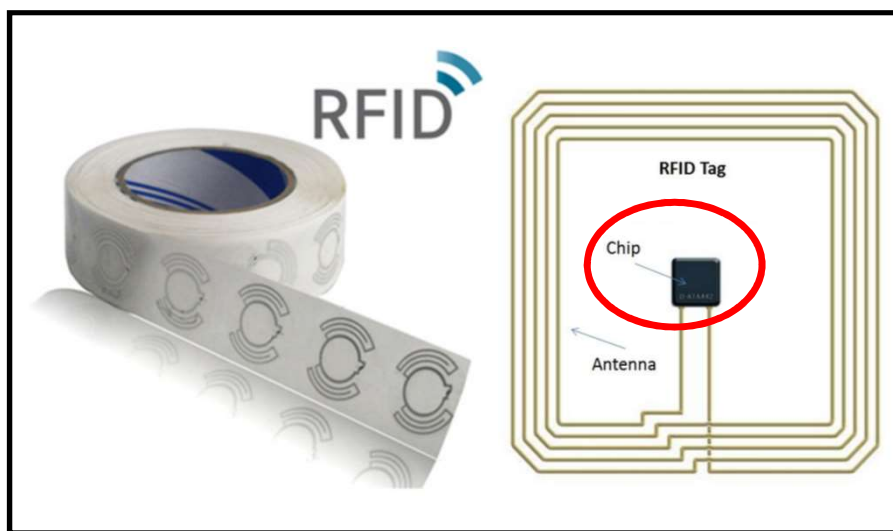
集成电路需求多样化



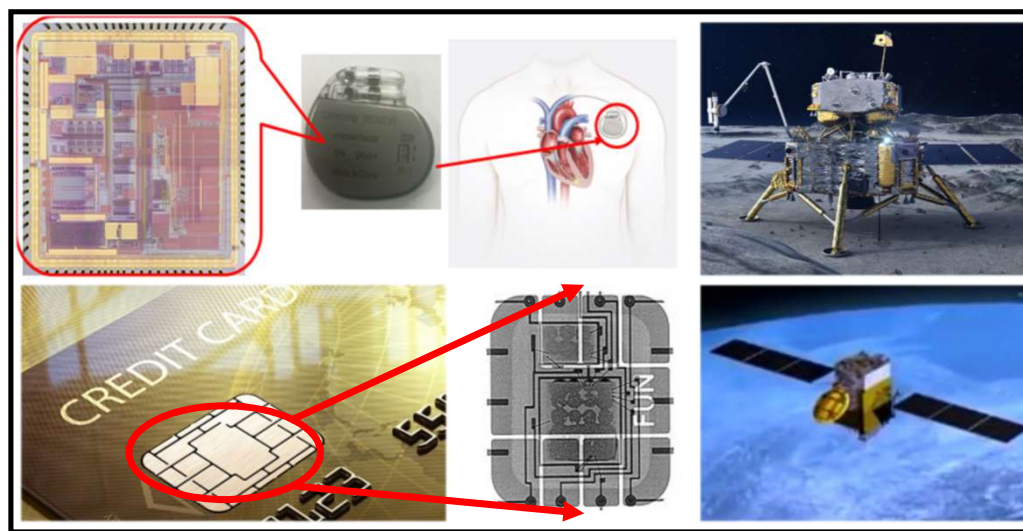
数据中心及云服务需求高算力



边缘计算需要高能效

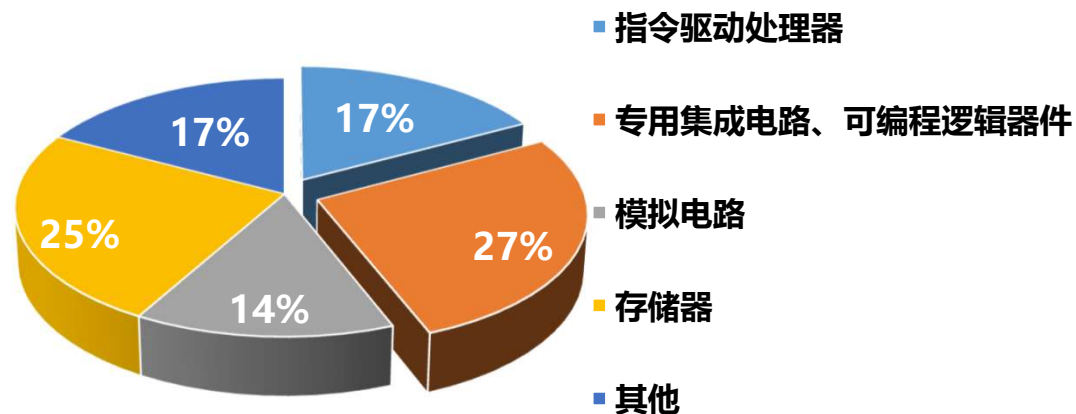
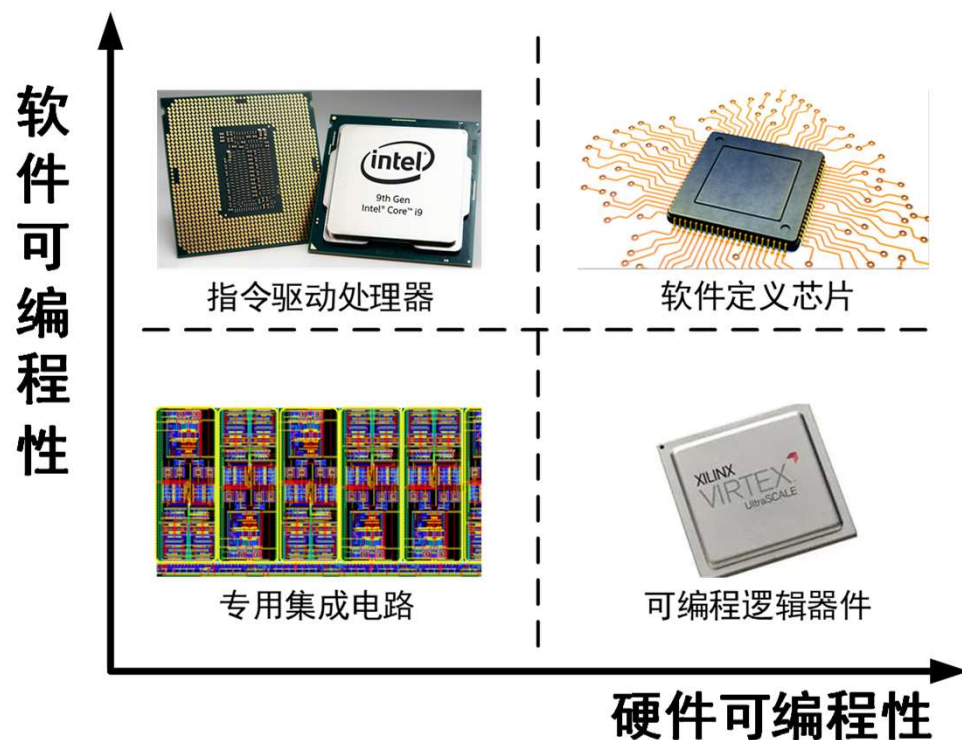


IoT节点 (RFID标签) 需要低成本



医疗、信用卡、航天需要高安全、高可靠

数字集成电路分类

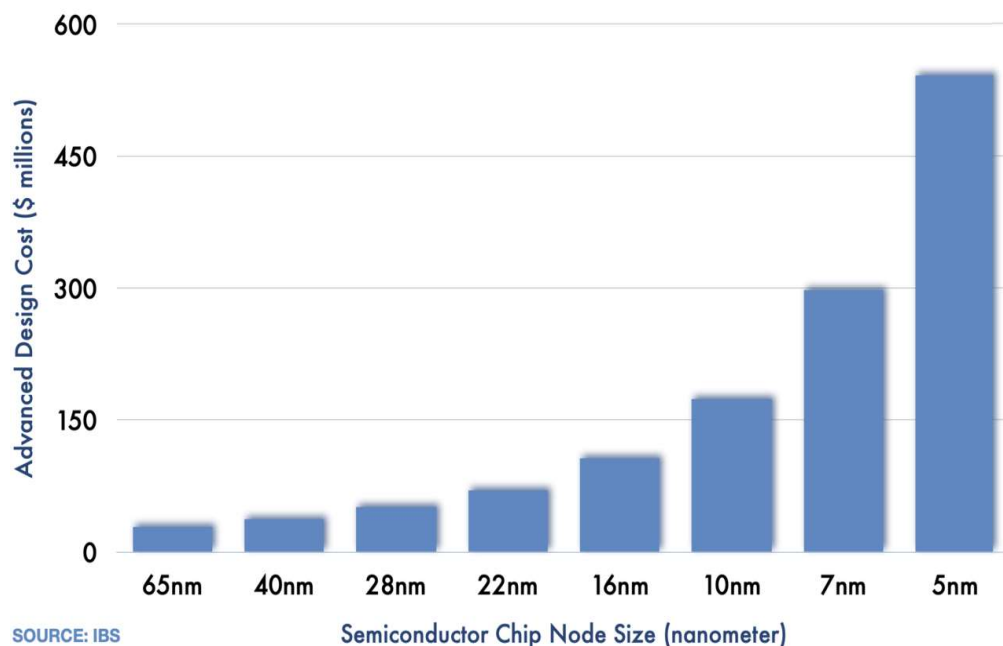


[1] Dr. Handel Jones, Whitepaper: Semiconductor Industry from 2015 to 2025, International Business Strategies (IBS), 2018

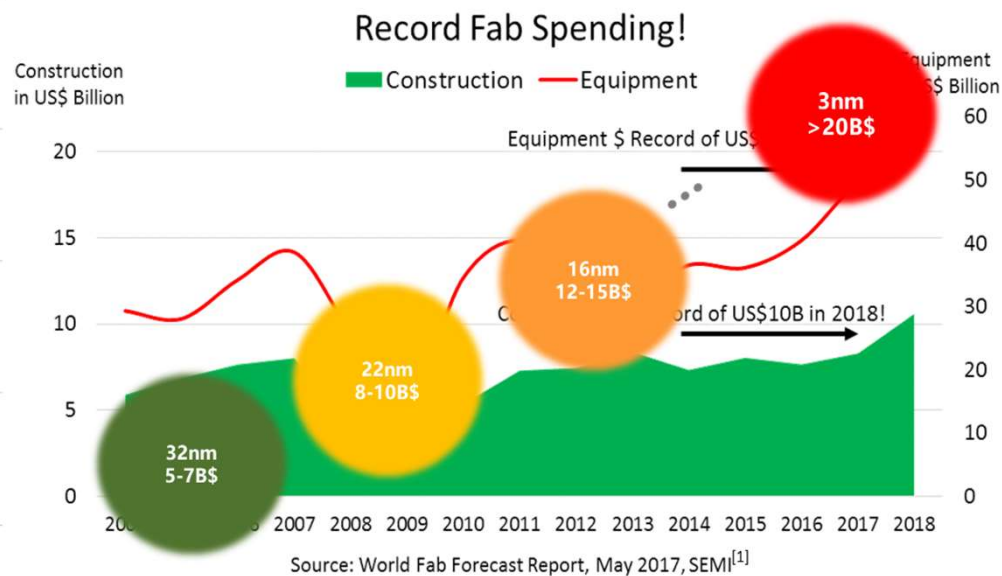
专用集成电路：ASIC



- 速度最快，功耗最低
- 销量较小，成本很高（灵活性差，硅后功能不变）



先进工艺下芯片成本高¹

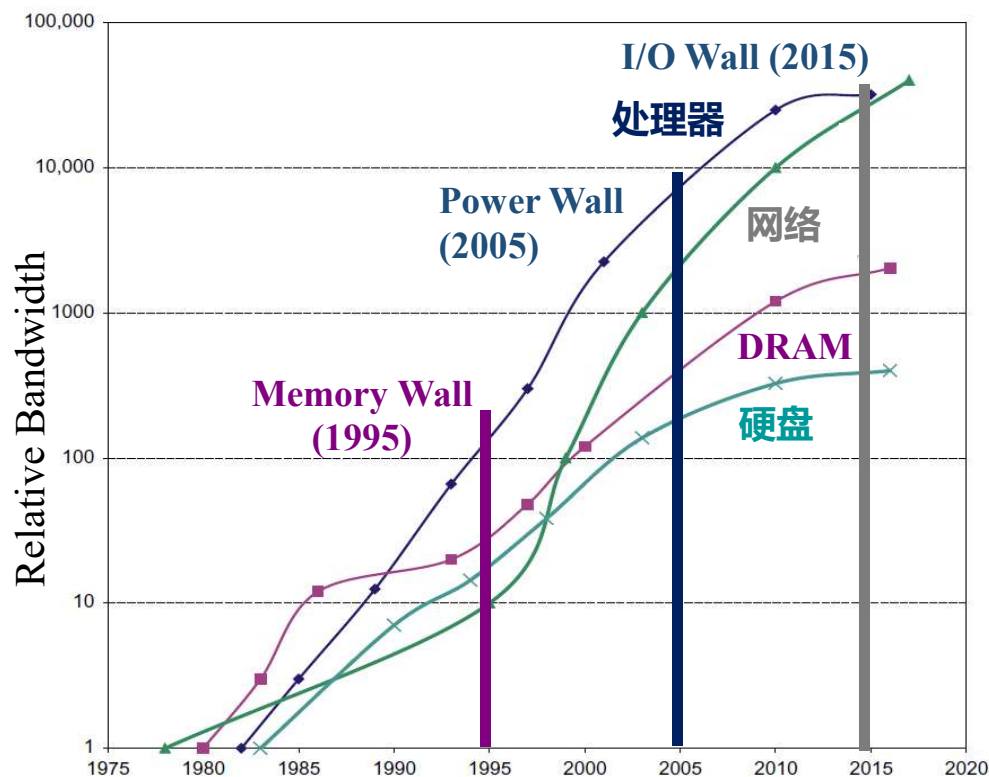


Fab建设及设备投入高²

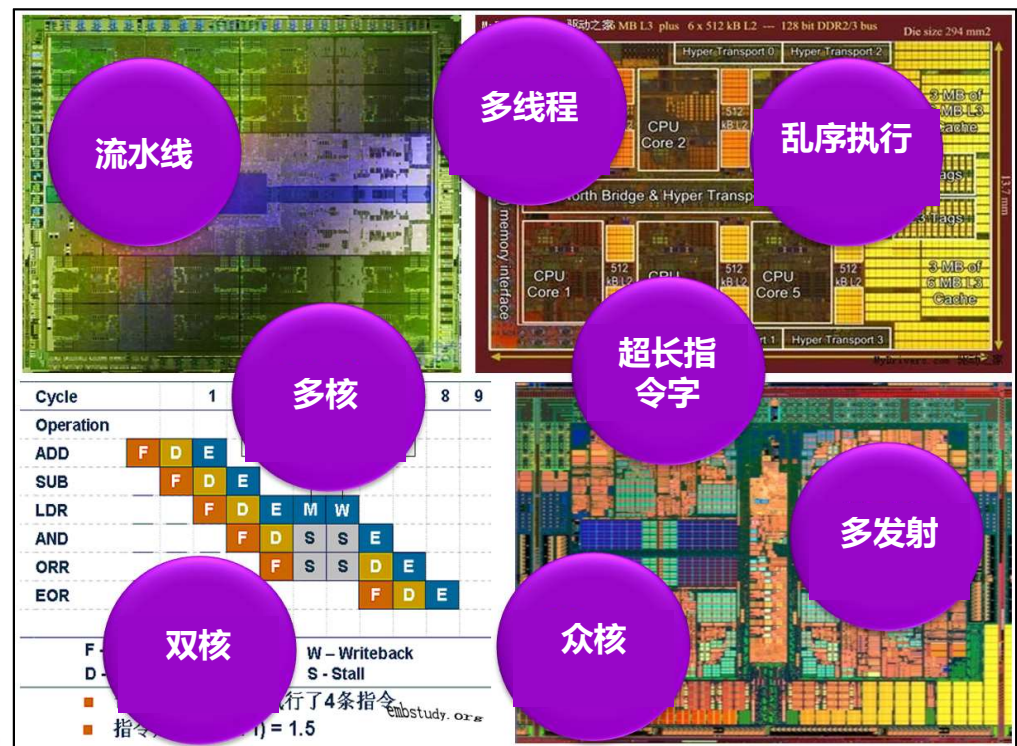
[1] Handel Jones, IBS, 2018. [2] 国际半导体产业协会(SEMI), 2017.

指令驱动处理器：CPU、GPU、DSP、ASIP等

- 灵活性最高，可编程性强，生态系统最为完备
- 能量效率太低、功耗大、面积大



“三墙” 制约¹

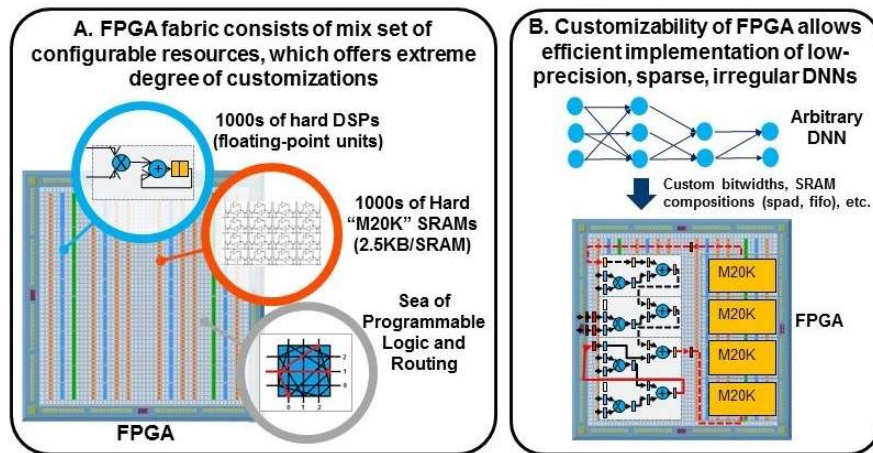


提高CPU并行计算能力的技术²

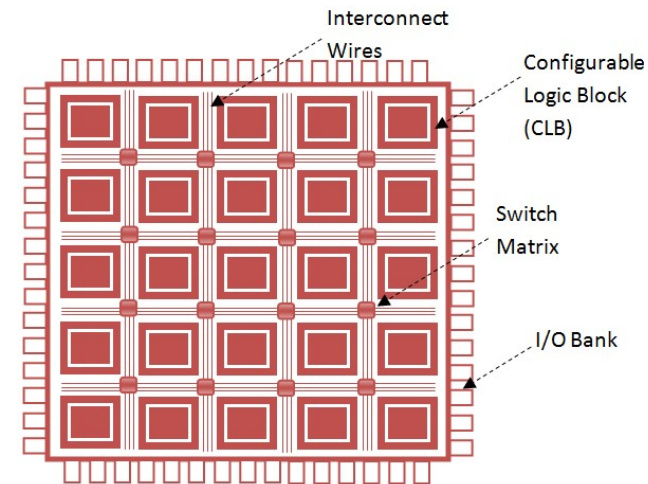
[1] Intel, 2016. [2] Asanovic, Krste, et al. "The landscape of parallel computing research: A view from berkeley." (2006): 19.

可编程逻辑器件：FPGA、CPLD

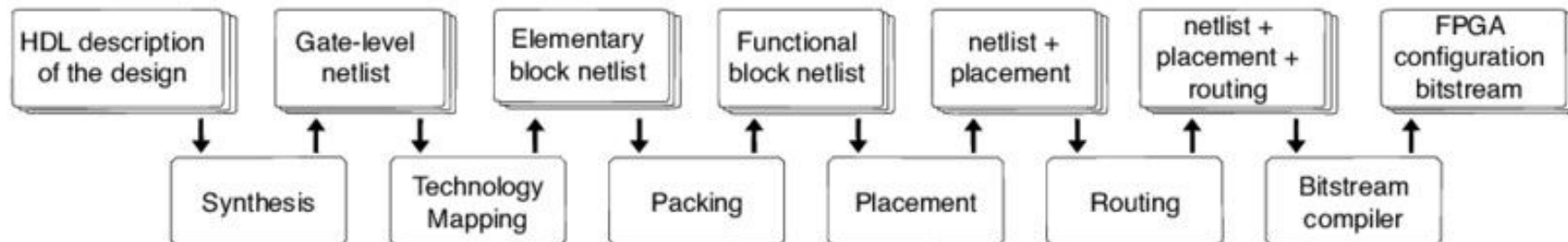
- 能快速完成定型，灵活性很高，大批量量产前被优先使用
- 编程门槛很高，能量效率较差，容量严重受限**



灵活性高¹



FPGA基本结构

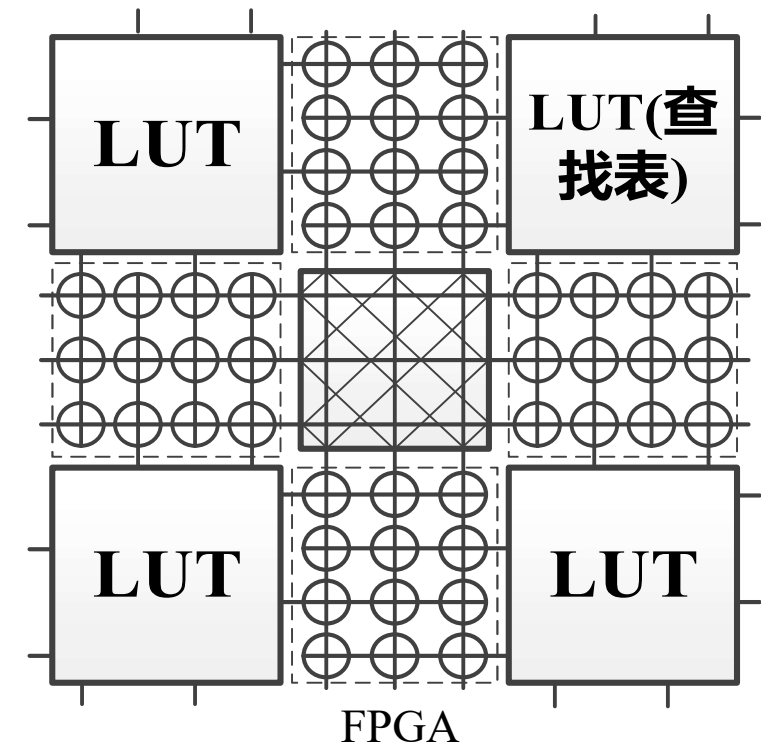
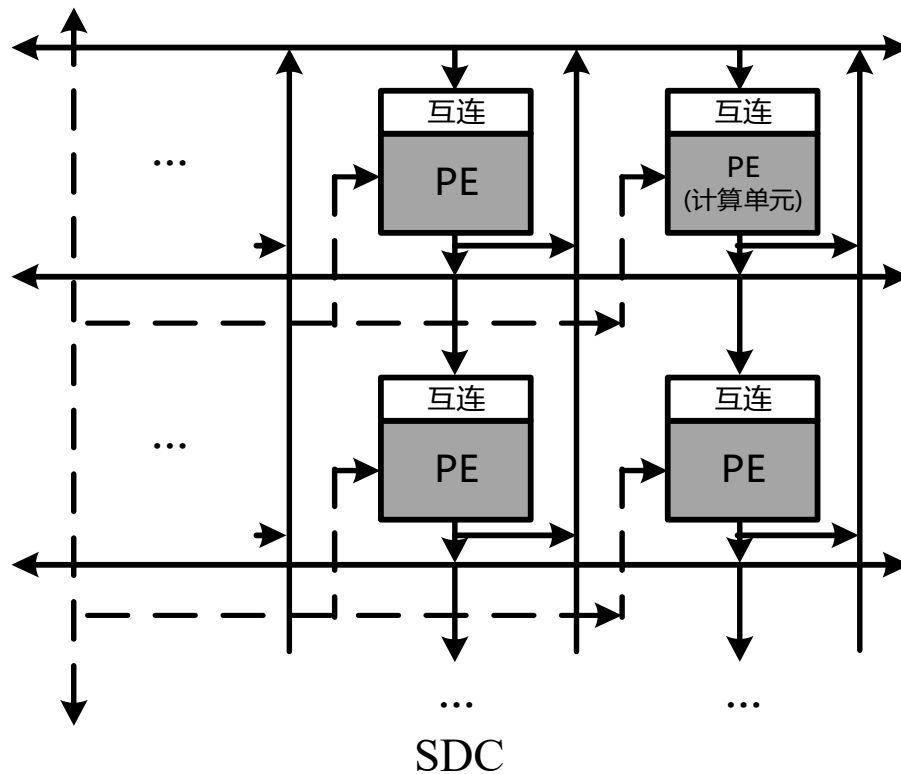


FPGA开发流程复杂²

[1] Intel Accelerator Architecture Lab (AAL), ISFPGA, 2017. [2] Dr Elias. Doctoral dissertation (2016).

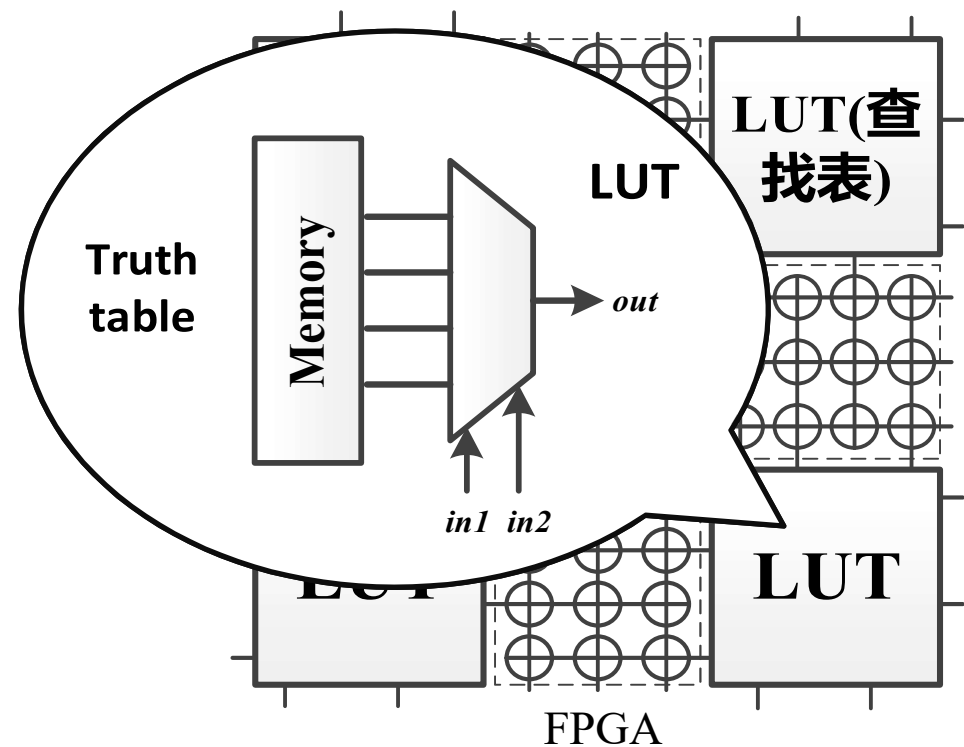
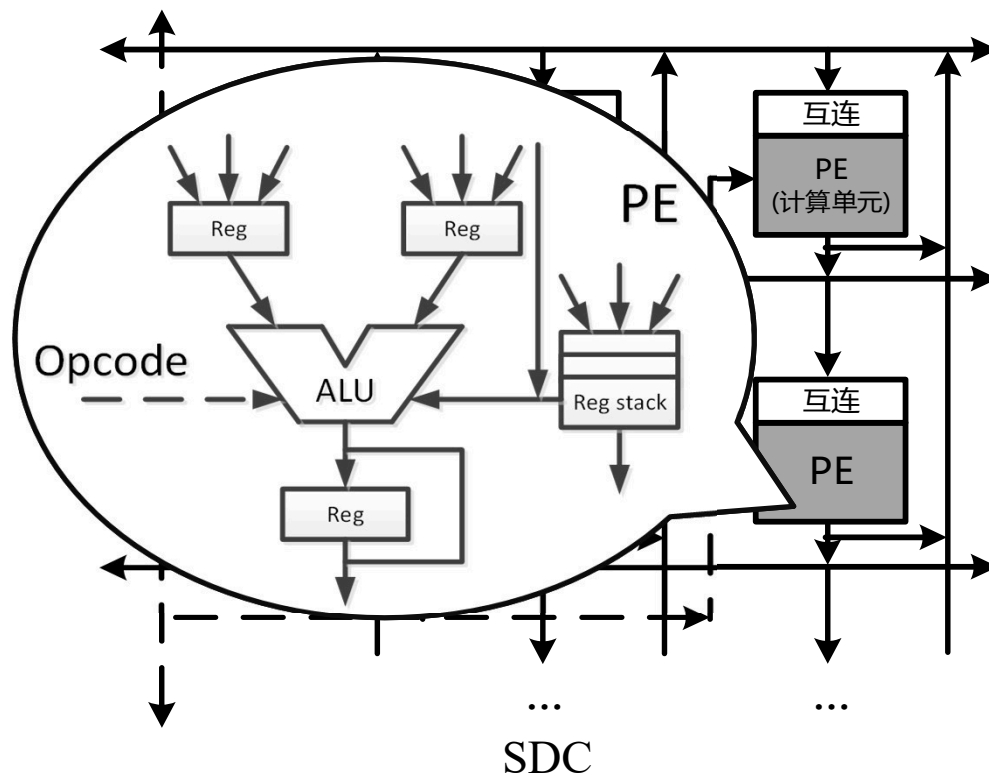
软件定义芯片：SDC

- 灵活性很高，能量效率很高（相对CPU/FPGA）
- **集成开发工具尚有关键技术需要突破，生态系统不完善**

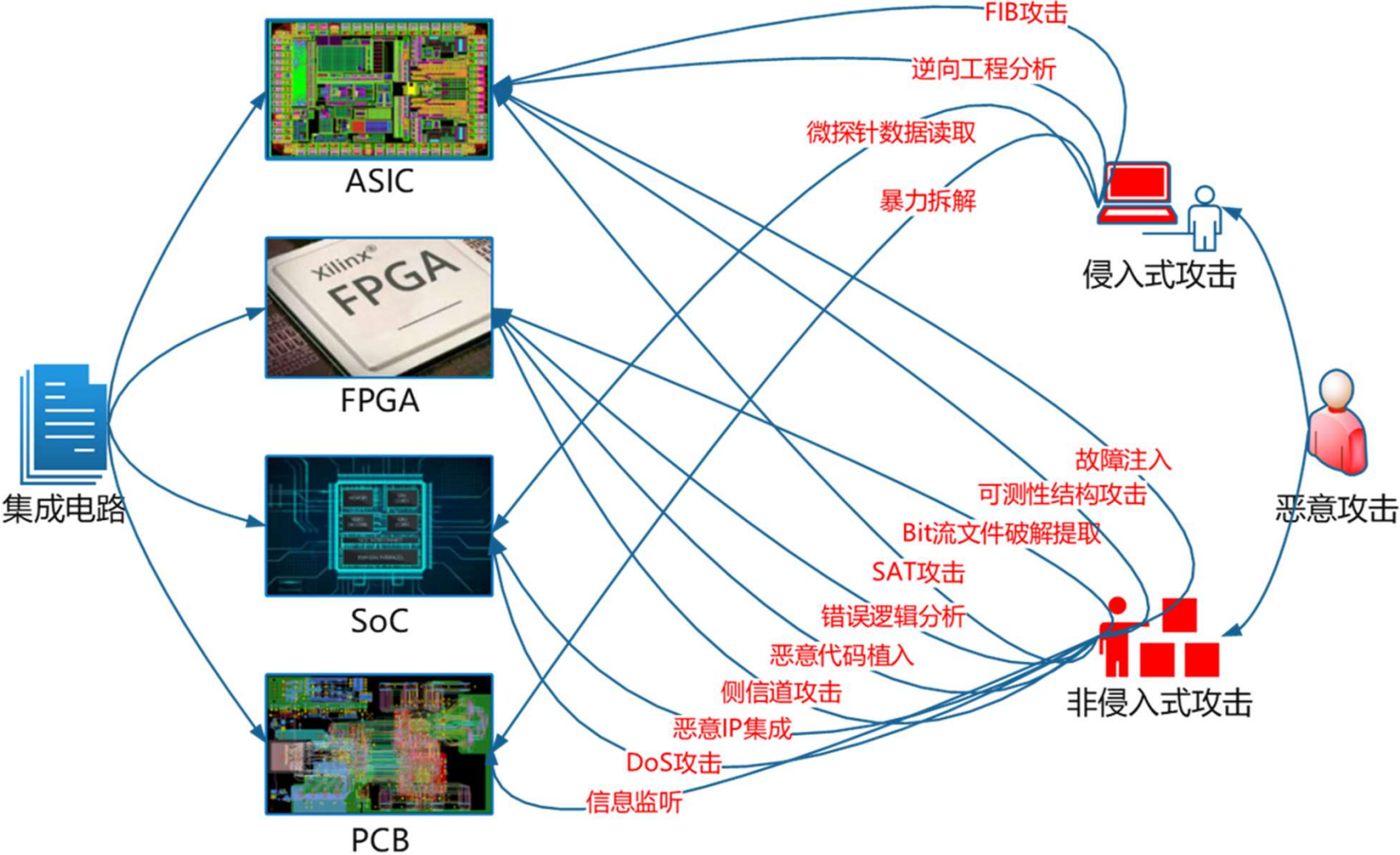


软件定义芯片：SDC

- 灵活性很高，能量效率很高（相对CPU/FPGA）
- **集成开发工具尚有关键技术需要突破，生态系统不完善**



集成电路的安全性和可靠性问题凸显



各类集成电路芯片均受到各类安全威胁

集成电路的安全性和可靠性问题凸显

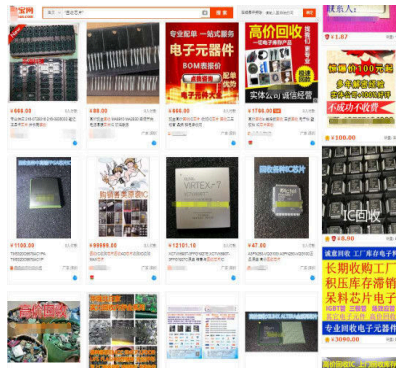


Security
Fake fuse: Bloke admits selling counterfeit chips for use in B-1 bomber, other US military gear

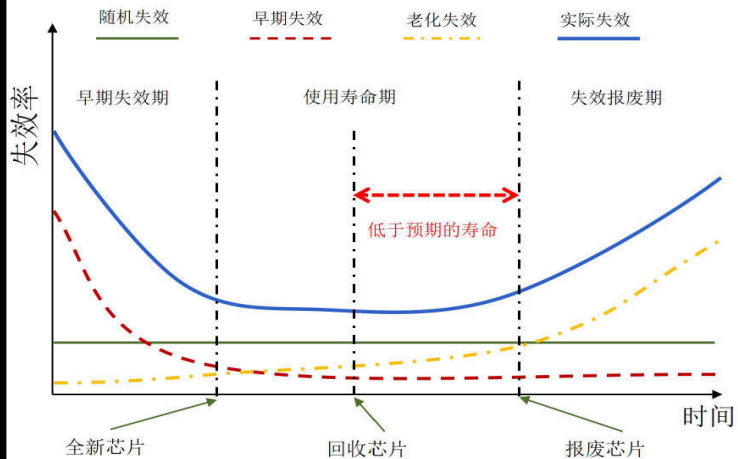
E-waste partly to blame for proliferation of deceptively marketed silicon

By Thomas Claburn in San Francisco 5 Feb 2019 at 00:58 59 SHARE

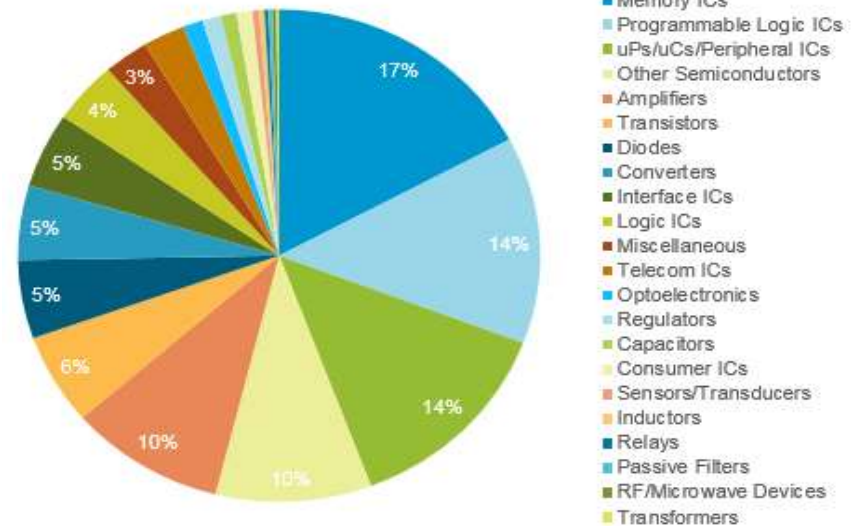
https://www.theregister.co.uk/2019/02/05/us_counterfeit_chips/



[<https://dwz.cn/elGWD6gY>]



回收芯片致使使用周期内的系统失效率大幅增加



[Source: IHS]

Composition of counterfeit IC

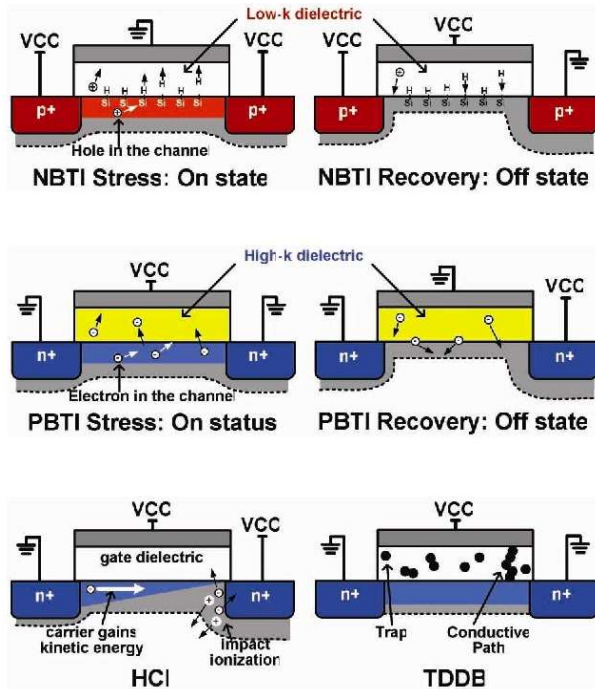
市场上80%的假冒伪劣芯片是回收芯片和标识打磨芯片 [1]

消费和工商业市场每年因为假冒伪劣芯片损失2500亿美元 [2]

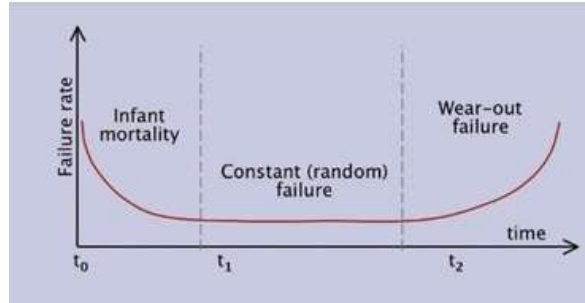
[1]. U. Guin, et al., Counterfeit Integrated Circuits: A Rising Threat in the Global Semiconductor Supply Chain, in Proceedings of the IEEE, 2014.

[2]. M. Shindell, et al., The ticking time bomb of counterfeit electronic parts. <http://www.industryweek.com/>, accessed at 2019-04-12

集成电路的安全性和可靠性问题凸显



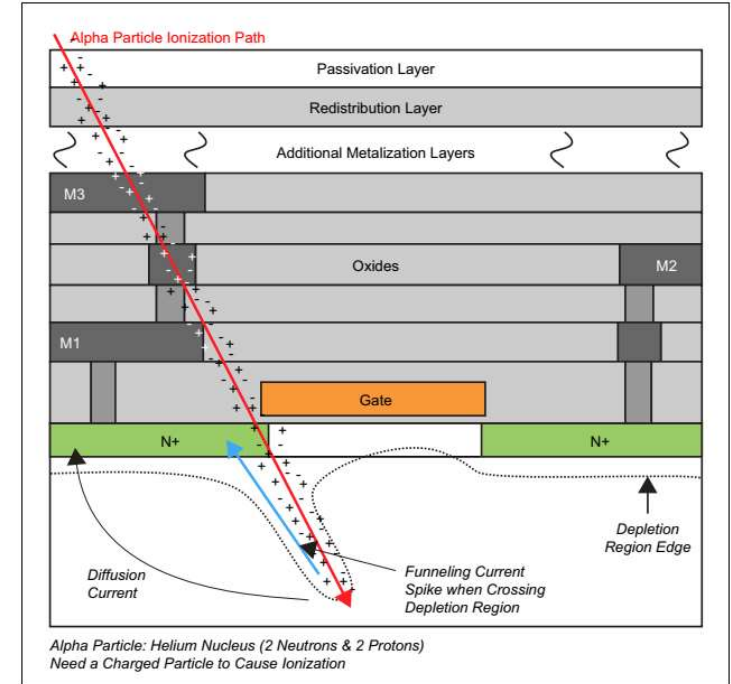
晶体管的各种物理效应¹



可靠性浴盆曲线²



嫦娥五号轨返组合体与上升器分离³



阿尔法粒子辐射效应⁴

- [1] Kim T T H, Kong Z H. Impact analysis of NBTI/PBTI on SRAM V_{MIN} ... Journal of Semiconductor Technology and Science, 2013.
 [2] <https://www.newelectronics.co.uk/electronics-blogs/how-to-improve-power-supply-mtbf-and-reliability/141531/>, 2016
 [3] <https://www.10ii.com/37212.html>, 2020.
 [4] Alter, Introduction to Single-Event Upsets, White Paper, 2013.



一、提升能量效率

二、降低芯片成本

三、增强硬件安全性、可靠性



能量效率提升技术

- 能量效率 = $\frac{\text{任务}}{\text{能耗}} = \frac{\text{任务}}{\text{功耗} \times \text{时间}} = \frac{\text{性能}}{\text{功耗}}$
- 工艺技术
 - Moore's Law $\Rightarrow$ 尺寸物理极限为单个原子直径, 约0.1nm¹
 - Dennard Scaling $\Rightarrow$ 功率密度 = $\frac{\text{功耗}}{\text{面积}}$ $\uparrow$

• 设计技术

- 性能 $\uparrow$: bandwidth or throughput, 给定时间内完成的任务量
 - 并行技术
 - 领域定制
- 功耗 $\downarrow$: 1 W=1J/s, 单位时间内消耗的能量
 - 电压技术
 - 时钟技术

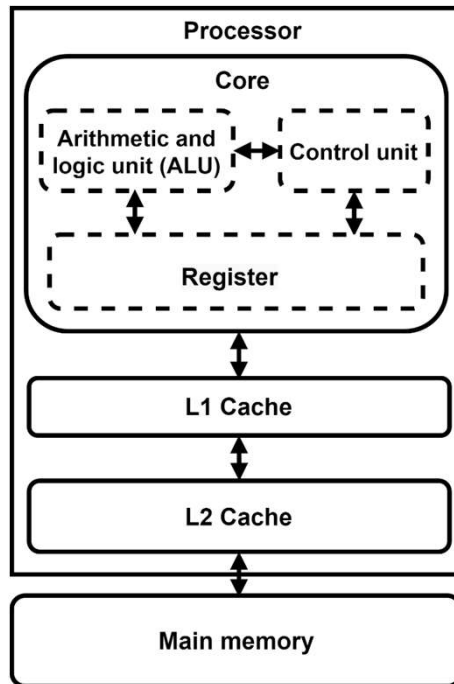
- 异构集成
- 新计算方法
 - 近存/存内计算
 - 近似计算
 - 概率计算

[1] Chen M L, Sun X, Liu H, et al. A FinFET with one atomic layer channel. Nature Communications, 2020.

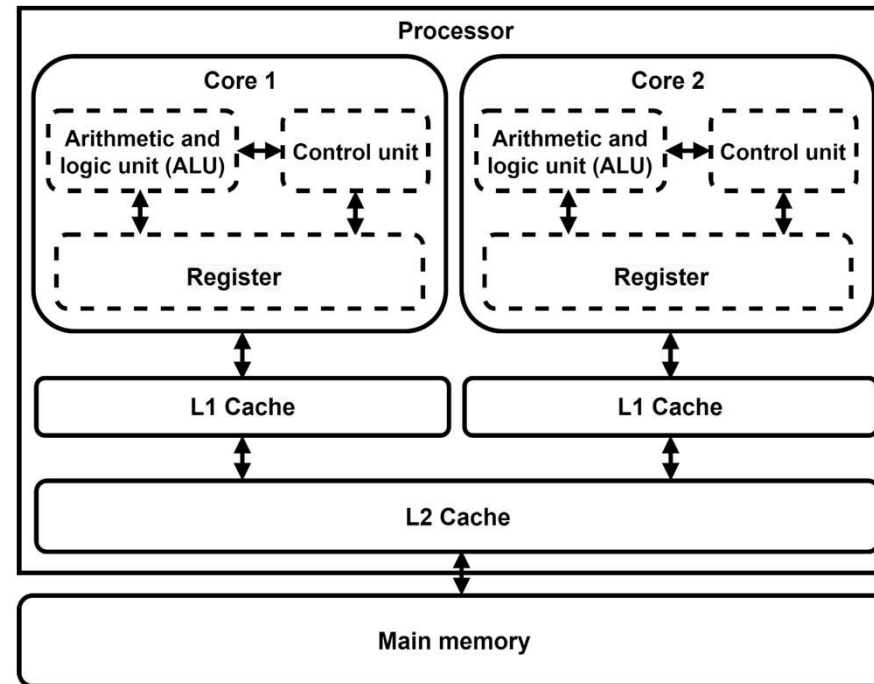


并行技术 —— 多核处理器^[1]

- 为应满足高性能需求，90年代中期多核技术萌芽，早期主要应用于服务器
- 为应对功率密度带来的芯片散热问题，2005年Intel推出了PC端双核处理器



单核处理器结构



共享内存多核处理器

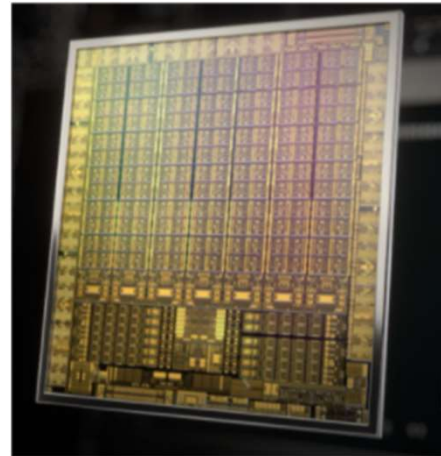
[1]Hennessy J L, Patterson D A. Computer Architecture: A Quantitative Approach, 6th edition, 2017.

并行技术 —— 多核处理器

- CPU核：64个
- 线程数：128
- 基准时钟频率：2.9GHz
- 单核最高时钟频率：4.3GHz



AMD Ryzen™ Threadripper™ 3990X¹



NVIDIA RTX 3090²

- CUDA核：10496个
- 基准时钟频率：1.4GHz
- 最高时钟频率：1.7GHz

- 编程复杂度提高，性能受程序质量的约束
- 指令集的约束
- 在执行线性或顺序任务时，多核比单核处理器需要更长的等待时间



- **优化编程算法及软件操作系统**
- **优化核心间互连结构和通信协议**
- **增加领域定制的核心**

[1] <https://www.amd.com/en/products/cpu/amd-ryzen-threadripper-3990x>, 2020.

[2] <https://www.nvidia.com/en-us/geforce/graphics-cards/30-series/rtx-3090>, 2020.

[3] Hennessy J L, Patterson D A. Computer Architecture: A Quantitative Approach, 6th edition, 2017.

领域定制



- 多核技术

- Amdahl's Law: 当10%的任务为串行时, 多核技术最高可提升10倍的性能, 堆加更多的核数不会进一步提升性能。



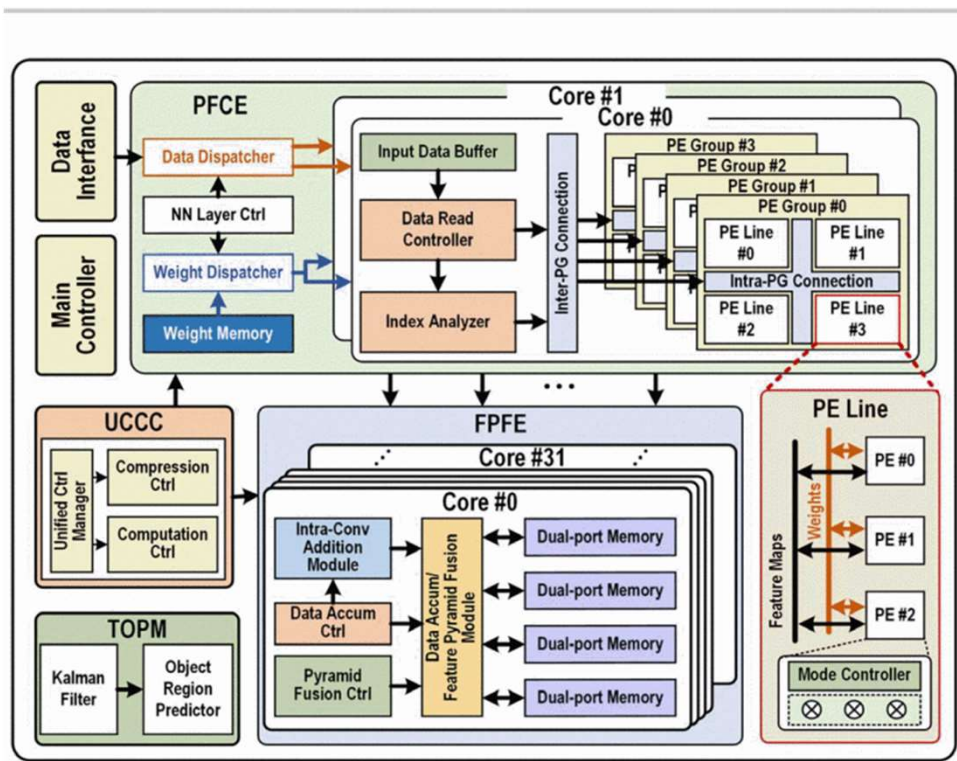
- 领域定制

- 缩短数据搬移距离, 使用专用内存
- 根据应用领域, 增强计算单元或内存的设计
- 根据应用调整数据大小和格式
- 最大可能使用并行化
- 使用面向应用的最佳编程方法

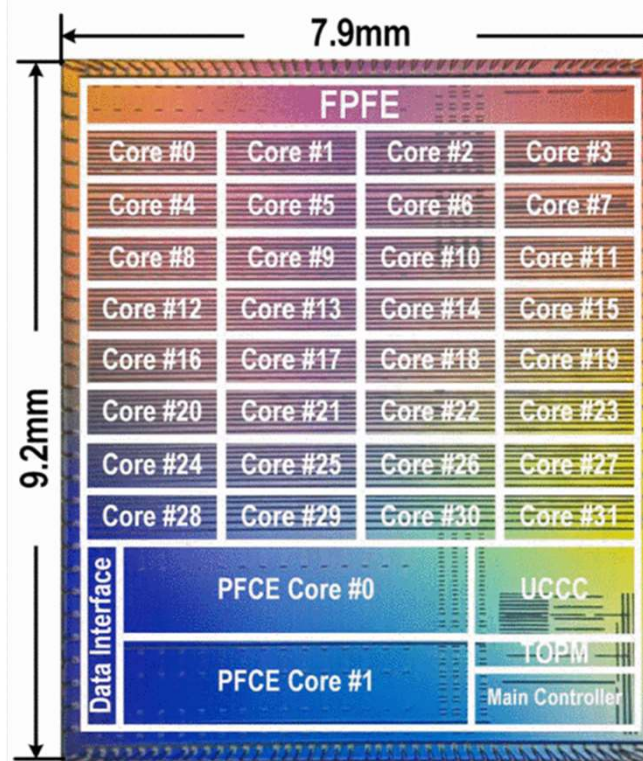
领域定制



高效领域定制深度学习视觉对象处理单元（DL-VOPU）



DL-VOPU设计结构¹



版图

	Specifications
Technology	40 nm
Die Area	72.68 mm ²
Data Precision	INT8
SRAM Size	2 MB
Supply Voltage	1.1 V
Frequency	225 MHz
Peak Performance	2226.43 GOPS
Power	92.45 mW
Energy Efficiency	24.08 TOPS/W

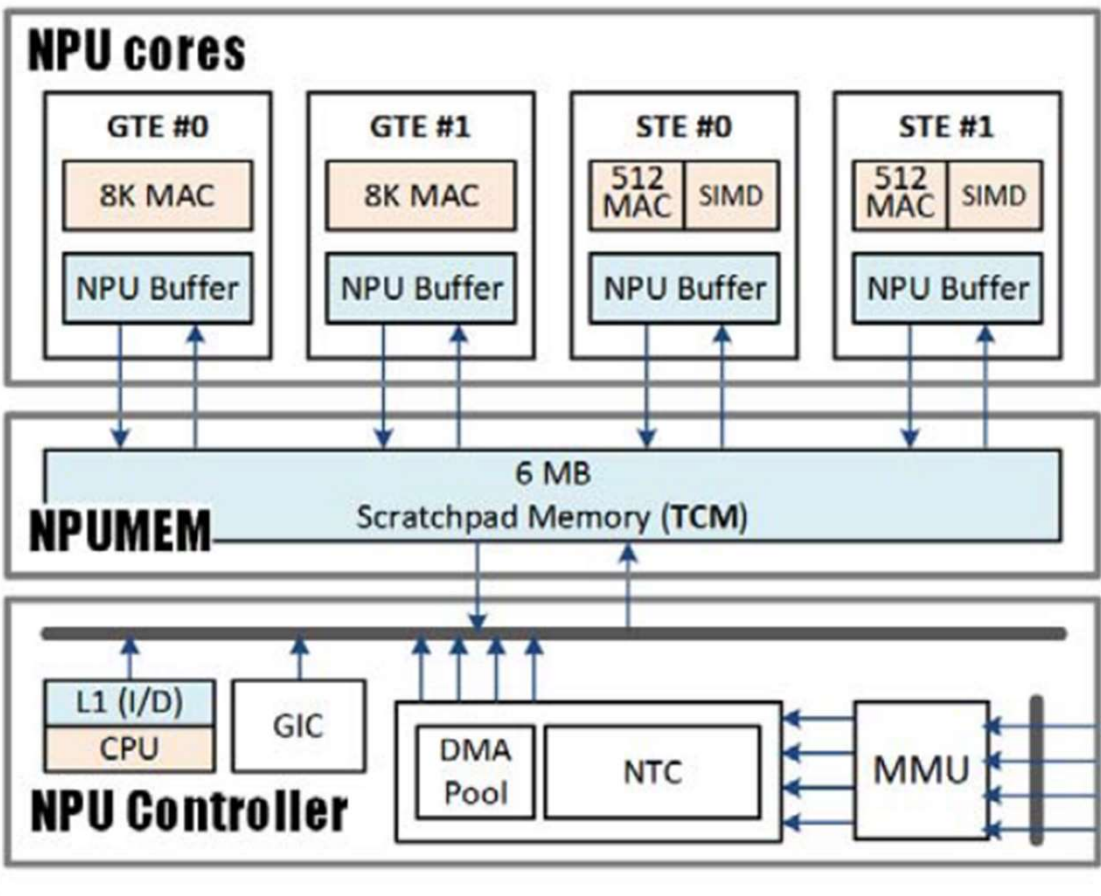
参数列表

[1]. Gong, T. Zhang, et al., "22.7 dl-vopu: An energy-efficient domain-specific deep-learning-based visual object processing unit supporting multi-scale semantic feature extraction for mobile object detection/tracking applications," in ISSCC, 2023

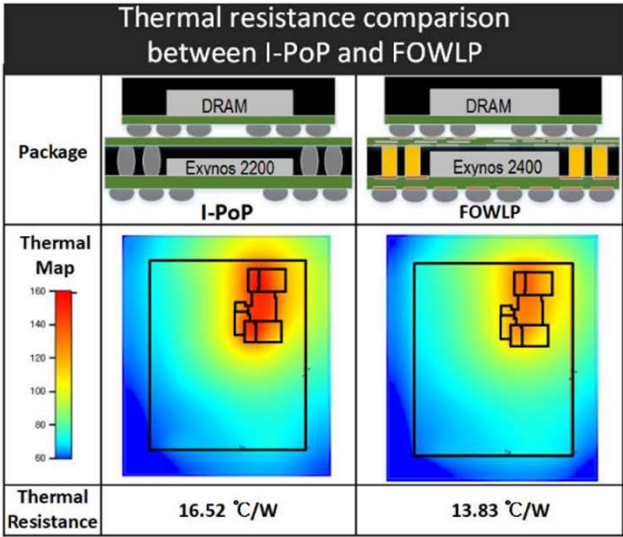
基于Wafer-level封装的异构端侧加速器



- 三星Exynos 2400: GTE(计算密集), STE(更高访存), SIMD(负责非线性) 异构架构



芯片架构设计



Wafer-level的封装

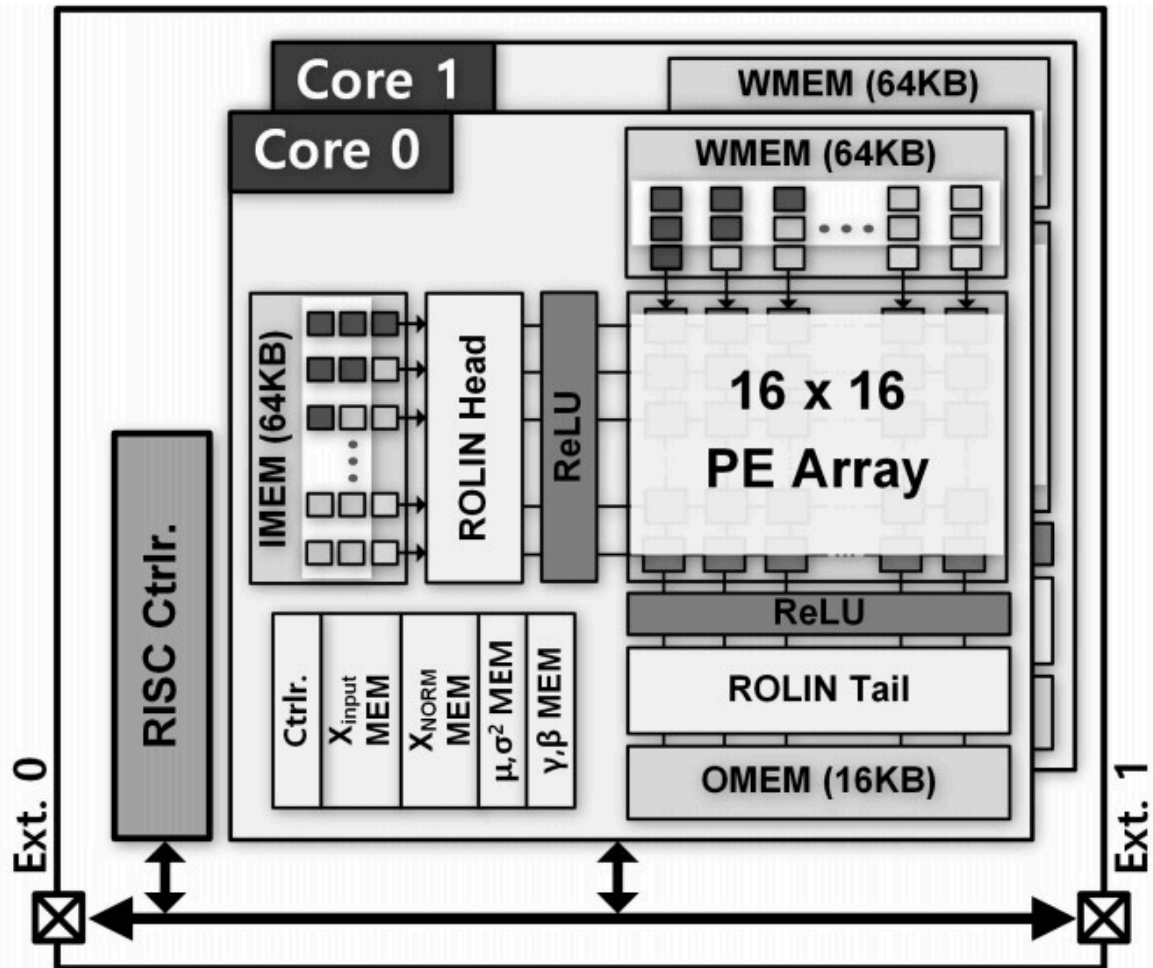
	This WORK	ISSCC 2022 [11]	ISSCC 2021 [12]	ISSCC 2021 [13]	ISSCC 2020 [14]
Process (nm)	4	4	5	7	7
Area (mm ²)	12	4.74	5.46	19.6	3.04
Supply Voltage (V)	0.55 - 0.83	0.55 - 1.0	0.55 - 0.9	0.55 - 0.75	0.575 - 0.825
Working Frequency (MHz)	533 - 1196	332 - 1196	332 - 1196	1,000 - 1,600	290 - 880
On-Chip Data Memory (kB)	6,144 (NPUMEM)	2,048	3,072	8,192	2,176
Bit Precision	INT 8, 16 / FP16	INT 4, 8, 16 / FP16	8, 16	INT4, 2 / FP8, 16, 32	8, 16
Multiplier Number	17,408 (INT8)	8,192 (INT8)	6,144 (= 2,048 / core)	-	-
Peak Performance (TOPS)	41.8 (8b) 20.9 TFLOPS (FP16)	39.3 (4b), 19.7 (8b) 9.8 TFLOPS (FP16)	14.7 (8b) @ no Skip 29.4 (8b) @ max Skip	39.3 (4b), 19.7 (8b) 9.8 TFLOPS (FP16)	3.6 (8b)
Power (mW)	730 - 6177	MobileNetTPU (INT8): 381-5114 DeepLabV3 (FP16): 393-5133	327 @ 0.6V 794 @ 0.9V	-	173 @ 0.575V 1053 @ 0.825V
Inferences/second	MobileNetTPU (INT8): 6246.8 @ 0.83V	MobileNetTPU (INT8): 3433 @ 1.0V DeepLabV3 (FP16): 211 @ 1.0V	InceptionV3: 622.7 @ 0.9V	-	-
Area-efficiency (Peak TOPS/mm ²)	INT8: 3.48 FP16: 1.74	INT4: 8.90 INT8: 3.45 FP16: 1.72	2.69	INT4: 5.33, INT8: 1.33 FP16: 0.68	1.184

能效对比

[1] Jun-Seok Park, et al., "An On-Device Generative AI Focused Neural Processing Unit in 4nm Flagship Mobile SoC with Fan-Out Wafer-Level Package," ISSCC, 2025.

领域定制

生成对抗网络（GAN）训练加速器



- 1 RISC控制器
- 2 训练核
- $16 \times 16 \times 2$ 脉动PE阵列
- ReLU激活函数模块
- ROLIN：实例规范化层重排序模块
- 能效：0.38-TFLOPs/W
- 32TB HBM

• 灵活性较差
• 应用领域受限且成本高



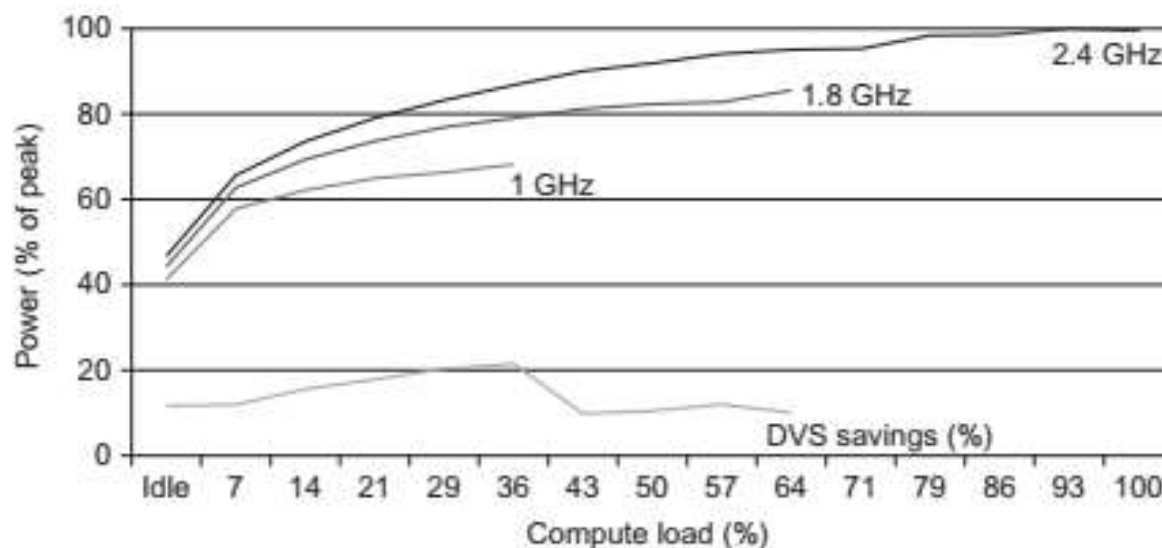
• 引入SDC技术
• 优化整体架构

[1] S. Kim, et al., "An energy-efficient GAN accelerator with on chip training for domain-specific optimization," in JSSC, 2021.

功耗降低



- 功率密度 $\Rightarrow$ **散热**
 - 风冷、水冷、热管制冷、半导体制冷、压缩机制冷、液氮制冷
- 移动终端应用 $\Rightarrow$ **电池使用时间**
- 宽电压、近/亚阈值 $\Rightarrow$ 性能 $\downarrow$
可靠性 $\downarrow$
- 门控电压/时钟 $\Rightarrow$ 动态功耗 $\downarrow$
- 动态可调电压和频率 (DVFS)

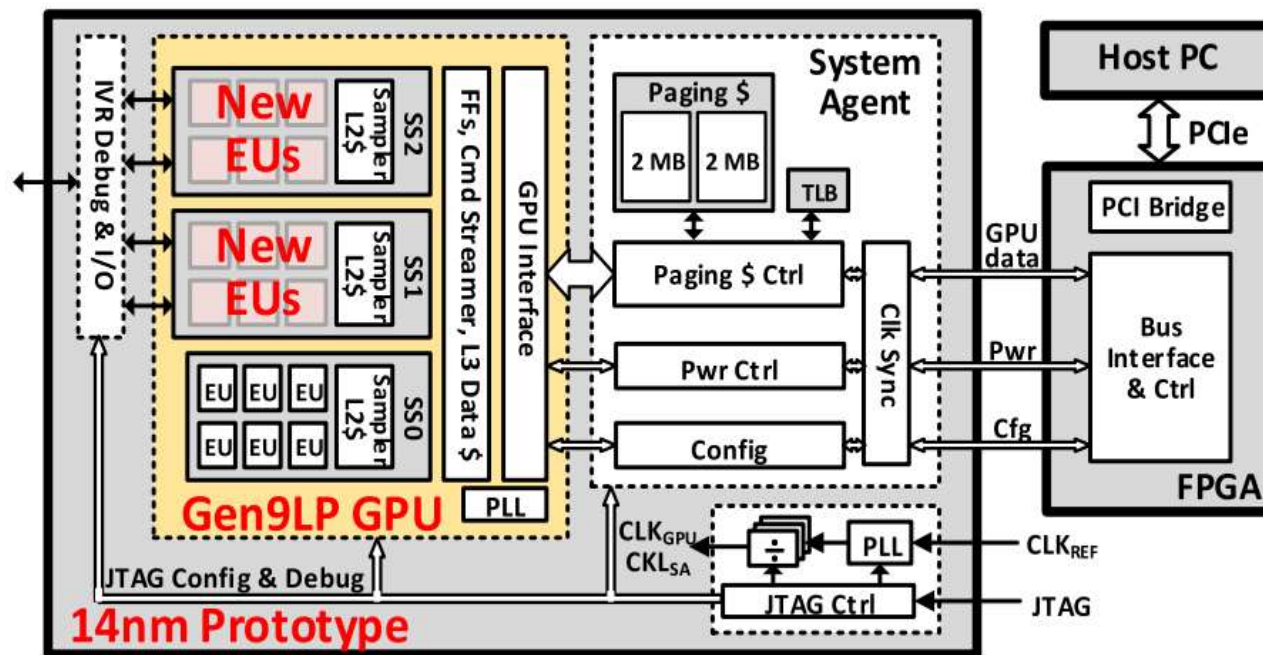


DVFS技术在峰值功耗上的表现¹

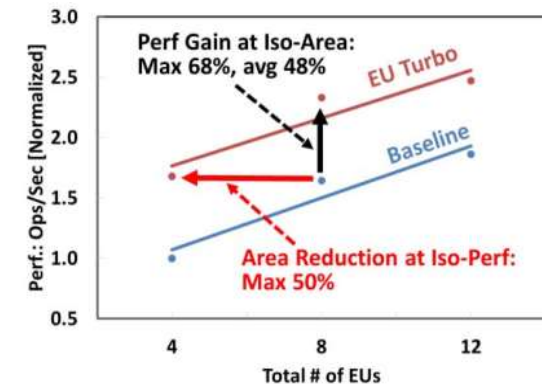
[1] Hennessy J L, Patterson D A. Computer Architecture: A Quantitative Approach, 6th edition, 2017.

基于细粒度DVFS的GPU

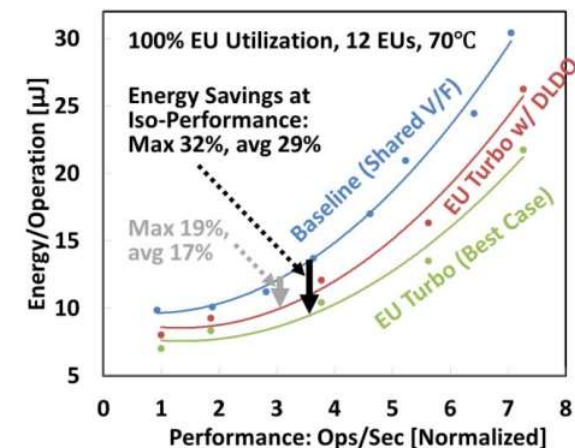
- Intel Gen9LP GPU
- 为每一个执行单元 (EU) 配置了一个电压调节器
- 0.51V-1.20V, 50MHz-400MHz



基于细粒度DVFS的GPU结构



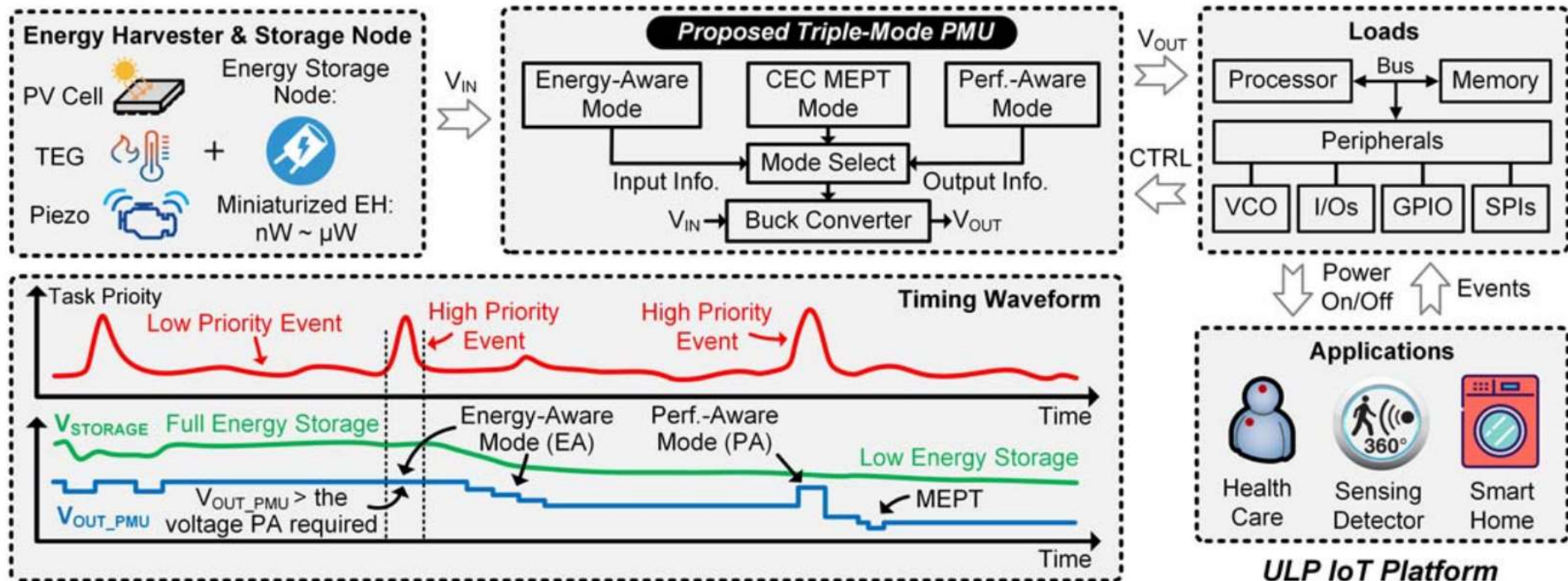
性能对比



能效对比

[1]P. A. Meinerzhagen et al., "An Energy-Efficient Graphics Processor in 14-nm Tri-Gate CMOS Featuring Integrated Voltage Regulators for Fine-Grain DVFS, Retentive Sleep, and VIN Optimization," JSSC, 2019.

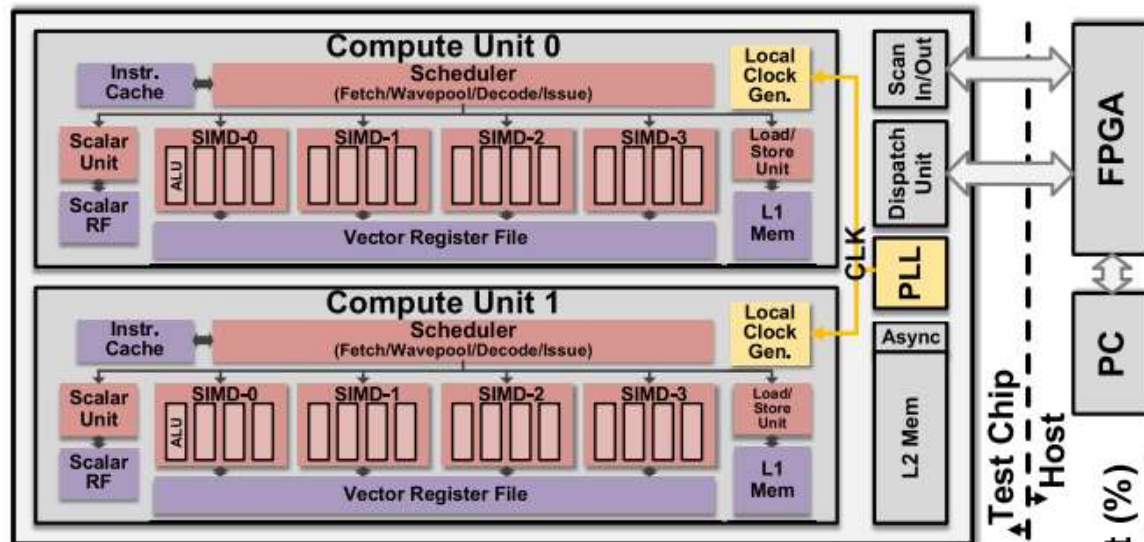
自供电边缘端设备的DVFS功耗管理



- 自体供电片上系统 (self-powered system-on-chip)
- 通过事件驱动的DVFS实现最小能耗点的动态追踪
- 194nW SoC, 5.4nW 功耗管理单元, 10^4 动态调控范围

[1] X.Liu et al., "A 194nW Energy-Performance-Aware IoT SoC Employing a 5.2nW 92.6% Peak Efficiency Power Management Unit for System Performance Scaling, Fast DVFS and Energy Minimization," ISSCC, 2022.

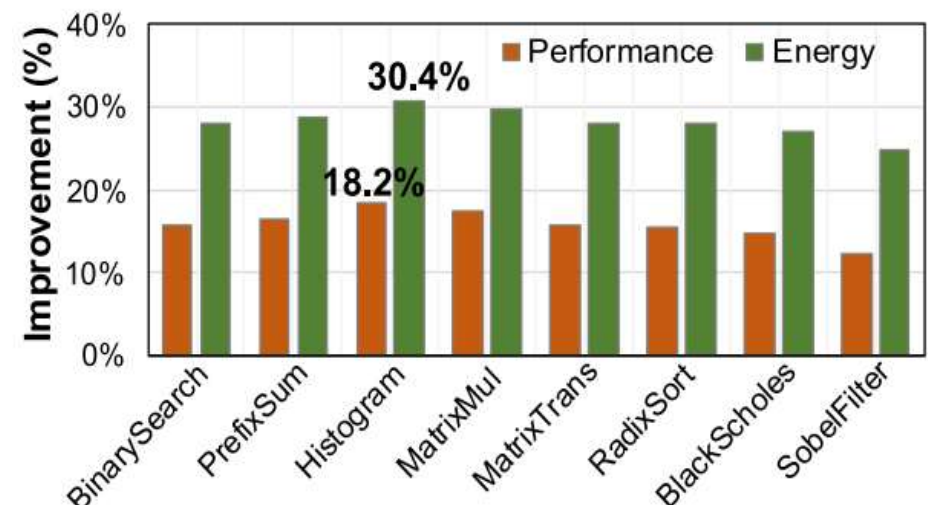
A GPGPU with an Adaptive Clock



基于自适应时钟的GPGPU结构

- 性能提升18%
- 能耗降低30%

- 基于指令的自适应时钟系统
- 动态时序松弛
- 关键路径监测技术
- 全局PLL+本地PLL



基于自适应时钟的GPGPU与传统设计性能对比



DVFS的局限性

- 增加额外电路，芯片面积增大
- DVFS需要实时响应中断
 - 任务执行时间增加
 - 对PLL系统的稳定性要求较高，降低了系统的可靠性

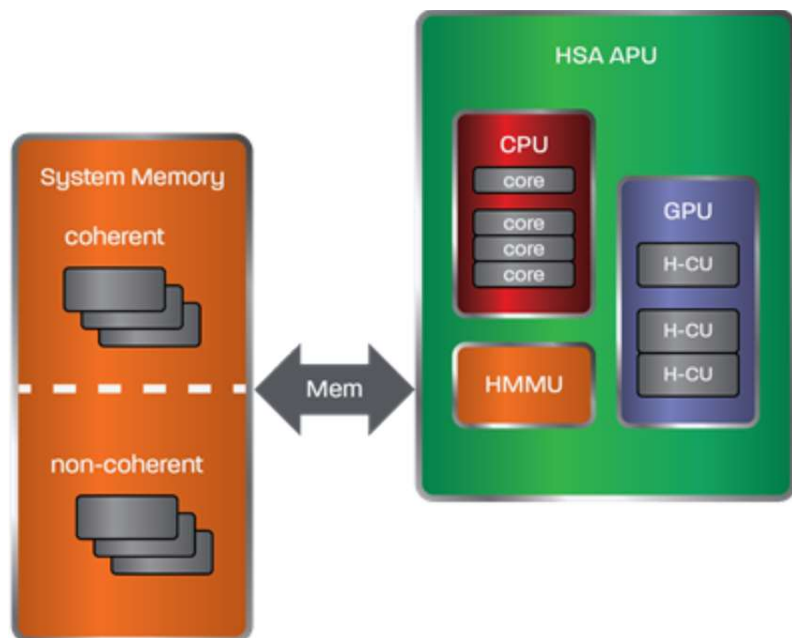


- **防止过度的宽电压或频率，平衡开销与收益**
- **针对具体应用使用DVFS技术，优化时钟系统的稳定性及实时响应能力**

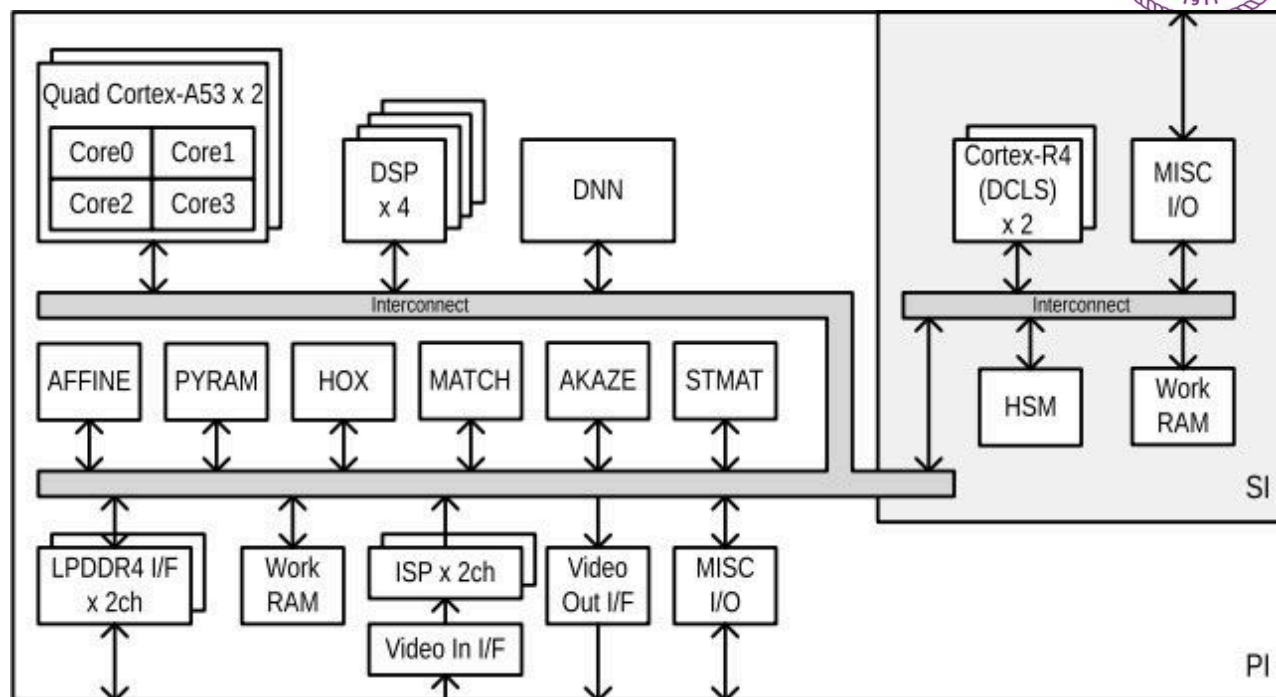
异构集成¹



- 结构特点
 - 多种指令集
 - 共享内存
 - 定制设计
- 硬件特性
 - 提高能效
 - 降低成本



CPU+GPU异构系统结构



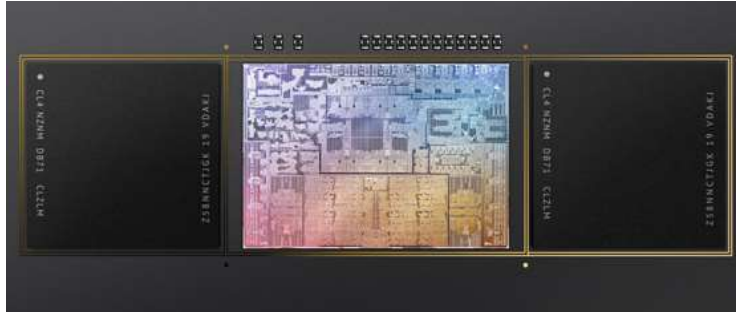
驾驶员辅助系统SoC²

- 10个处理器、4个DSP、8种加速器
- 工艺：16nm
- 峰值性能：217GOPs、能效：2.1TOPs/W

[1] George Kyriazis. Heterogeneous System Architecture: A Technical Review, AMD, 2012.

[2] Y. Yamada et al., "A 20.5 TOPS Multicore SoC With DNN Accelerator and Image Signal Processor for Automotive Applications," JSSC, 2020

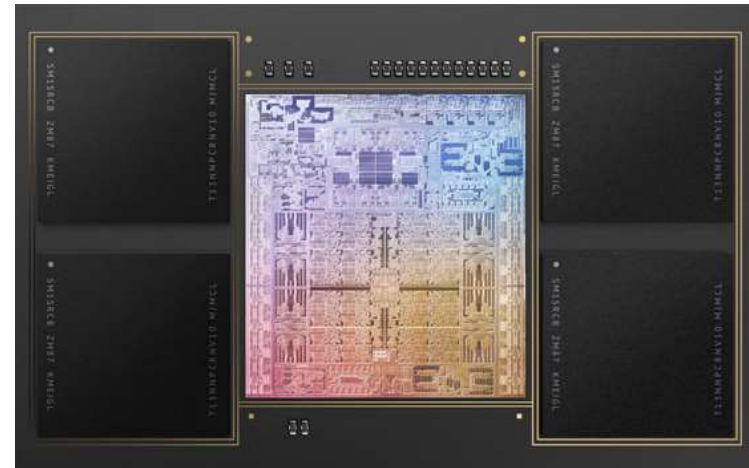
M1 Pro & M1 Max



M1 Pro (337亿晶体管, TSMC 5nm)

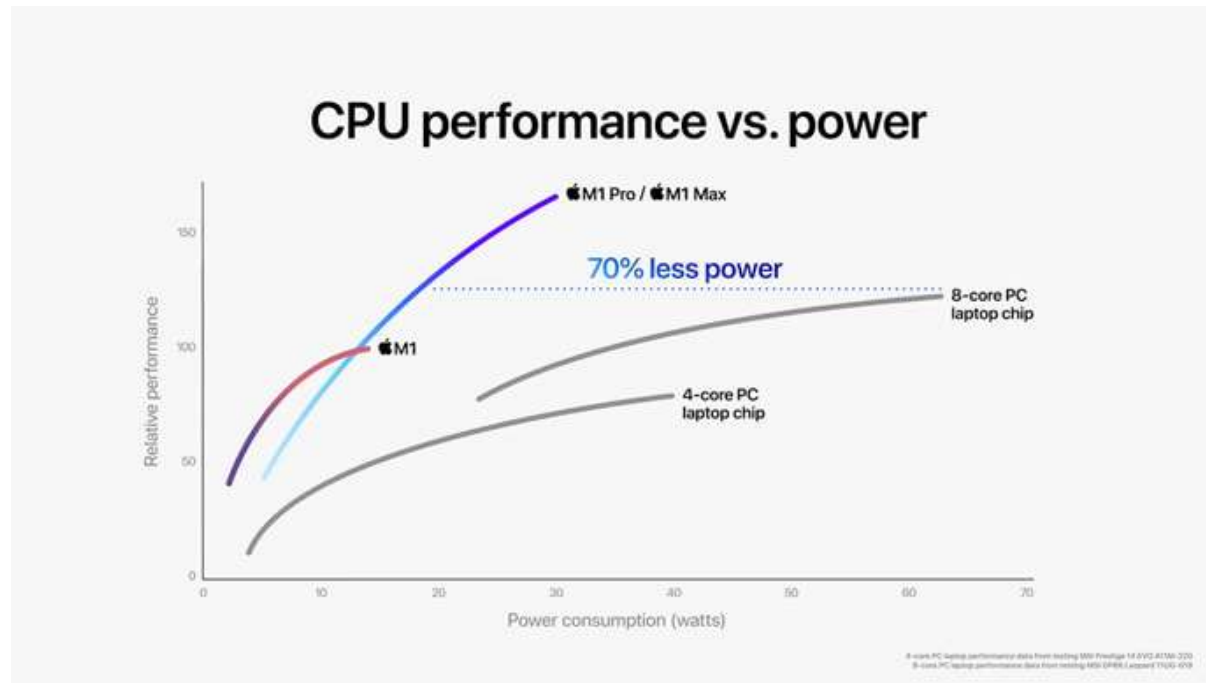
- 最高CPU核心数量: 10
- 最高GPU核心数量: 16
- 最高统一内存: 32GB
- 最高内存带宽: 200GB/s

- 最高CPU核心数量: 10
- 最高GPU核心数量: 32
- 最高统一内存: 64GB
- 最高内存带宽: 400GB/s



M1 Max (570亿晶体管, TSMC 5nm)

M1 Pro & M1 Max



- 相同功耗下，M1 Pro & M1 Max的速度是Intel最新8核处理器Coffee Lake i9 的1.7到2倍
- 当以i9峰值性能运行时，功耗减少了70%

异构集成的局限性



- 不同架构使用不同指令集，增加了编程难度
- 现有软件移植到新架构上的问题

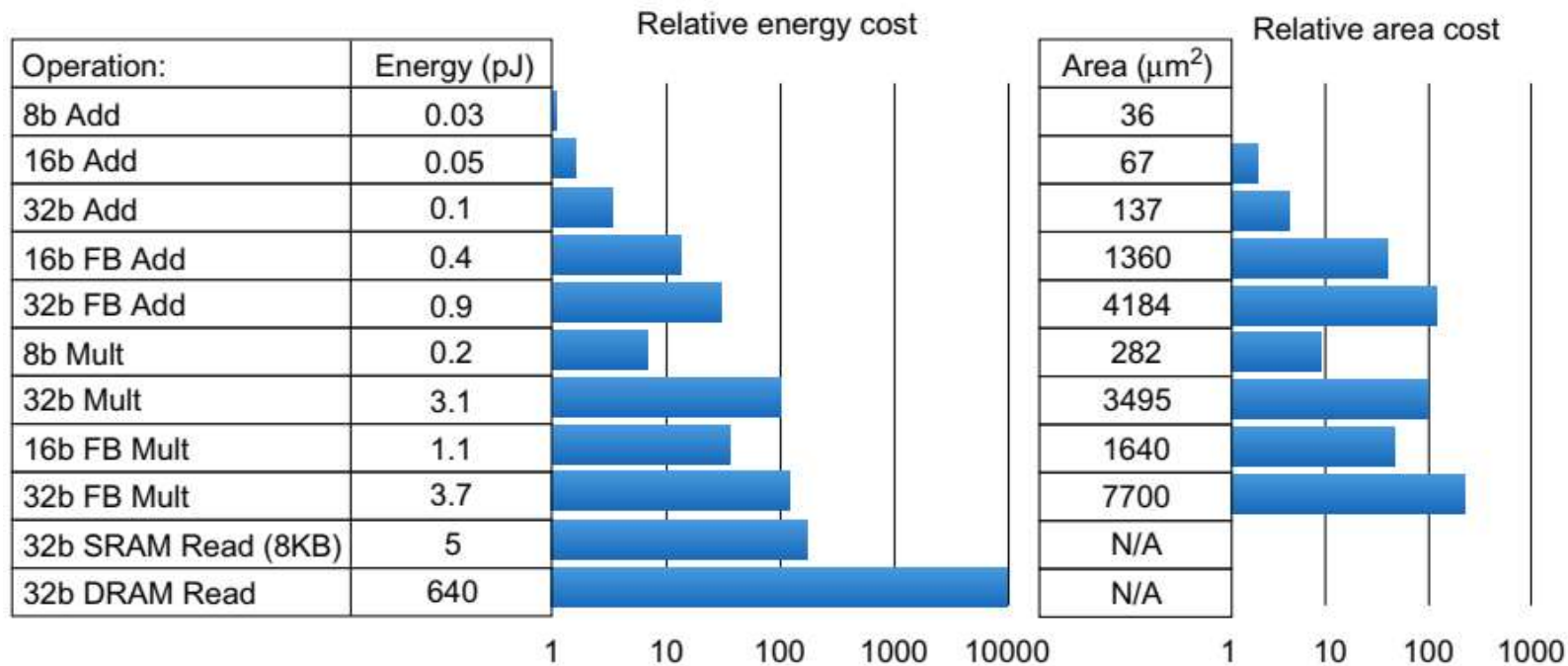


- **制定统一编程标准，建立高层次抽象的编程环境**
- **为现有软件建立新的库函数，以适应新的架构**

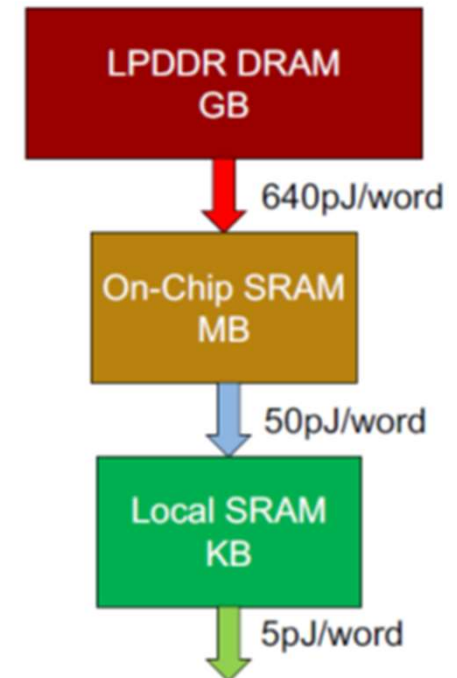


近存/存内计算：数据搬移带来的问题

- 算法向内存优化、减少全局内存访问、本地内存占用空间不宜过大

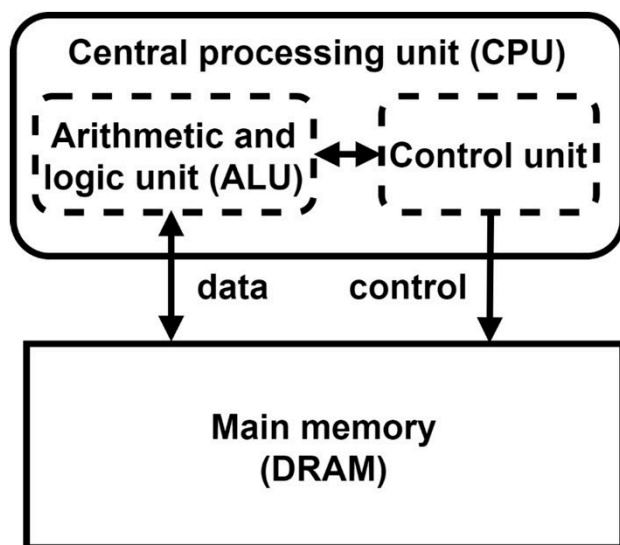


数据搬移与计算的能耗、面积对比

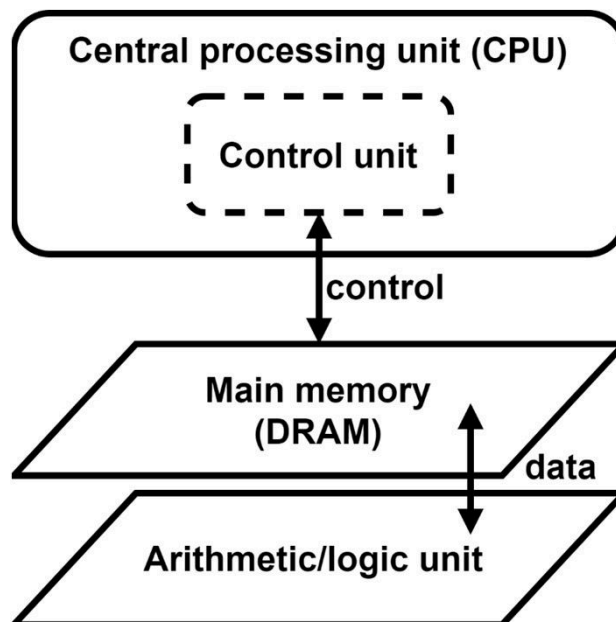


数据搬移能耗

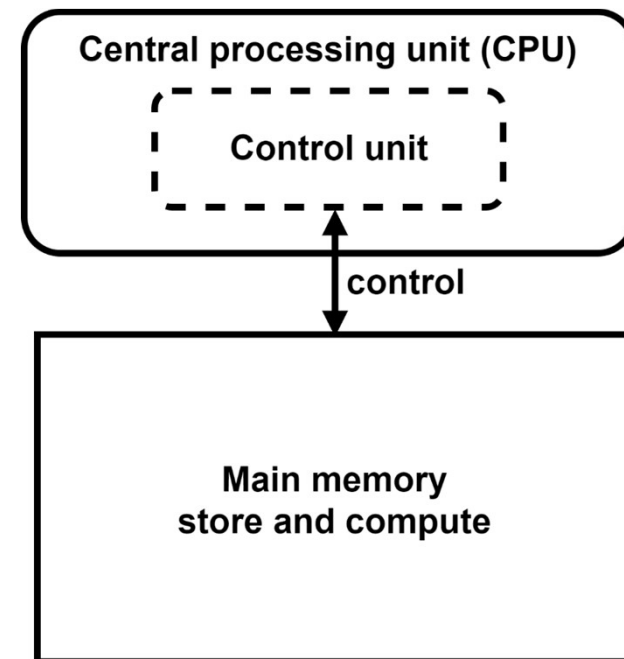
近存/存内计算：数据搬移带来的问题



传统冯·诺依曼架构



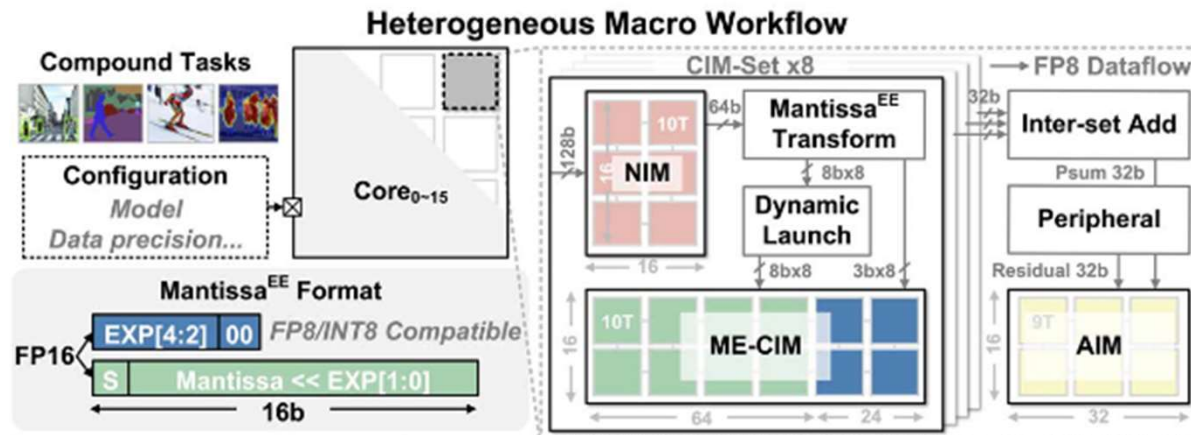
基于3D集成的近存计算架构



存内计算基本架构

基于SRAM的存内稀疏端侧芯片

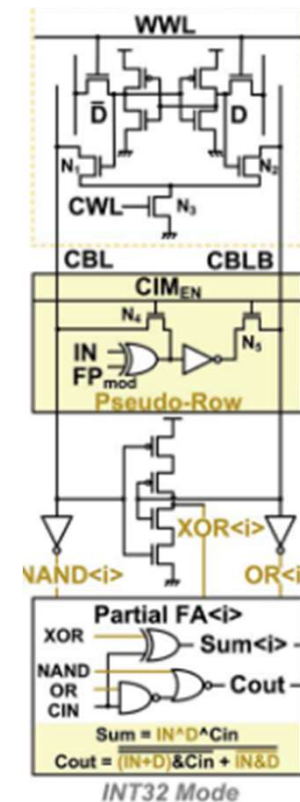
- 异构浮点计算单元 + 全通路存内设计 + 动态适应稀疏模式



芯片架构设计

	ISSCC2024[5]	ISSCC2024[6]	ISSCC2024[7]	ISSCC2023[8]	ISSCC2023[9]	ISSCC2023[10]	This Work
Technology	16nm	28nm	3nm	22nm	28nm	28nm	28nm
CIM Function	MAC	MAC	MAC	MAC	MAC	MAC	MAC/Route/Accumulate
Voltage (V)	0.4~0.8	0.7~0.95	0.525~1.0	0.6~0.8	0.6~0.9	0.397~0.9	0.62~0.9
Frequency (MHz)	250	208	600~1800	156	147	10~400	100~525
Macro Size	96Kb	192Kb	156KB	832Kb	64Kb	16Kb	224Kb
Die Area (mm ²)	8	1.94	1.37	18	0.146	4.536	2.67
IN/W Precision	BF16	BF16	INT12	BF16	BF16	FP16	FP16/FP8/INT8
Output Precision	FP32	8b ADC	31b	FP32	BF16	21b	FP32/INT32
Full Precision	Yes	No	Yes	Yes	No	No	Yes
Energy (TFLOPs/W) ^{*1}	33.2~45.4	13.55~32.78	16.3~23.2	16.22~17.59	14.04~31.6	3.2~16.9	15.1~51.6 (MAC) ^{*2} 4.8~16.2 (Chip) ^{*3}
Area (TFLOPs/mm ²) ^{*1}	2.63	2.21	12	0.069	2.05	0.36~2.1 (INT4)	0.46~2.44

能效对比



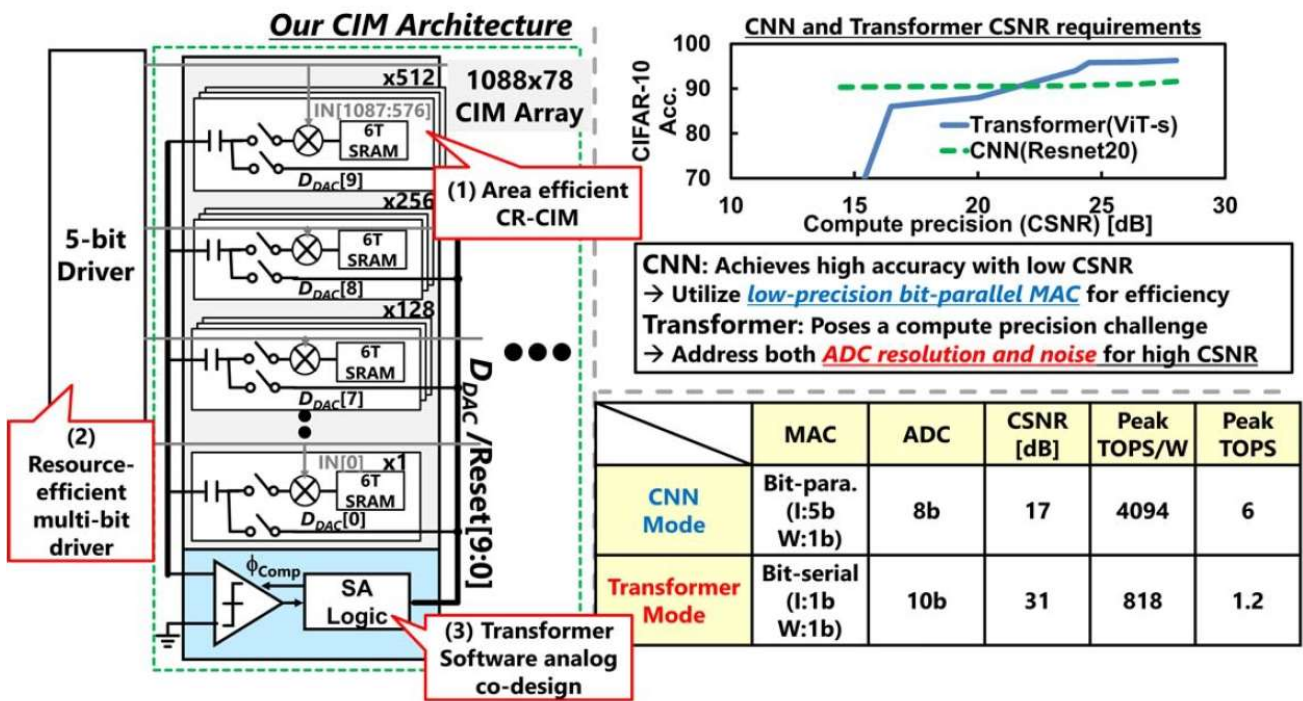
CIM计算单元

[1] Zhiheng Yue, et al., "A 51.6TFLOPs/W Full-Datapath CIM Macro Approaching Sparsity Bound and <2⁻³⁰ Loss for Compound AI," ISSCC, 2025.

基于SRAM存内计算(CIM)的CNN/Transformer加速器



- 使用SRAM阵列做计算（同时也是模拟计算/近似计算）



芯片平面图

	This work	[3] JSSC 2020	[4] ISSCC 2023	[5] VLSI 2021	[6] JSSC 2023
CIM type	Charge (MAC)	Charge	Charge	Charge	Charge
Process	65nm	65nm	12nm	28nm	28nm
Supply Voltage[V]	0.6-1.1	0.85-1.2	0.5/0.85	0.6-0.9	0.8
Array Size	10KB	72KB	128KB	36KB	16KB
Bit Precision	4-8b	1-8b	1-8b	1-5b	1-16b
Application	CNN	Transformer	CNN	CNN	CNN
Peak TOPS Normalized to 1-b	6	1.2	2.1	6.4	6.1
Peak TOPS/mm² Normalized to 1-b	12.5	2.5	0.6	N.A.	12
Peak TOPS/W Norm. to 1-b	4094*	818*	400	4534	5796
Peak TOPS/W Norm. to 65nm**	4094	818	400	837	2496
ADC bit	8	10	8	8	8
SQNR [dB]	26.7	45.3	22	N.A.	17.5
CSNR [dB]	16.8	31.3	17	N.A.	10.5
CIFAR-10 Accuracy	91.7	95.8	92.4	N.A.	91.1

能效对比

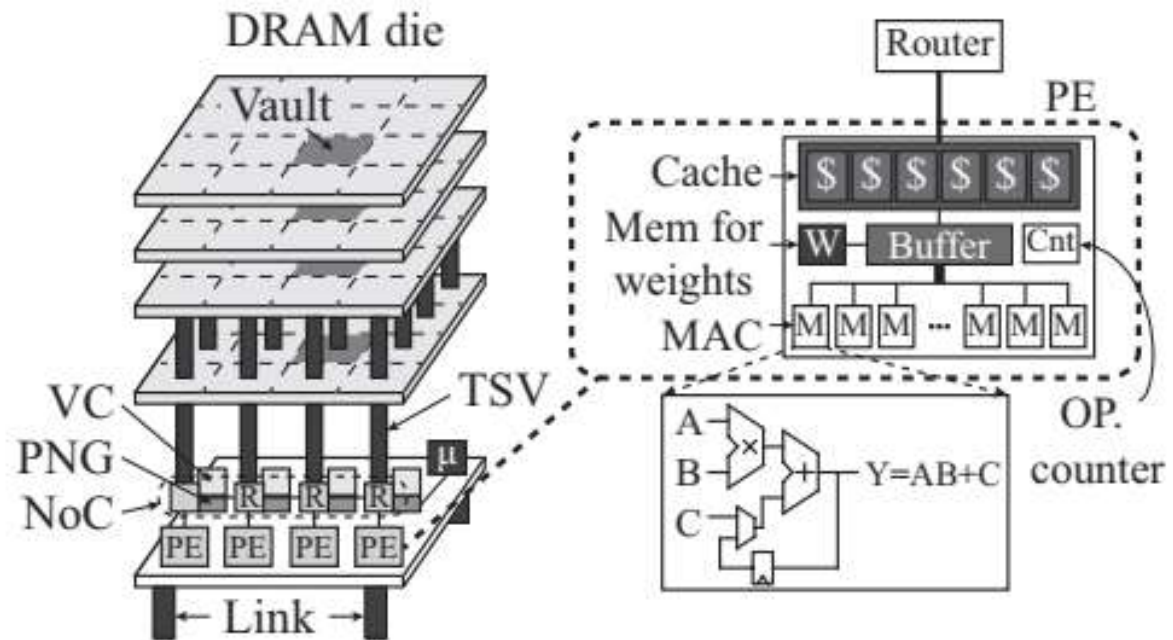
电路结构设计（左），设计动机与设计目标（右）

[1]K. Yoshioka, "A 818-4094TOPS/W Capacitor-Reconfigured CIM Macro for Unified Acceleration of CNNs and Transformers," ISSCC, 2024.

[2]H. Valavi, "A 64-Tile 2.4-Mb In-Memory-Computing CNN Accelerator Employing Charge-Domain Compute," JSSC, 2019.

基于DRAM的近存计算

- 工艺: 15nm FinFet
- 工作频率: 5GHz
- 性能: 132.4 GOPs
- 能效: 38.82GOPs/W



基于SRAM的近存计算3D集成结构

BANK层级的存内计算设计

Bank-Level In Memory Processors (BLIMP)
and with vector engines (BLIMP-V)

识别阶段:

通过静态分析源代码来确定哪些区域可以被卸载到PIM引擎上

分离阶段:

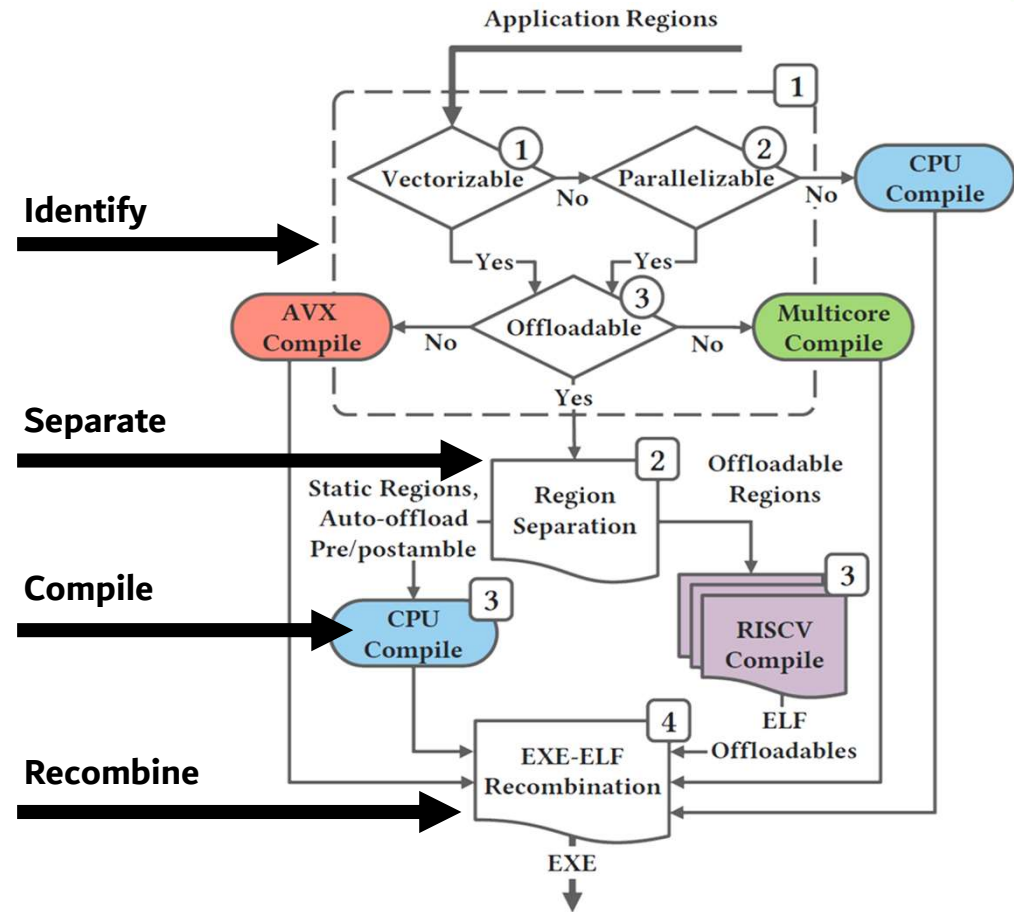
将可以被卸载的区域与主机CPU上的代码分离

编译阶段:

对于已经分离的区域，使用向量化或并行化技术进行优化，并生成可在PIM引擎上运行的代码

重组阶段:

将优化后的代码与主机CPU上的代码重新组合



BLIMP-V编译流程



近存 / 存内计算的局限性

- 对应用场景要求严格，只对大规模的、有规律的并行计算效率较高
- 往往需要改变现有存储器的结构、增加硬件消耗
- 计算结果一般依赖于后端模拟信号的处理、一般需要模数转换或感应放大器



- **适合应用于领域定制芯片中**

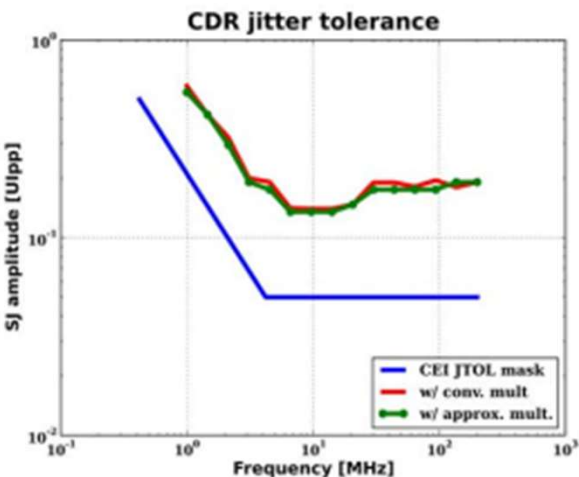
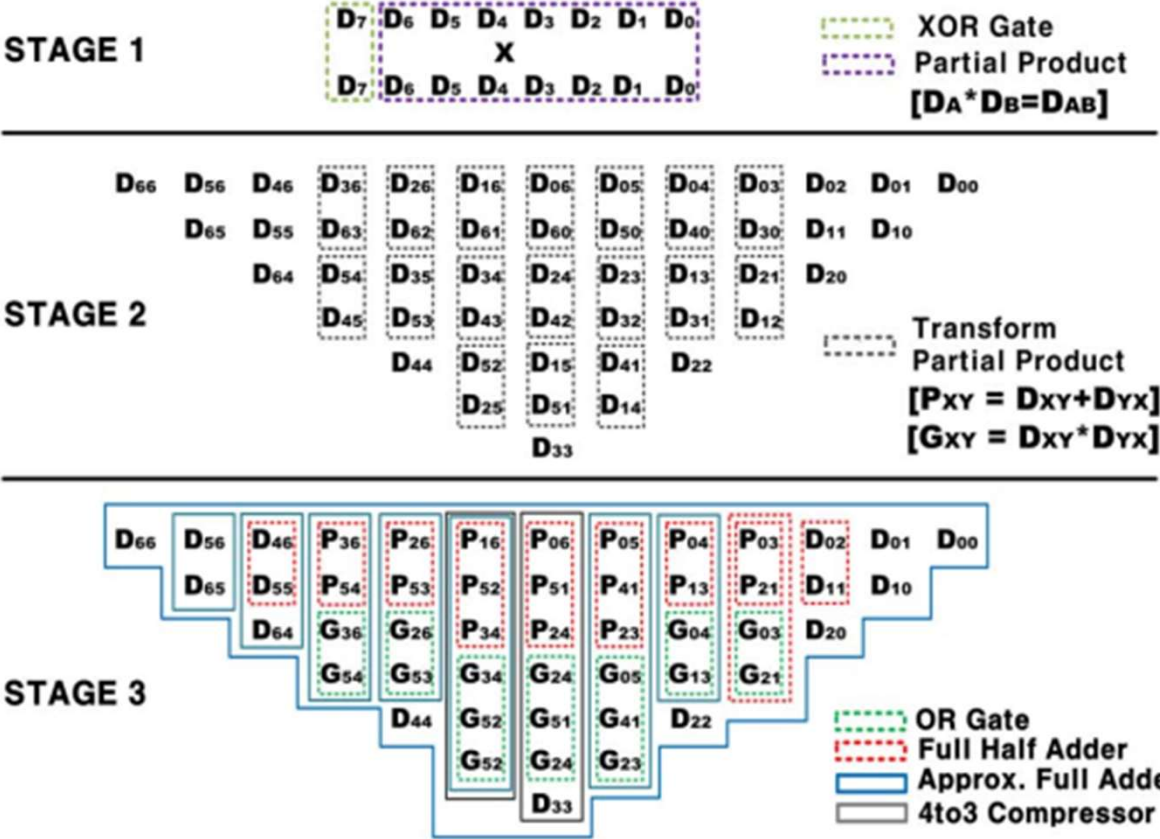
近似计算

- 利用人类感知的局限性、应用或算法的容错性
- 损失少许精度，换取性能、能效的提升
- 设计方法
 - 电路级
 - 对准确计算电路进行删减、修改、简化
 - 在电路综合过程中，移除部分逻辑单元
 - 算法级
 - 利用数学原理，对复杂计算操作的算法进行简化
 - 在不同的数据表示形式下进行电路设计，在转化过程中进行近似





有线收发器中近似乘法器的设计



Architecture Example for EQ	Area (μm^2)	Power (mW)
Accurate Multiplier	42454	85.364
Approximate Multiplier	28669	50.868
Improvement	67.5%	59.6%

8x8近似乘法器的实现与电路测量结构

[1]B. Yoo et al., "6.4 A 56Gb/s 7.7mW/Gb/s PAM-4 Wireline Transceiver in 10nm FinFET Using MM-CDR-Based ADC Timing Skew Control and Low-Power DSP with Approximate Multiplier," ISSCC, 2020.

近似计算的局限性



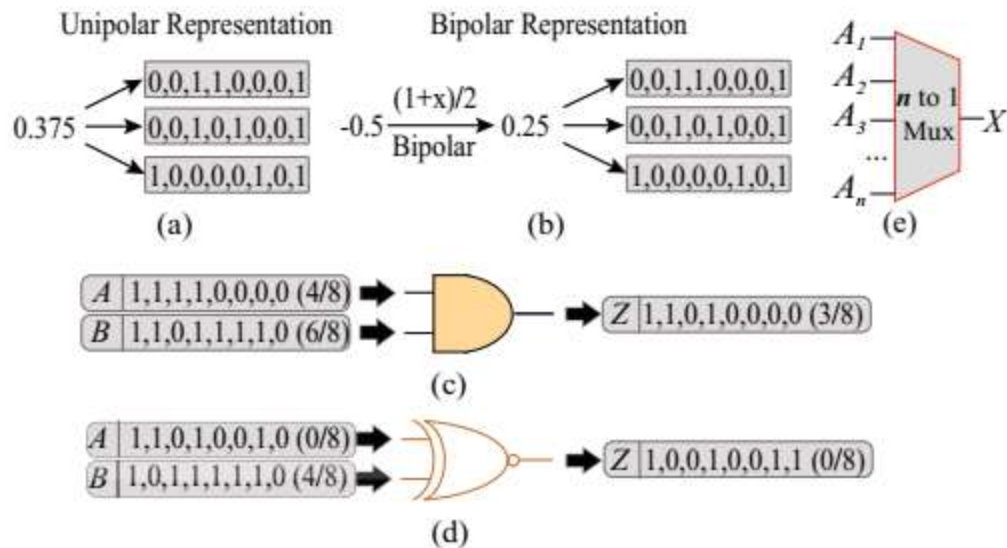
- 不同的设计对应用的容错性要求不同
- 近似计算单元对数据及计算架构敏感
- 近似计算单元对应用结果的精度影响不明确



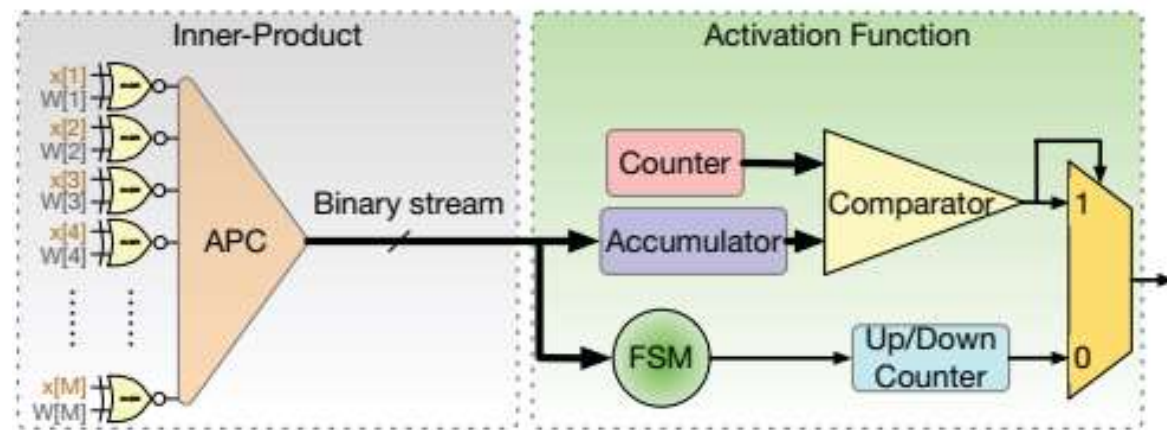
- **适用于领域定制芯片中**
- **利用可配置/可重构技术提高精度的选择性**
- **建立面向应用的误差模型**

概率计算

- 利用概率学原理，将数据表示为一长串的0、1序列
- 增加计算结果的鲁棒性
- 极大地简化了运算电路

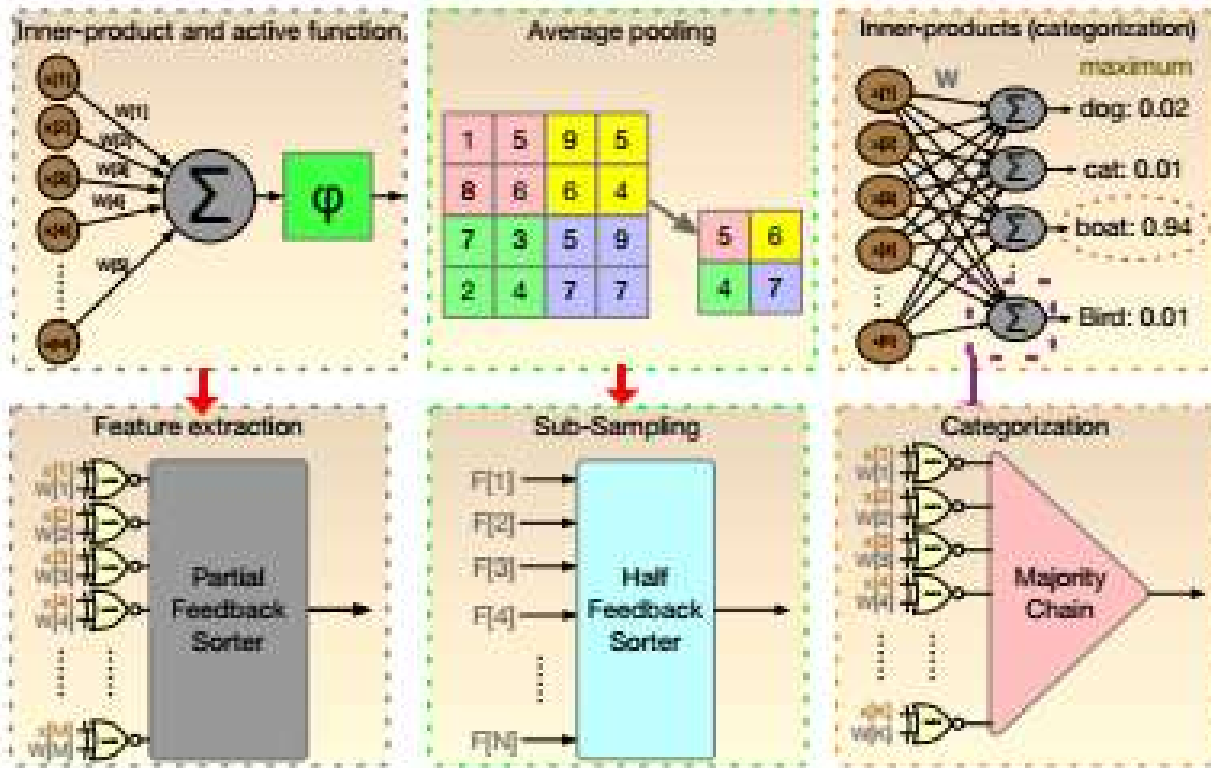


乘法运算电路



累加运算电路

基于概率计算的DNN加速器



DNN加速器各功能单元的概率计算实现

准确度、能耗、性能对比

Network	Platform	Accuracy	Energy(μ J)	Throughput(images/ms)
SNN	Software	99.04%	-	-
	CMOS	97.35%	39.46	231
	AQFP	97.91%	5.606E-4	8305
DNN	Software	99.17%	-	-
	CMOS	96.62%	219.37	229
	AQFP	96.95%	2.482E-3	6667

- 96%准确率 (MNIST)
- 69,000×能量效率



概率计算的局限性

- 序列长度对准确度影响很大，为保证精度则需要很长的序列，从而影响性能
- 随机数的相关性影响计算结果的精度
- 随机数发生器及数据格式之间的转换会带来额外开销



- **探索新的概率计算形式**
- **优化随机数发生器**
- **通过优化计算架构，尽量减少数据格式间的转换**



一、提升能量效率

二、降低芯片成本

三、增强硬件安全性、可靠性

降低集成电路芯片成本

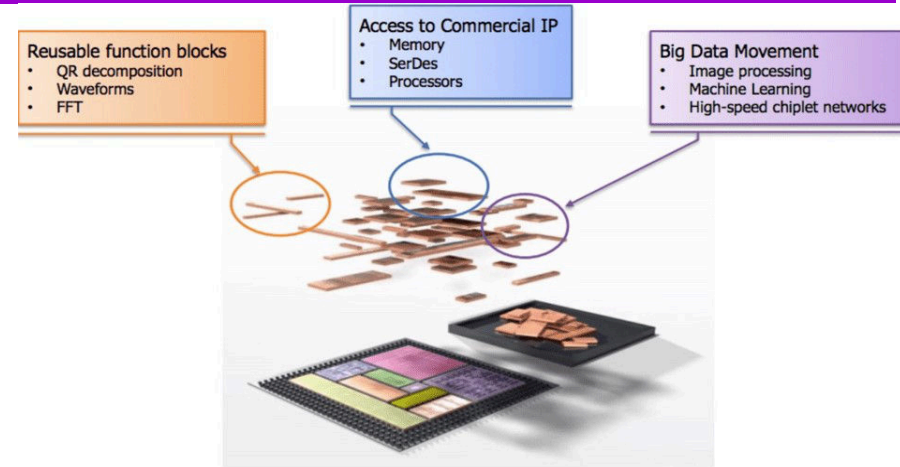


- 集成电路高成本源于
 - EDA工具许可费、IP许可费/Royalty提高
 - 半导体工艺进步，但难以让单个晶体管成本继续降低
 - 封装和测试的成本也相应提高
 - 从业人员开销巨大
 - 应用需求碎片化和多样化，单颗芯片销售量减少
- 应对手段
 - 异构集成/芯粒，Chiplet
 - 软件定义芯片，SDC
 - 开源化，RISC-V/LibreCores + HLS/OpenROAD
 - 敏捷开发，Agile Design

芯粒/ “小芯片” (Chiplet)

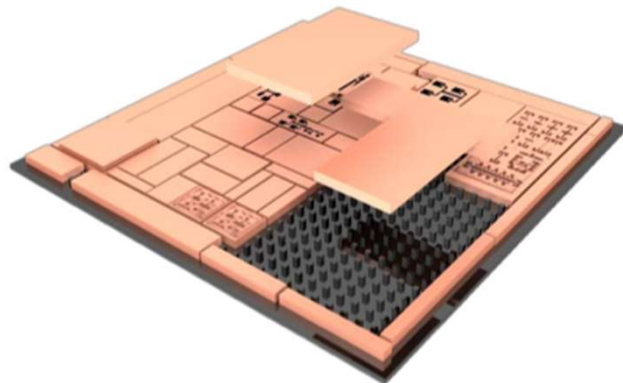
• Chiplet和高级封装

- 不同的芯片被集成在一起
- 将大SoC拆解成多个Chiplet，再集成
- 优势：
 - a) 良率提升、成本下降
 - b) 提高芯片集成度
 - c) 缩短互连延迟

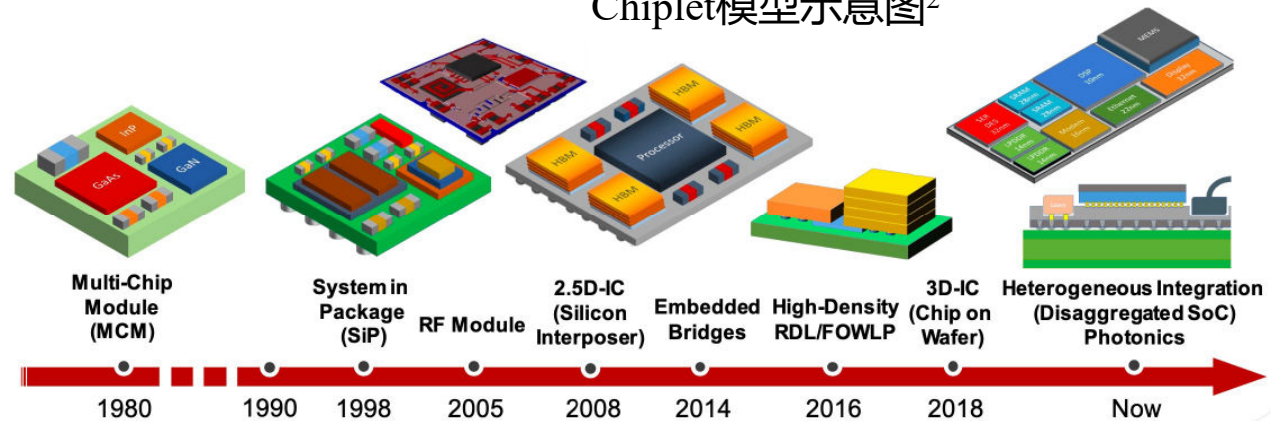


CHIPS modularity targets the enabling of a wide range of custom solutions

Chiplet模型示意图²



Chiplet概念示意图

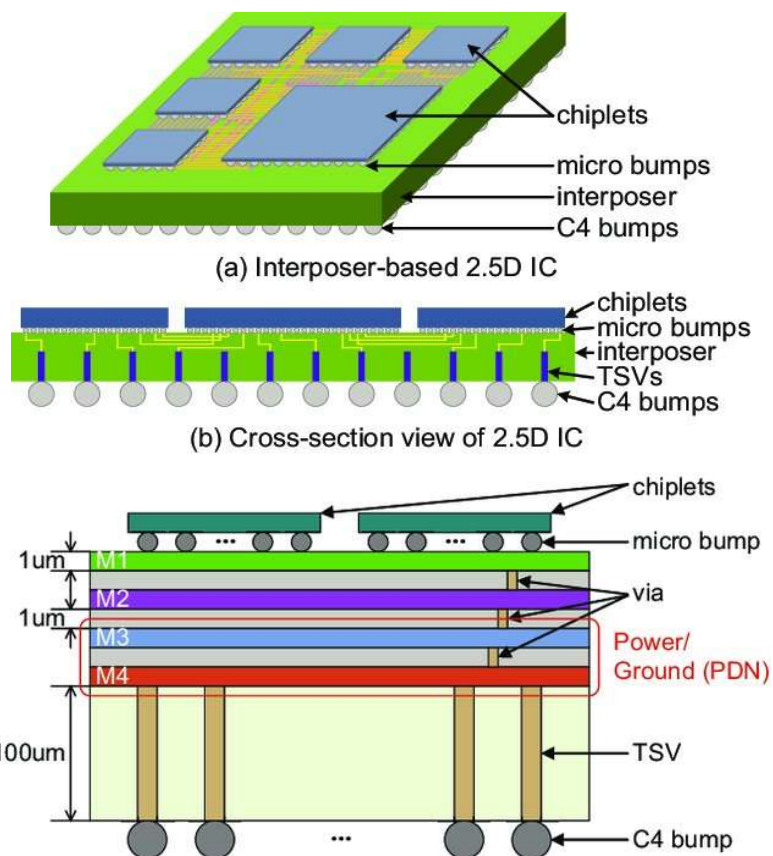


集成封装发展¹

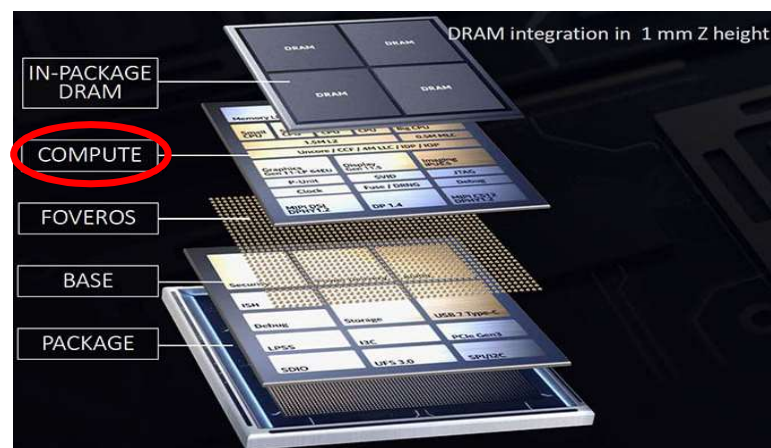
[1] Park, John. “The bifurcation of advanced packaging.” Additional Conferences (Device Packaging, HiTEC, HiTEN, & CICMT), 2019.

[2] Chiplet model. Source: DARPA

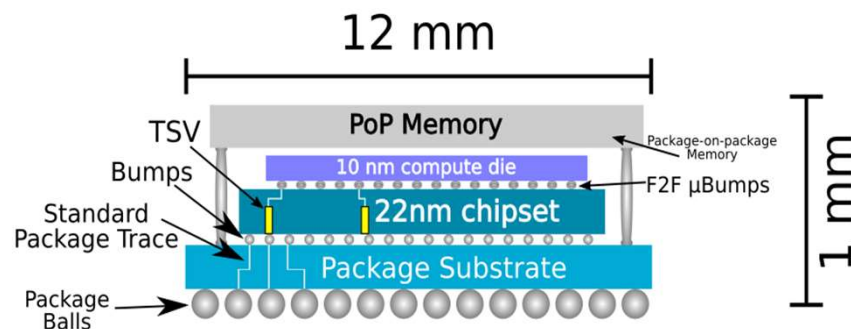
芯粒 / “小芯片”



2.5D Chiplet integration¹



Chiplets in Compute Layer

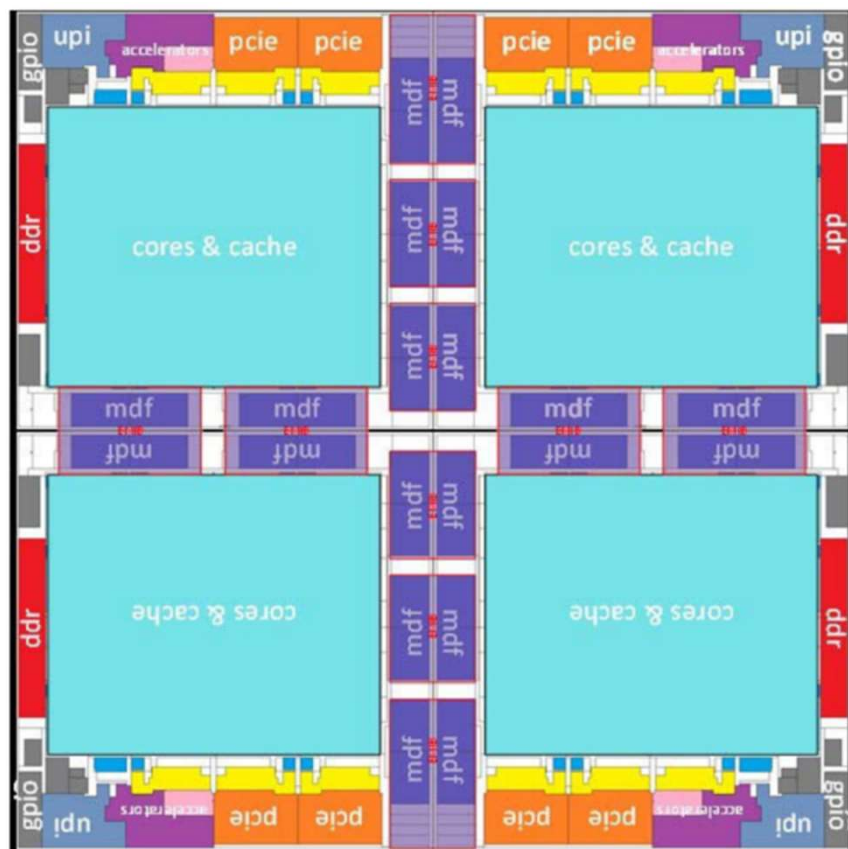


Intel 3D Lakefield Processor²

[1] Kim, Jinwoo, et al. "Architecture, chip, and package co-design flow for 2.5 D IC design enabling heterogeneous IP reuse." DAC, 2019.

[2] Intel. "Lakefield and Mobility Compute: A 3D Stacked 10nm and 22FFL Hybrid Processor System in 12×12mm², 1mm Package-on-Package. ISSCC, 2020

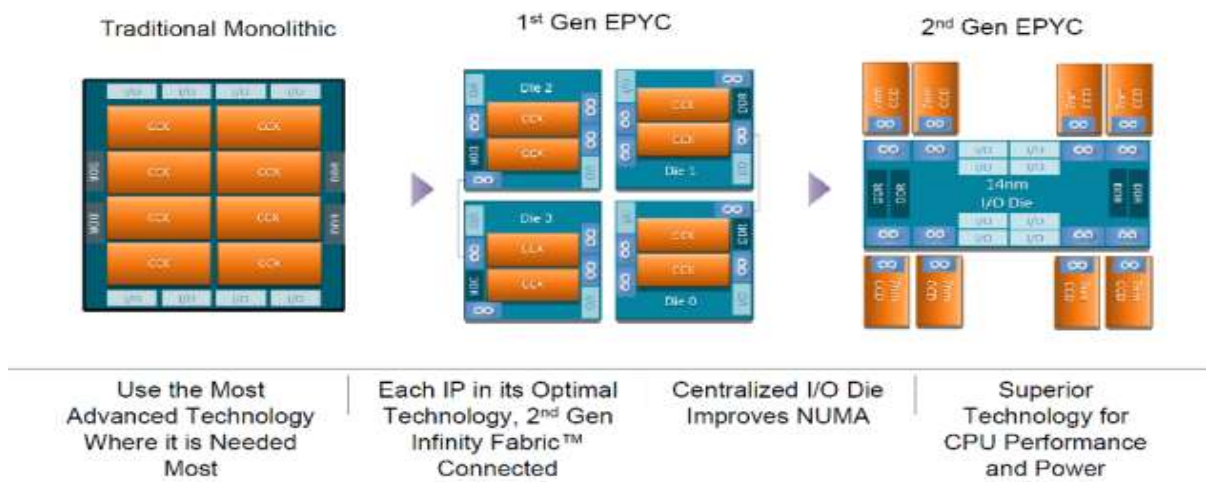
芯粒/“小芯片”



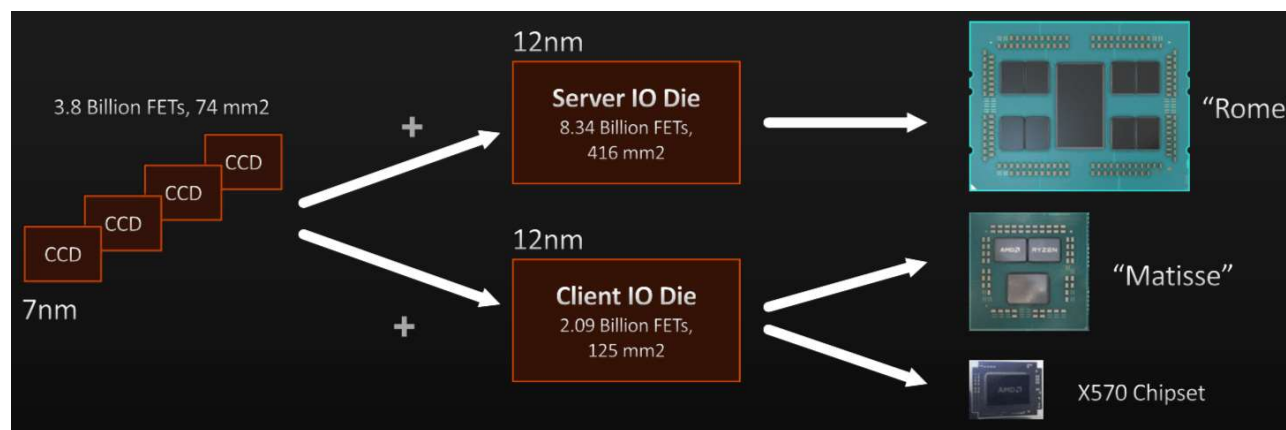
Intel Sapphire Rapids Exon Processor¹

[1] Intel. "Sapphire Rapids: The Next-Generation Intel Xeon Scalable Processor". ISSCC, 2022

芯粒/ “小芯片”



搭建低成本高性能计算核心¹

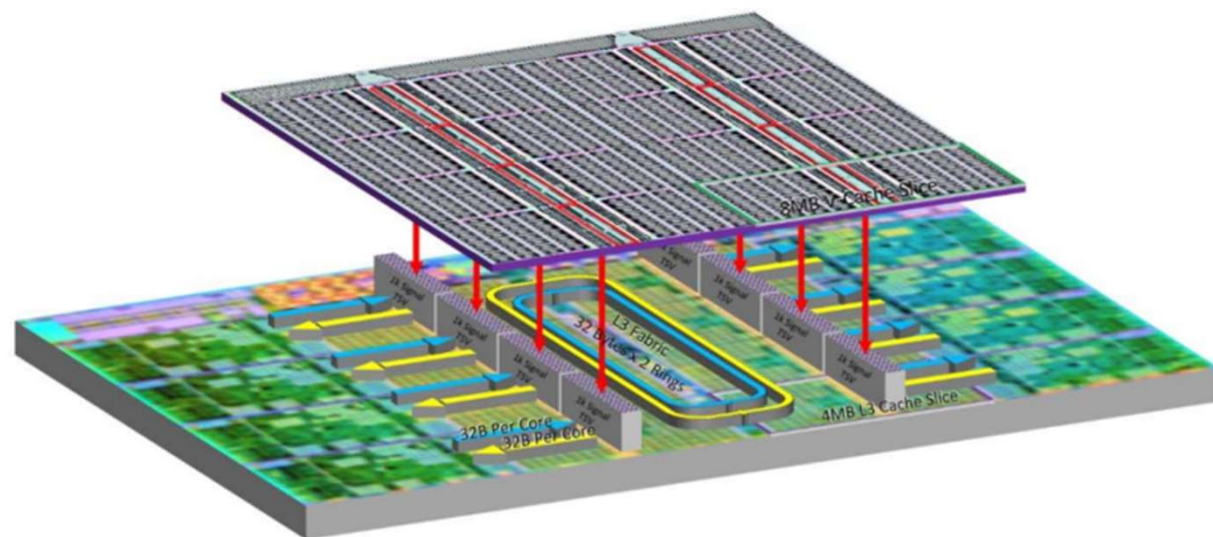
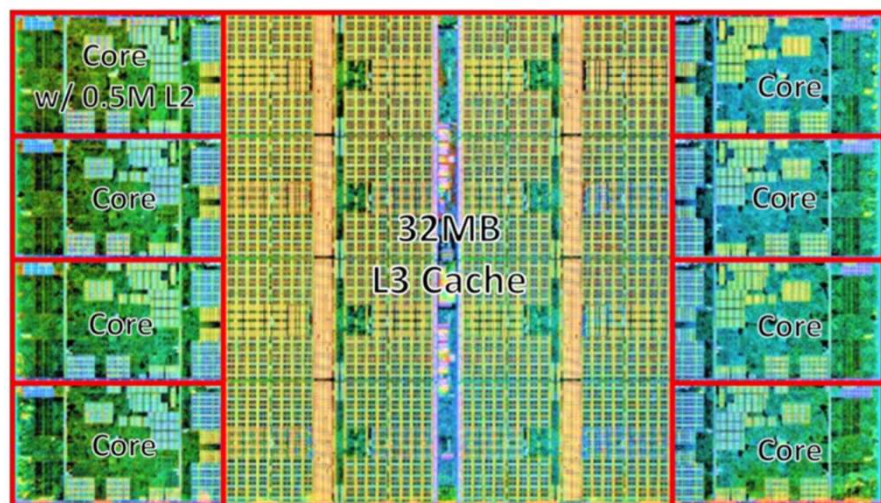


AMD EPYC Processors²

[1] AMD. Zen 2: The AMD 7nm Energy-Efficient High-Performance x86-64 Microprocessor Core. ISSCC, 2020.

[2] AMD. AMD Chiplet Architecture for High-Performance Server and Desktop Products. ISSCC, 2020

芯粒/“小芯片”



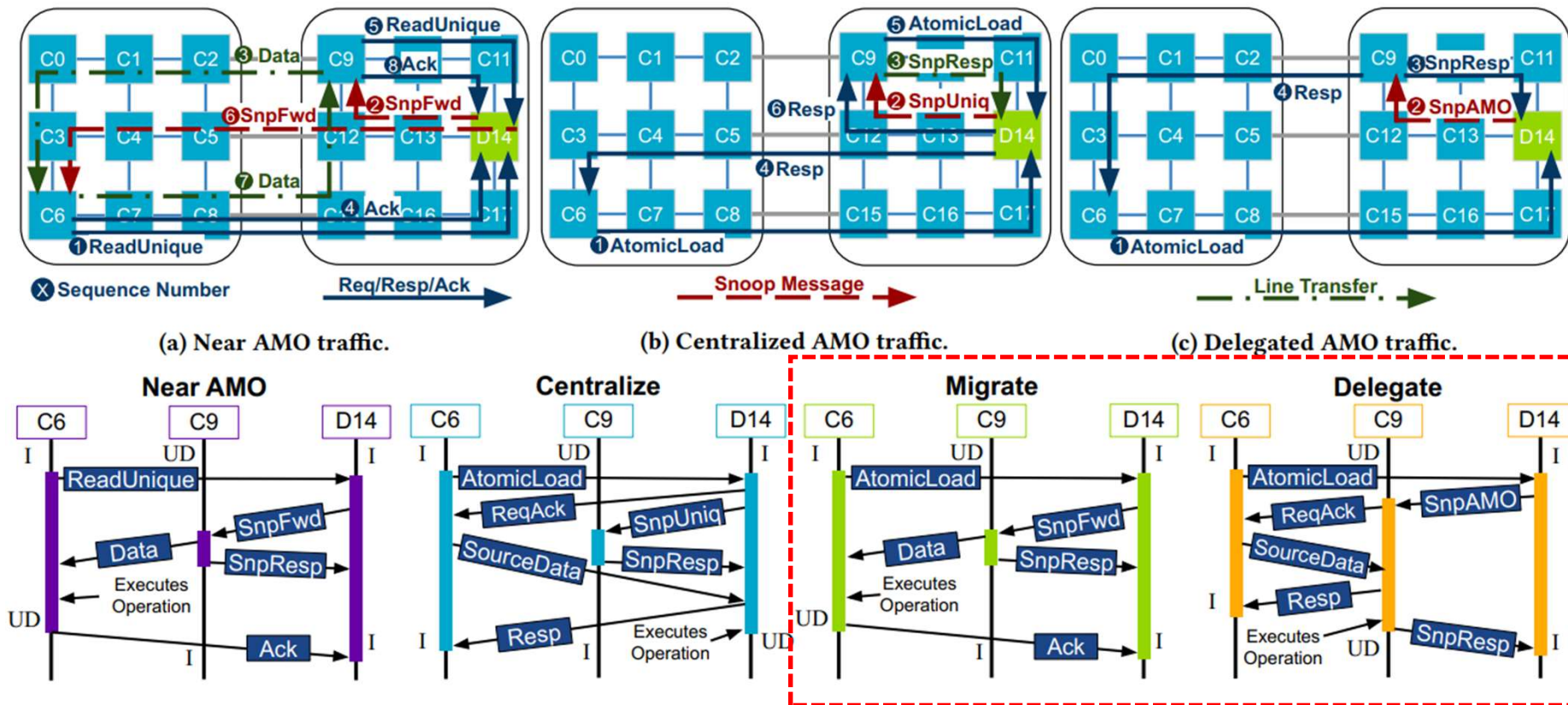
AMD Zen3 Processors¹

[1] AMD. Zen3: The AMD 2nd-Generation 7nm x86-64 Microprocessor Core. ISSCC, 2022.

异构集成/芯粒



原子内存操作（AMO）可感知的NUMA效应解决方案（Delegato）



在chiplet系统下三种不同的AMO方案样例（上）与协议流程（下）¹

[1] Soria-Pardos, V., Armejach, A., Mück, T., Gracia, D. S., Joao, J., & Moretó, M. (2025, October). Delegato: Locality-Aware Atomic Memory Operations on Chiplets. In Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture® (MICRO 2025) (pp. 1793-1808).

芯粒/“小芯片”



- Chiplet实现硅片级别的重用
- Chiplet技术开始落地，能大幅降低成本
- 借助3D堆叠以及NoC网络等技术，降低延迟



- **Chiplet边到边互联，可扩展性较差**
- **如何集成不容易扩展但却很必要的部分（如IO、电源管理等）**
- **会潜在地提升Serdes的使用频率**
- **未来IP授权会拓展到Chiplet**

软件定义芯片 (SDC) : 基本概念

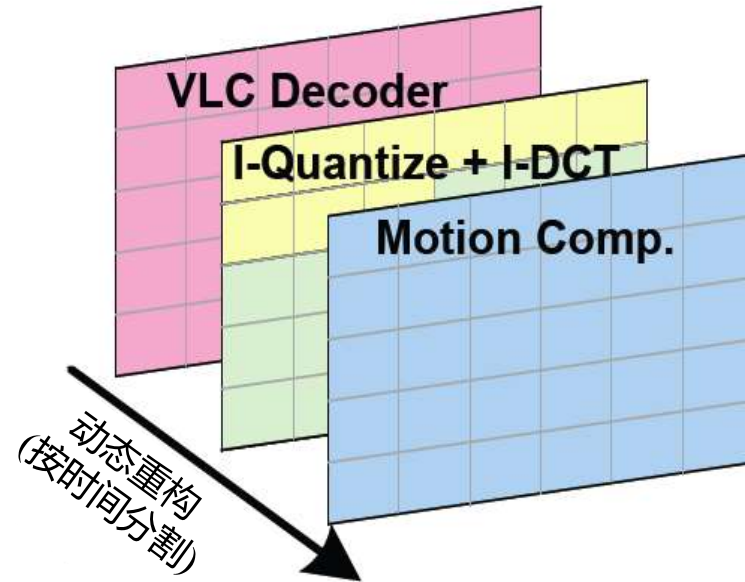
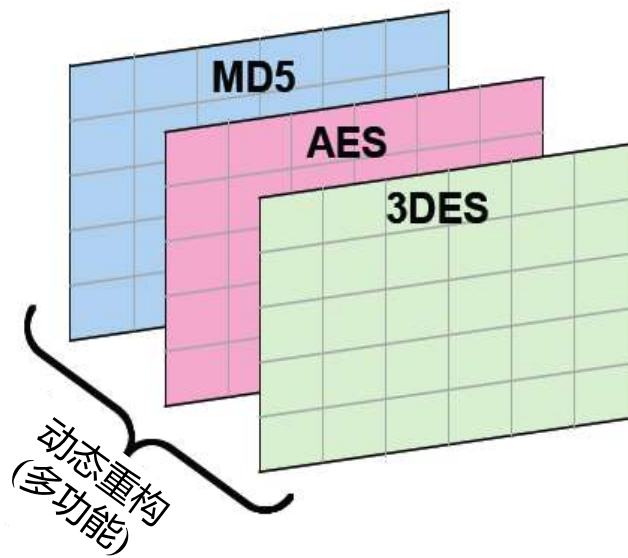


Gerald Estrin



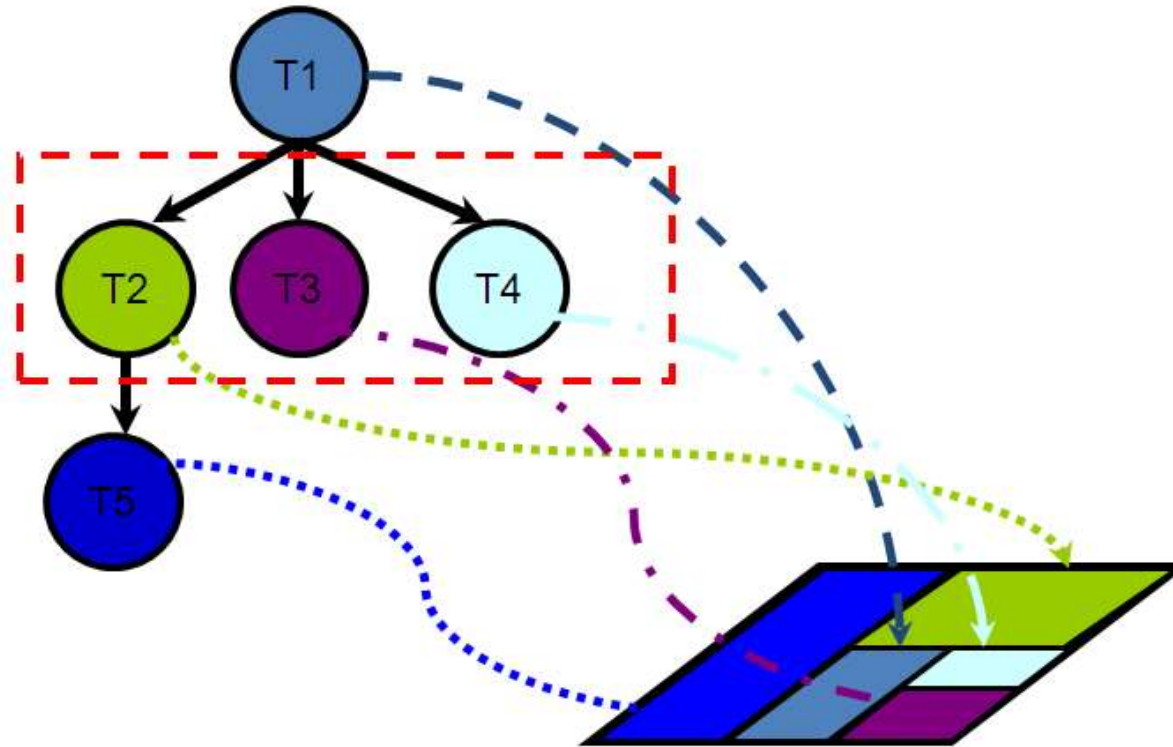
**“Organization of Computer Systems-The Fixed Plus Variable Structure Computer,”
Proc. Western Joint Computer Conf., Western Joint Computer Conference, New York,
1960, pp. 33-40.**

软件定义芯片 (SDC) : 基本概念



**“Organization of Computer Systems-The Fixed Plus Variable Structure Computer,”
Proc. Western Joint Computer Conf., Western Joint Computer Conference, New York,
1960, pp. 33-40.**

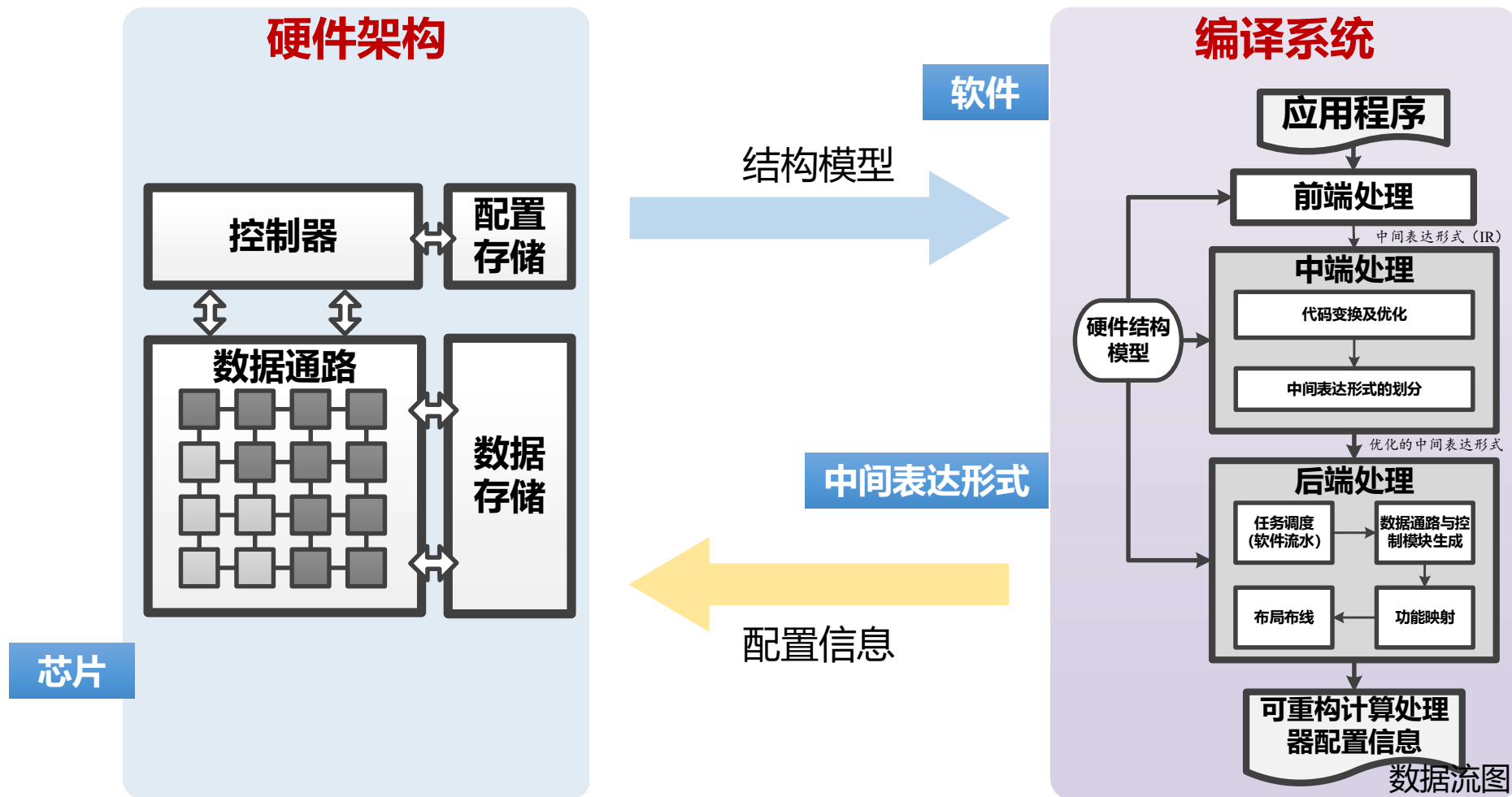
软件定义芯片 (SDC) : 基本概念



**“Organization of Computer Systems-The Fixed Plus Variable Structure Computer,”
Proc. Western Joint Computer Conf., Western Joint Computer Conference, New York,
1960, pp. 33-40.**

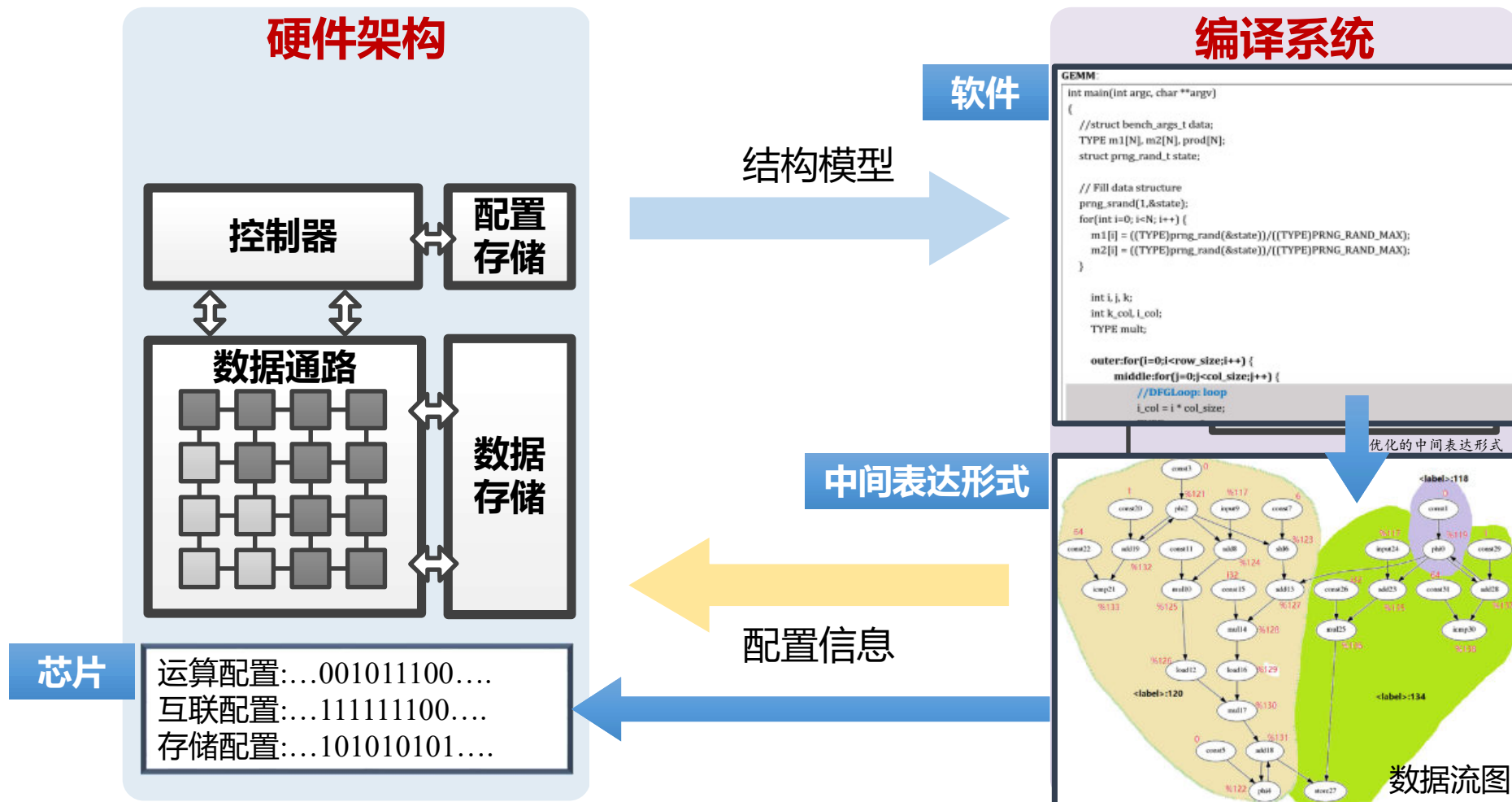


软件定义芯片（SDC）：基本原理



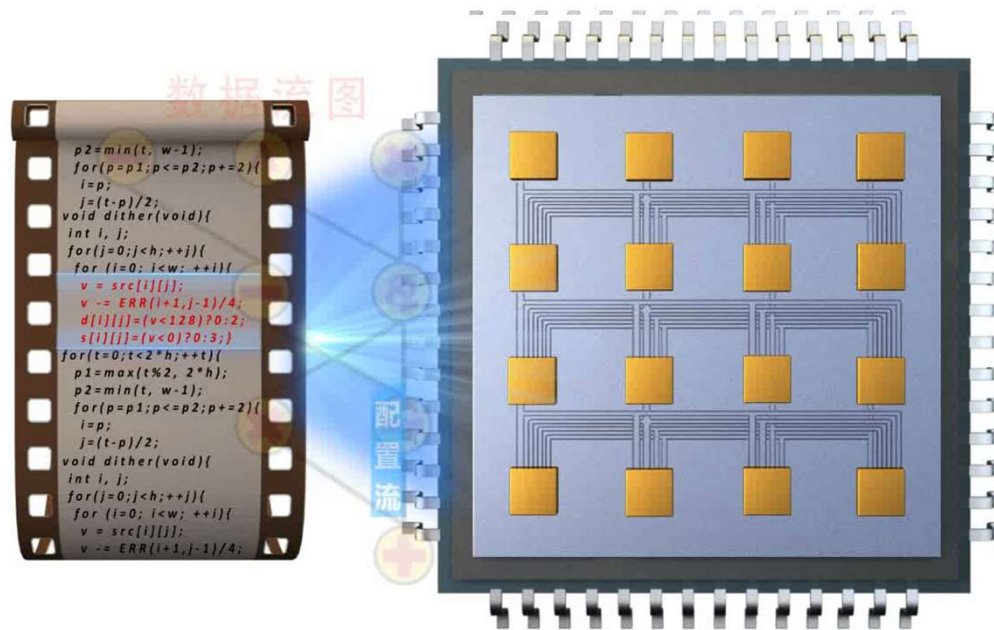


软件定义芯片 (SDC) : 基本原理



软件定义芯片（SDC）：基本原理

- 软件定义芯片以运算单元阵列为核心部件，由**配置流、数据流共同驱动，不使用指令。**
- 应用任务中的每个运算**都可以在运算阵列中找到对应的单元，单元间的互连与任务一一对应**，由此获得比拟专用电路的能量效率。



高级语言程序，如C/C++
(软件编程)

功能、互连等可按需动态改变的阵列
(芯片重构)



软件定义芯片 (SDC) : SDC vs. FPGA

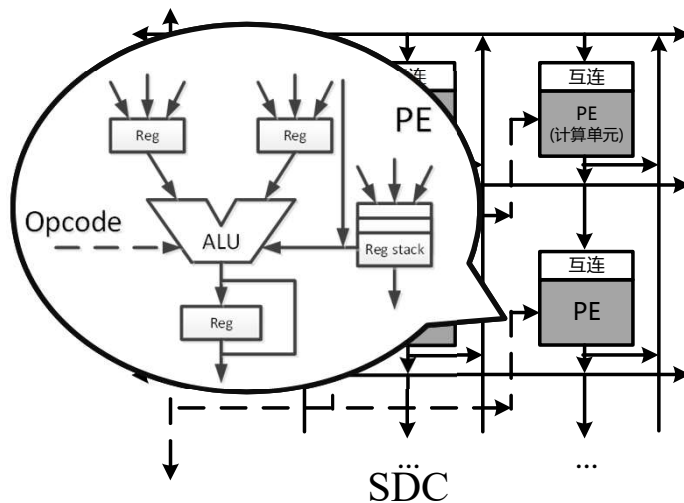
粒度: 8/16/32bit (ALU)

配置信息: 1~10KB

配置方式: **动态重构**

制造工艺: 标准工艺

编程方式: 高级语言编程, 编程者无需具备硬件电路知识,
所编写的程序不用转换成电路而直接生成配置文件



粒度: 1bit (LUT)

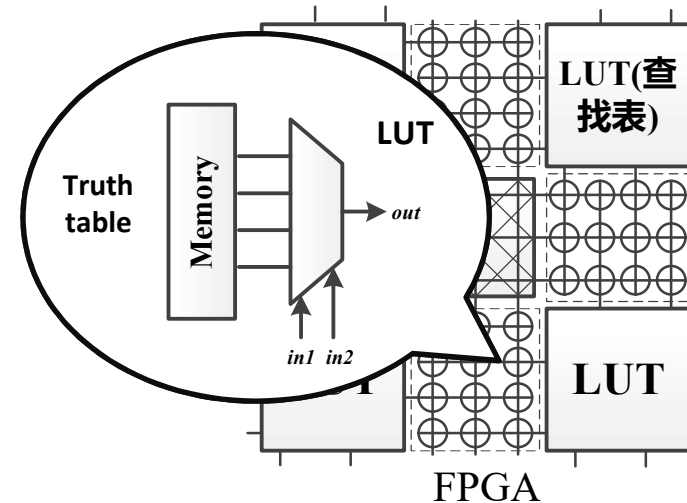
配置信息: 10~100MB

配置方式: **静态重构** (绝大多数情况)

制造工艺: 特殊工艺

编程方式: 无论使用高级语言还是HDL编程,
编程者必须具备较强硬件电路知识,

**所编写的程序需通过综合、布局布线
变成电路描述后, 生成配置文件**



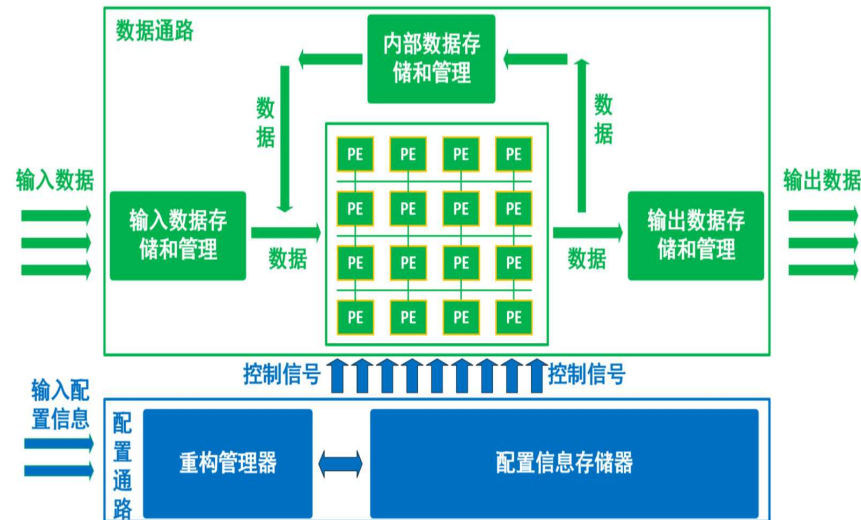
软件定义芯片 (SDC) : SDC vs. 多核处理器

执行模型: **不使用指令, 使用配置信息**
无程序计数器
数据驱动

并行度 : 细粒度并行

数据流 : **通过互连交换数据,**
 属显式数据通信

能量效率: 100~1000x CPU

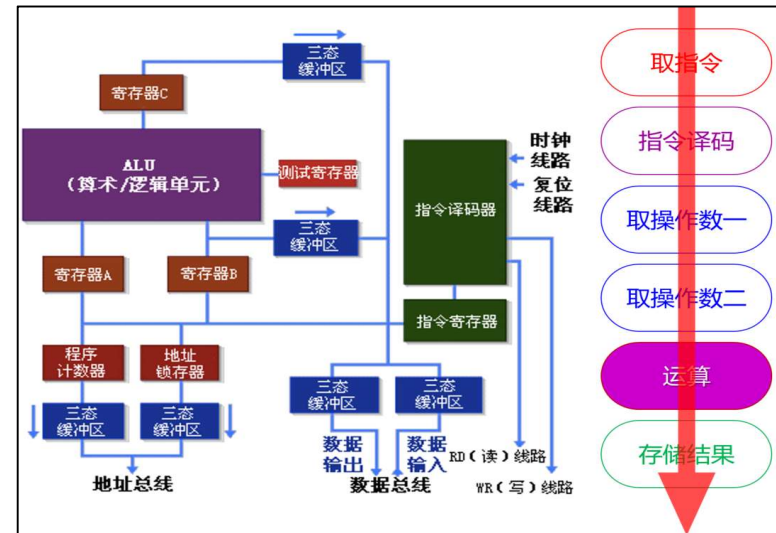


软件定义芯片

执行模型: **使用指令**
有程序计数器
指令驱动

并行度 : 串行化

数据流 : **通过存储交换数据,**
 属隐式数据通信



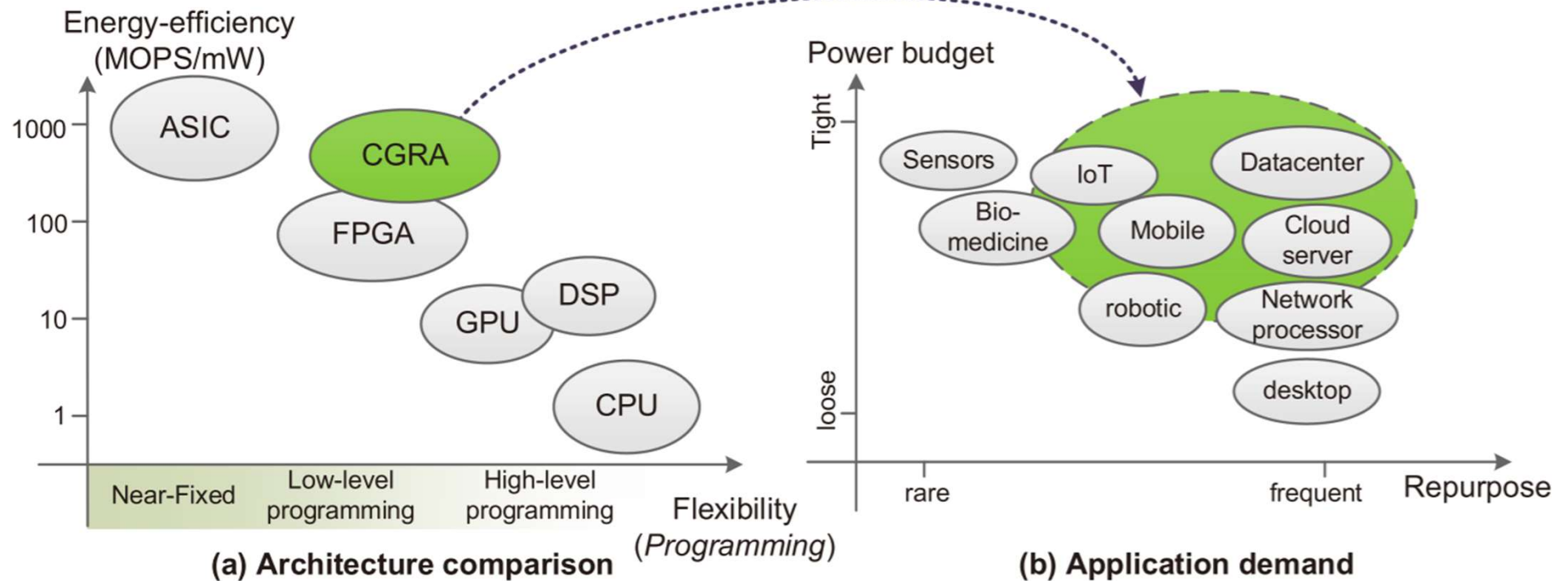
多核处理器

[1] Nowatzki, Tony, et al. "Pushing the limits of accelerator efficiency while retaining programmability." High Performance Computer Architecture (HPCA), 2016 IEEE International Symposium on. IEEE, 2016.

软件定义芯片（SDC）：应用范围

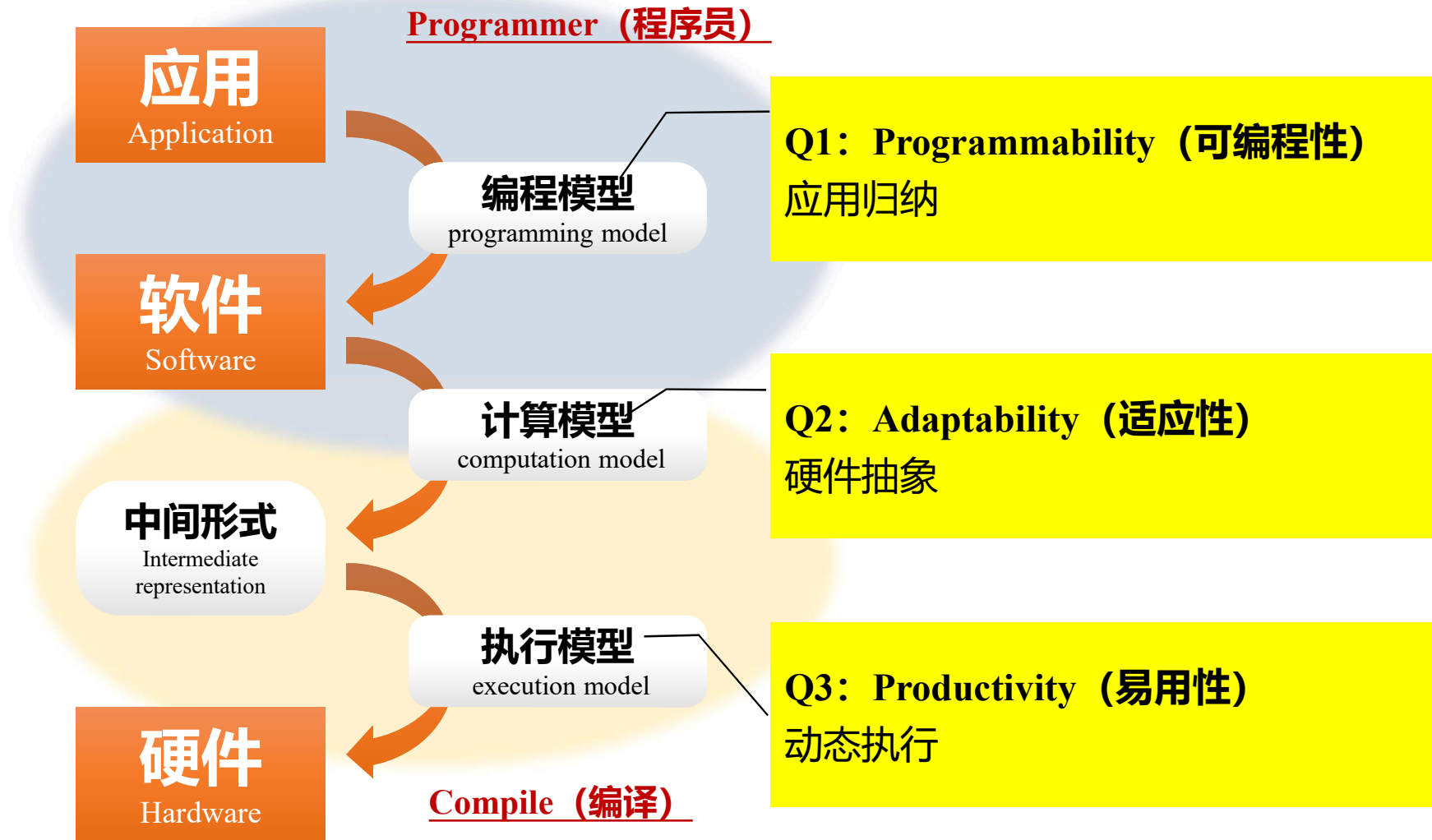
- 高性能（数据处理能力）→ 人工智能、网络、通信
- 高能效（计算效率）→ 数据中心、移动端、物端
- 高灵活（可编程性）→ 算法编程、协议更新、需求可变

Killer applications



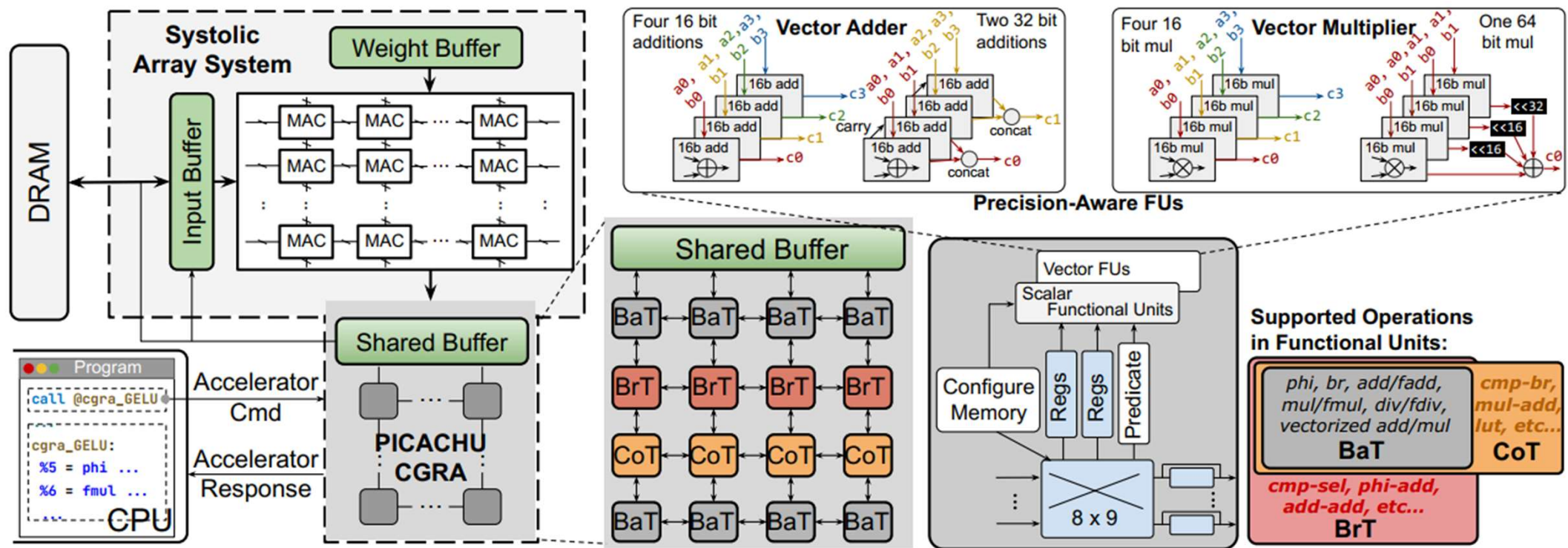


软件定义芯片 (SDC) : 技术难点



软件定义芯片 (SDC)

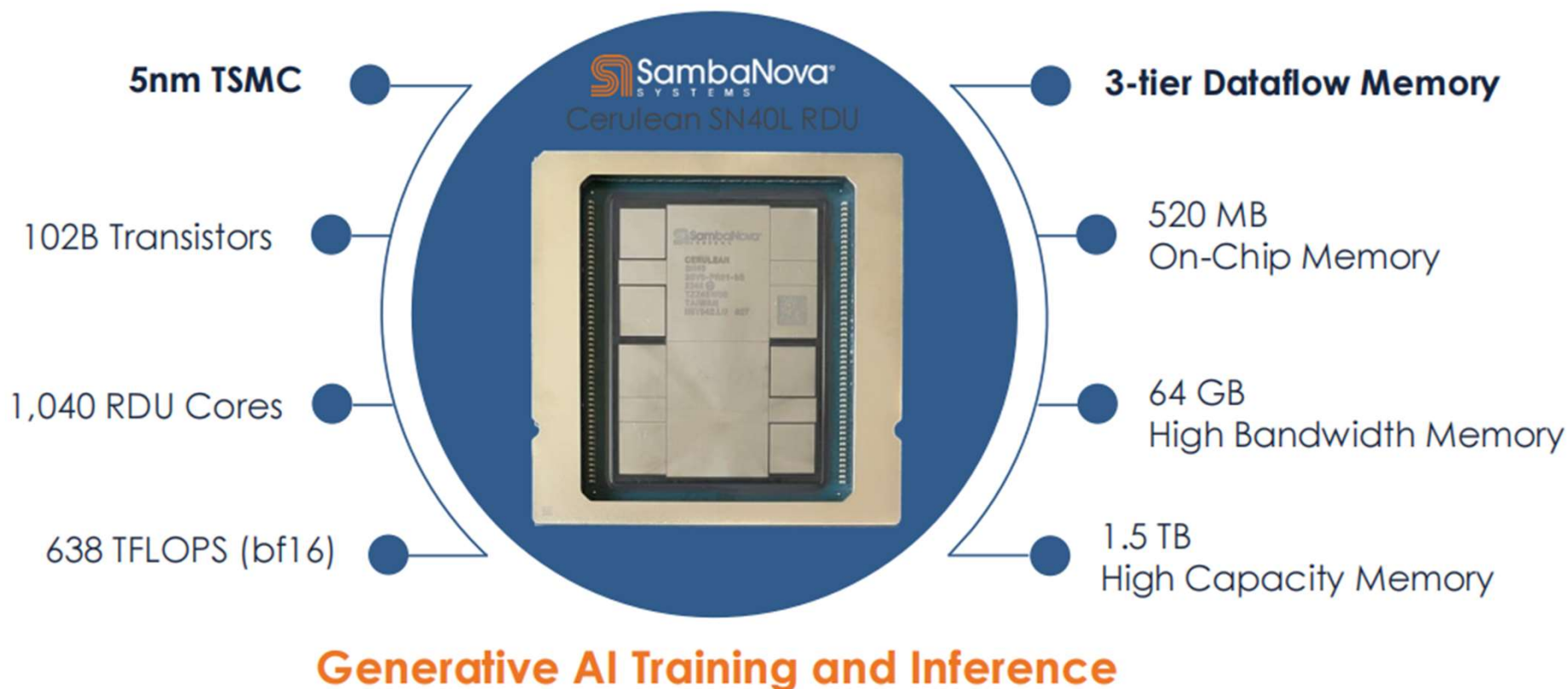
使用动态可重构单元加速AI领域特殊函数计算 (PICACHU)



PICACHU-CGRA加速单元架构设计¹

[1] Qin, J., Xia, T., Tan, C., Zhang, J., & Zhang, S. Q. (2025, March). PICACHU: Plug-In CGRA Handling Upcoming Nonlinear Operations in LLMs. In Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2025), Volume 2 (pp. 845-861).

软件定义芯片 (SDC) : 实际案例



设计开源化

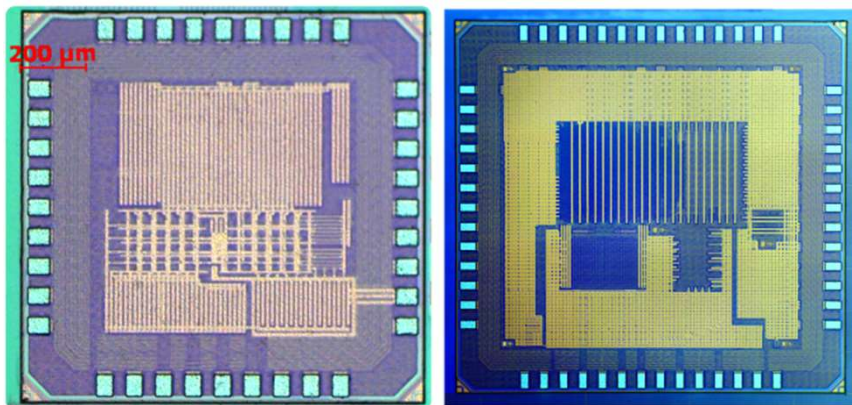


- Hardware whose **design** is made **publicly available** so that anyone can **study, modify, distribute, make**, and **sell** the design or hardware based on that design.¹
- 硬件开源化
 - RISC-V: Free and Open RISC Instruction Set Arch
 - OnChip Open-V
 - The Free and Open Source Silicon (FOSSi) Foundation
 - LibreCores
- EDA工具开源化
 - HLS
 - OpenROAD



[1] <https://www.oshwa.org/definition/>, Open Source Hardware Association.

硬件开源化 RISC-V



- Open-V 32-bit 处理器¹
- 全球第一款支持RISC-V的处理器
- AXI4-Lite通信总线协议
- 8个GPIOs接口
- 由Berkeley研究者开发 (2018年)

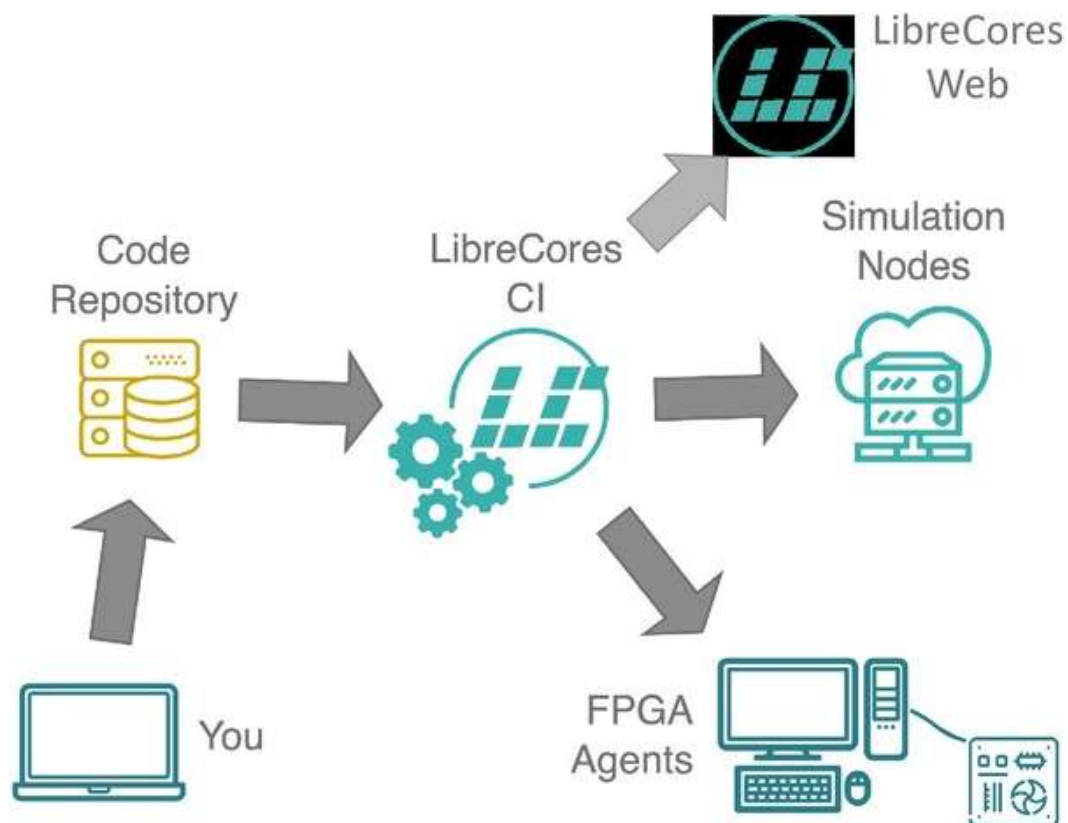


- 玄铁910处理器²
- 支持16核，主频为2.5GHz
- 3发射8执行的复杂乱序执行架构
- 扩展了50多条指令，增强计算、存储和多核等方面性能
- 平头哥半导体开发 (2019年)

[1] <http://www.onchipuis.io/open.html>

[2] T-Head. Alibaba Xuantie-910 RISC-V CPU. Hot Chips, 2020.

硬件开源化 LibreCores



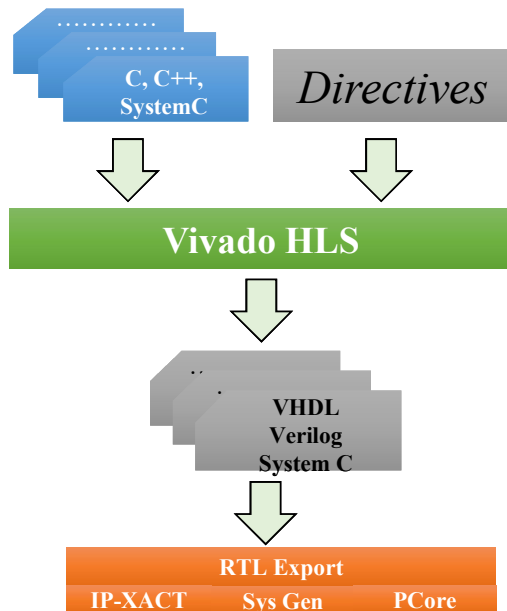
- 通过LibreCores CI服务，实现数字设计的共享¹
- 最新进展，仍在开发中

[1] <https://www.librecores.org/static/librecores-ci>

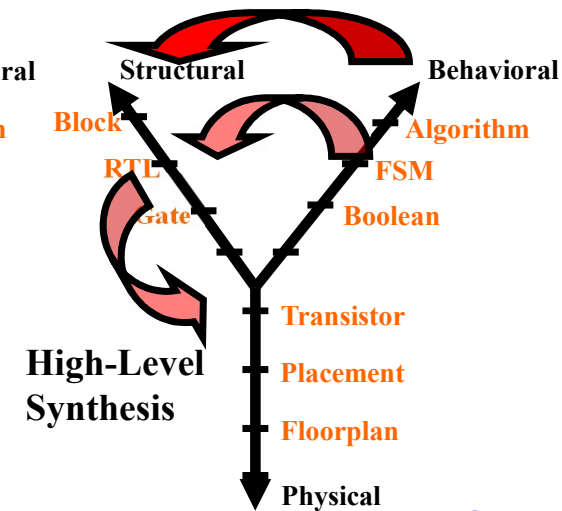
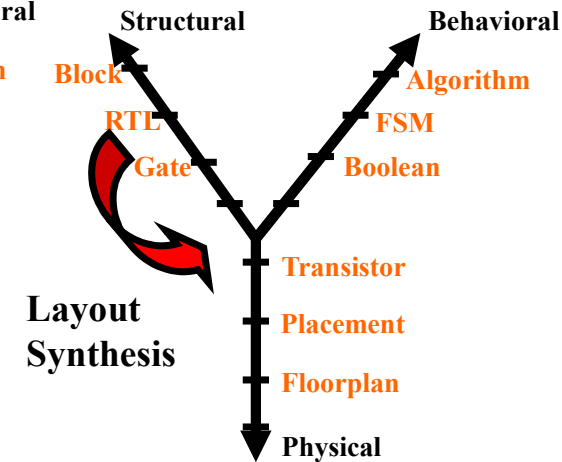
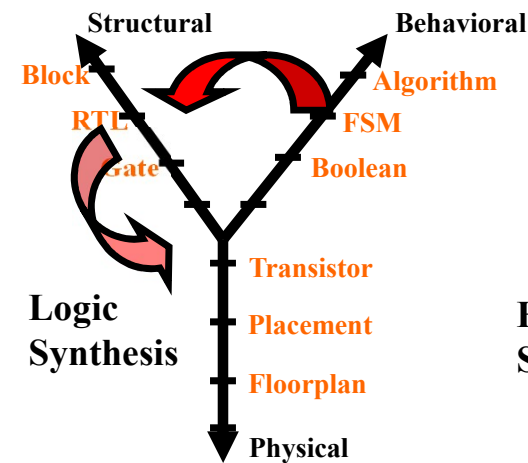
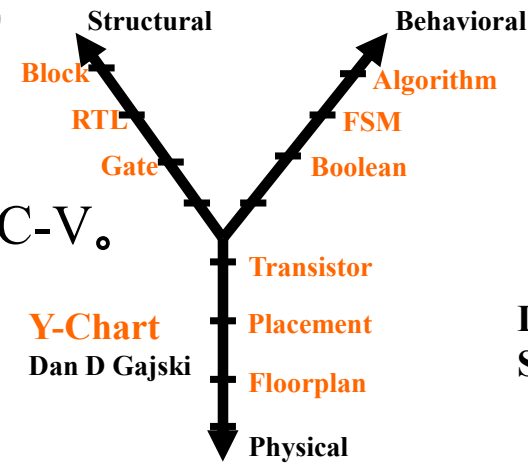
EDA工具开源化 HLS



- 高层次综合¹ (High Level Synthesis, HLS)
 - 将C/C++代码“编译”成为RTL代码
 - 降低硬件开发入门门槛
- 适用于新算法硬件实现需求，如AI、RISC-V。



Vivado HLS流程

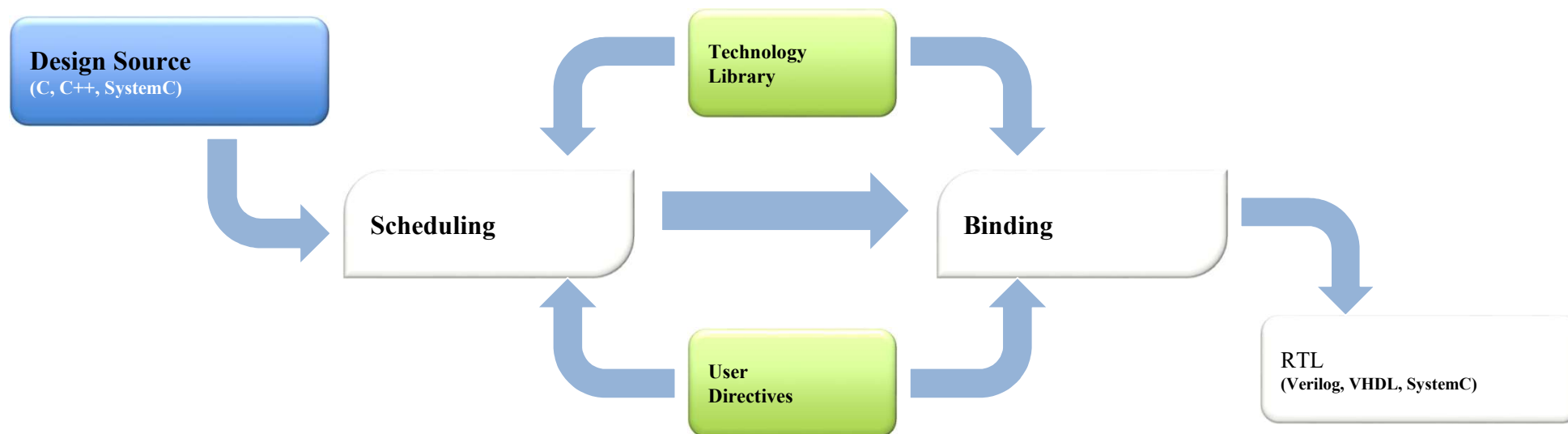


[1] Coussy, Philippe, et al. "An introduction to high-level synthesis." *D&T*, 2009.
[2] Dan D Gajski, etc. *New VLSI Tools*. 1983.

EDA工具开源化 HLS



- 工作流程
 - 代码分析及综合
 - 分析工艺文件、用户自定义条件
 - 完成代码映射



HLS具体工作流程

- 能实现从高级语言到电路模型转换
- 难以完全替代人工RTL实现

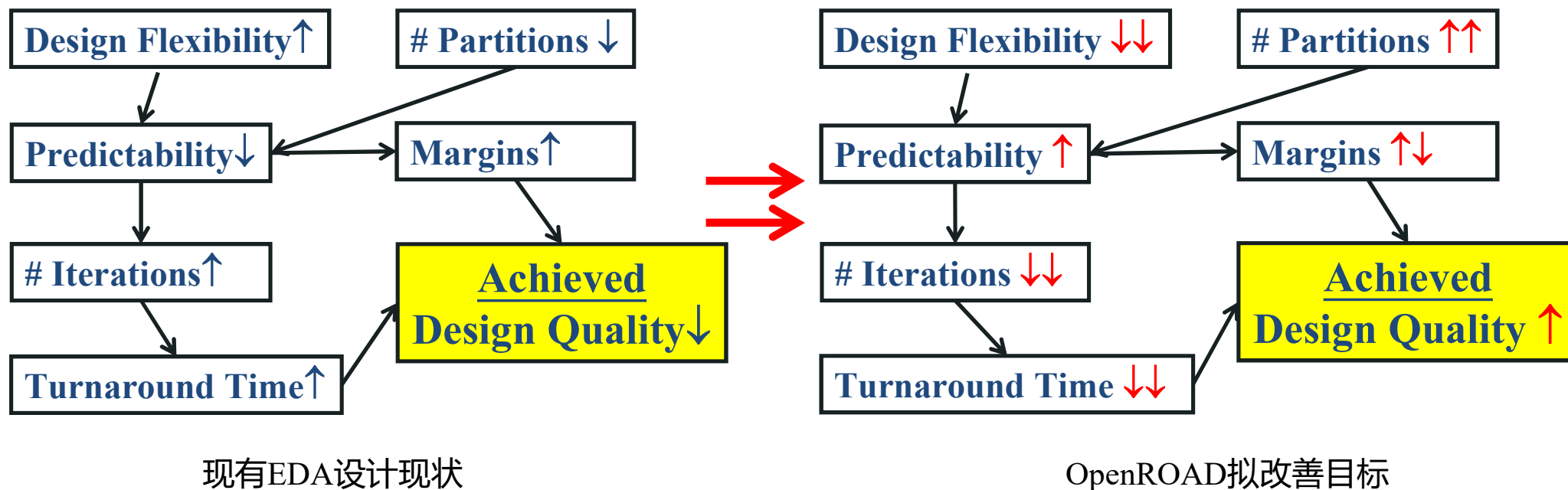


- 增加对于片内、片外各类存储单元的支持
- HLS的仿真和调试需要进一步探索

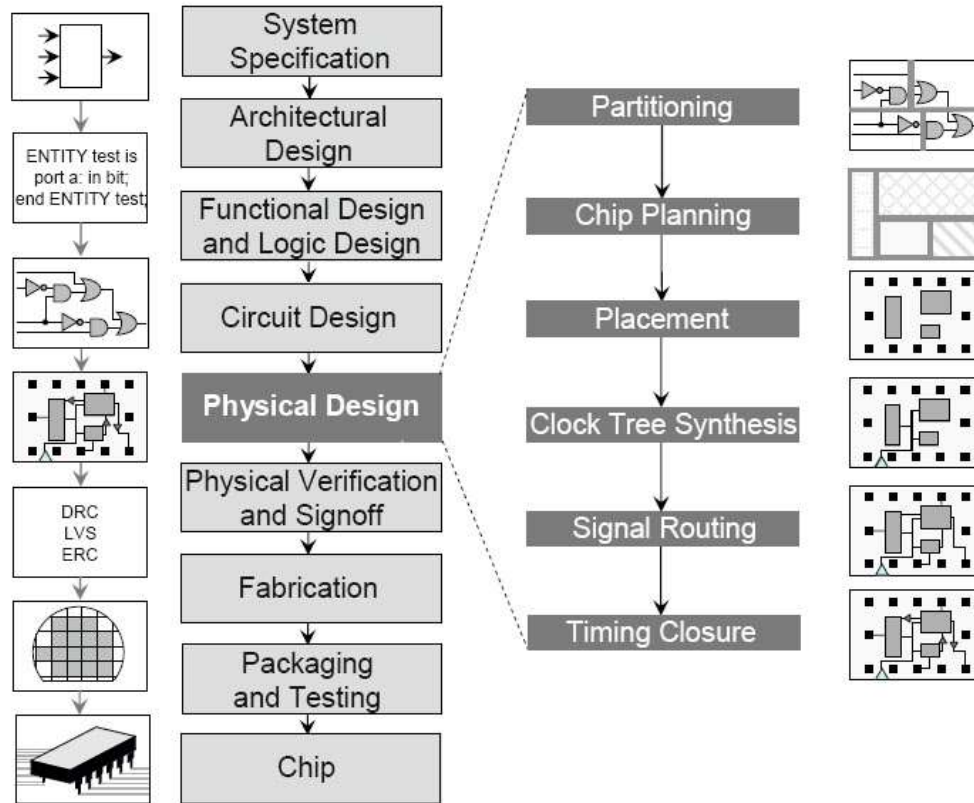
EDA工具开源化 OpenROAD



- 涵盖SoC、封装系统、印制电路板
- 系统工具全开源化

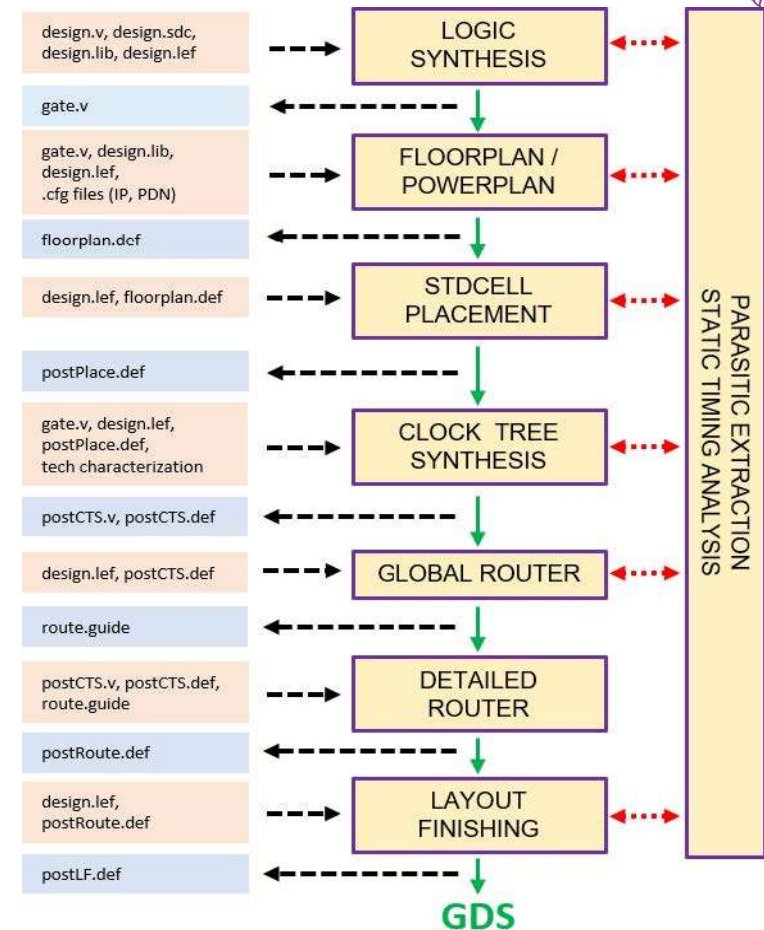


一般设计流程 vs OpenROAD



一般芯片设计流程¹

vs



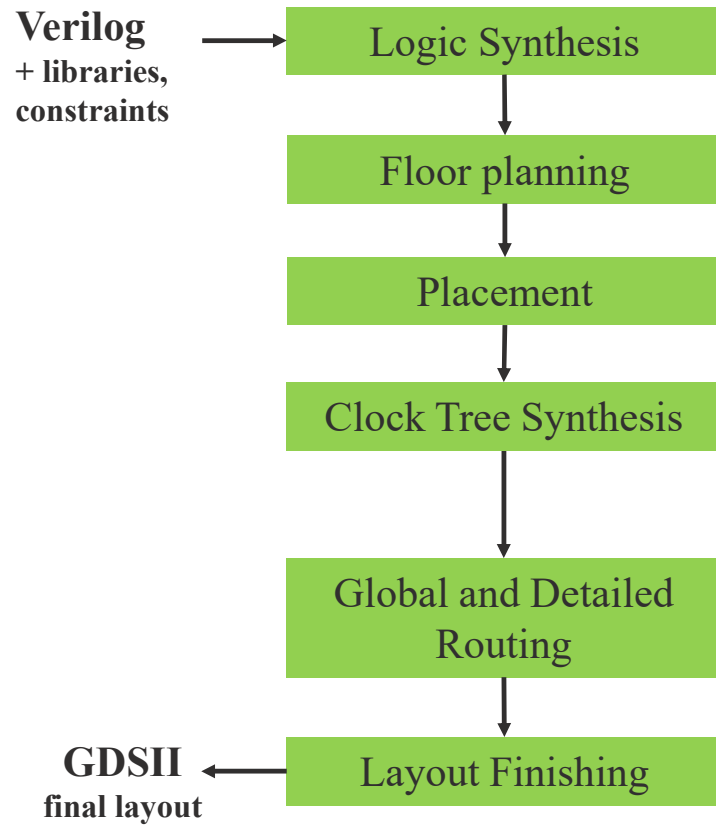
OpenROAD设计流程²

[1] <http://neeeasic.blogspot.com/2011/03/vlsi-design-flow.html>

[2] Ajayi, Tutu, et al. "Toward an open-source digital flow: First learnings from the openroad project." Proceedings of the 56th Annual Design Automation Conference 2019.



OpenROAD最新成果：从RTL到GDS



- 测试电路：RISC-V核心¹
 - 在各类SoC中进行过前期流片验证
 - 17K标准单元
 - 66K bits SRAM
- 测试工艺库：
 - TSMC 65LP
- 结果展示：
 - 从RTL一键式生成GDS文件
 - DRC无违例

[1] “VanillaBean”, U. Michigan, 2019.

OpenROAD最新成果：从RTL到GDS



Verilog
+ libraries,
constraints

Logic Synthesis

Floor planning

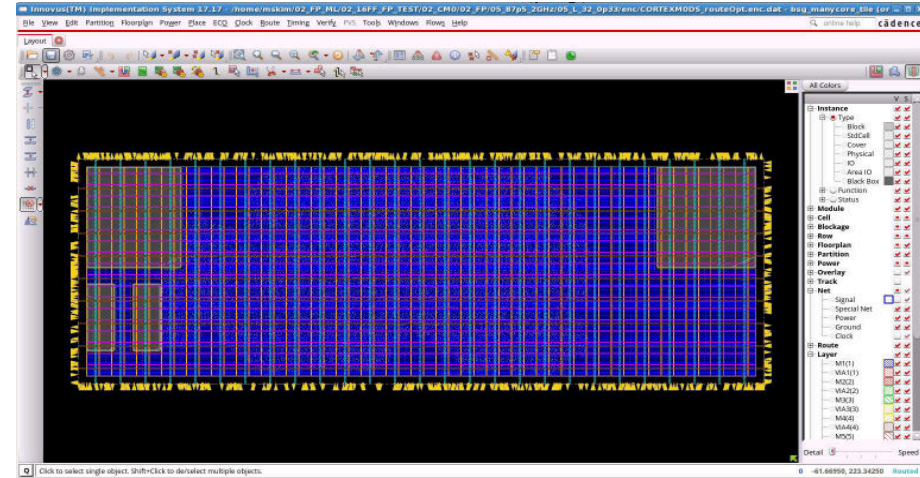
Placement

Clock Tree Synthesis

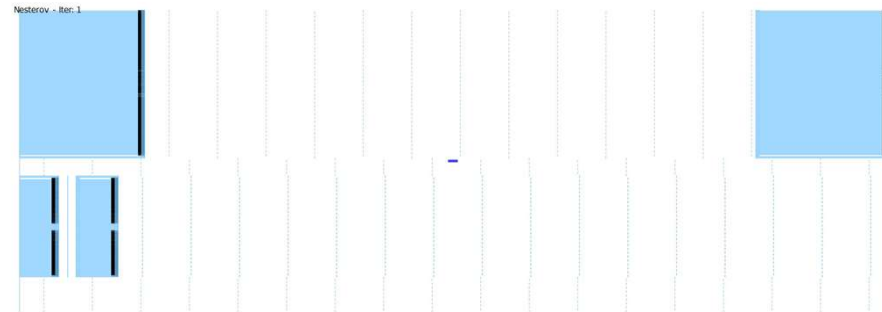
Global and Detailed
Routing

Layout Finishing

GDSII
final layout



Floor planning输出



布局过程

OpenROAD最新成果：从RTL到GDS



Verilog
+ libraries,
constraints

Logic Synthesis

Floor planning

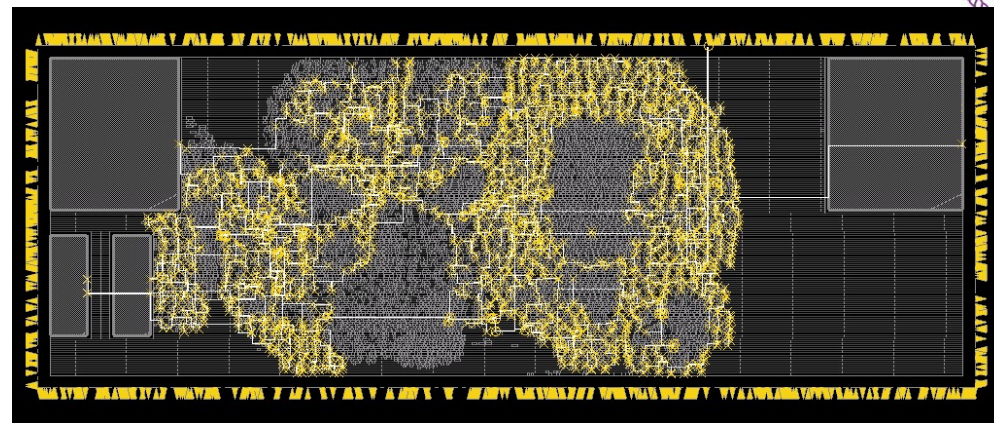
Placement

Clock Tree Synthesis

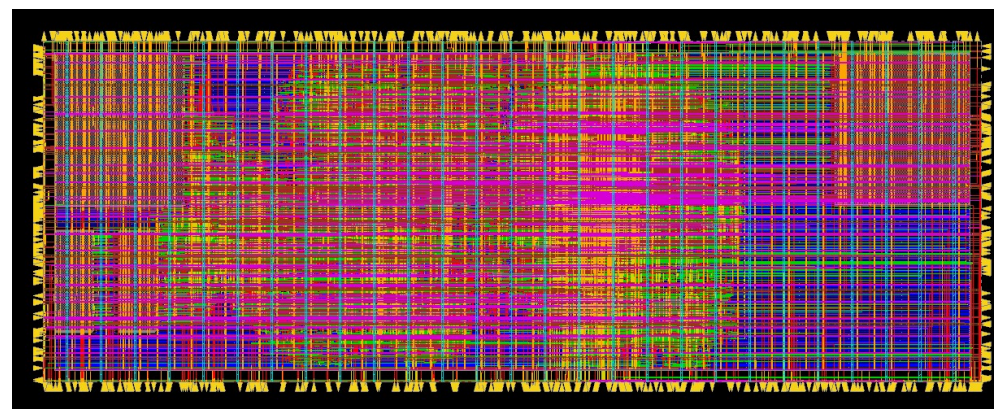
Global and Detailed
Routing

GDSII
final layout

Layout Finishing



时钟树综合—白色线



细布线最终结果



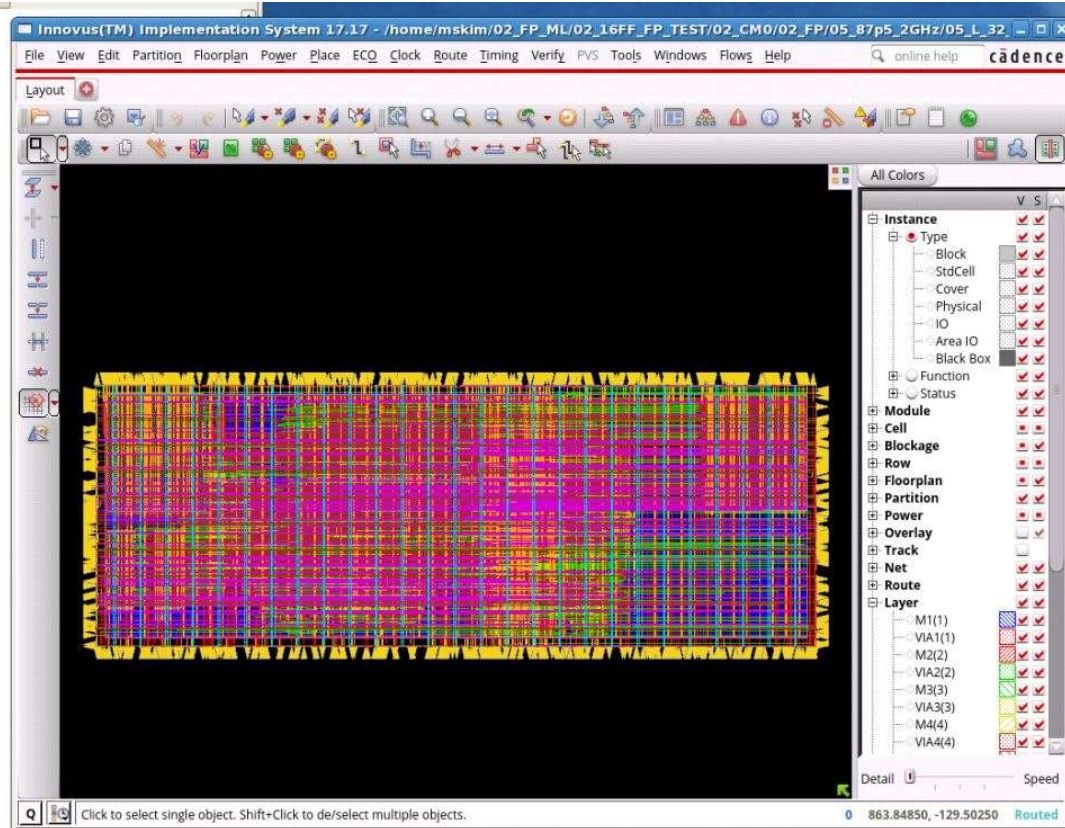
OpenROAD最新成果：从RTL到GDS

```
VERIFY DRC ..... Sub-Area: {254.880 0.000 339.840 80.640} 4 of 65 Thread : 1
VERIFY DRC ..... Sub-Area: {679.680 161.280 764.640 241.920} 35 of 65 Thread : 5
VERIFY DRC ..... Sub-Area: {764.640 0.000 849.600 80.640} 10 of 65 Thread : 6
VERIFY DRC ..... Sub-Area: {84.960 241.920 169.920 322.560} 41 of 65 Thread : 6
VERIFY DRC ..... Sub-Area: {84.960 80.640 169.920 161.280} 15 of 65 Thread : 3
VERIFY DRC ..... Sub-Area: {424.800 0.000 509.760 80.640} 6 of 65 Thread : 2
VERIFY DRC ..... Sub-Area: {594.720 0.000 679.680 80.640} 8 of 65 Thread : 7
VERIFY DRC ..... Sub-Area: {764.640 161.280 849.600 241.920} 36 of 65 Thread : 5
VERIFY DRC ..... Sub-Area: {509.760 241.920 594.720 322.560} 46 of 65 Thread : 4
VERIFY DRC ..... Sub-Area: {169.920 241.920 254.880 322.560} 42 of 65 Thread : 6
VERIFY DRC ..... Sub-Area: {849.600 322.560 934.560 400.000} 63 of 65 Thread : 4
VERIFY DRC ..... Sub-Area: {339.840 241.920 424.800 322.560} 44 of 65 Thread : 0
VERIFY DRC ..... Sub-Area: {934.560 322.560 1019.520 400.000} 64 of 65 Thread : 4
VERIFY DRC ..... Sub-Area: {254.880 80.640 339.840 161.280} 17 of 65 Thread : 1
VERIFY DRC ..... Sub-Area: {1019.520 322.560 1100.000 400.000} 65 of 65 Thread : 1
VERIFY DRC ..... Sub-Area: {84.960 161.280 169.920 241.920} 28 of 65 Thread : 6
VERIFY DRC ..... Sub-Area: {764.640 80.640 849.600 161.280} 23 of 65 Thread : 5
VERIFY DRC ..... Sub-Area: {679.680 322.560 764.640 400.000} 61 of 65 Thread : 3
VERIFY DRC ..... Sub-Area: {424.800 80.640 509.760 161.280} 19 of 65 Thread : 2
VERIFY DRC ..... Sub-Area: {254.880 241.920 339.840 322.560} 43 of 65 Thread : 0
VERIFY DRC ..... Sub-Area: {594.720 80.640 679.680 161.280} 21 of 65 Thread : 7
VERIFY DRC ..... Sub-Area: {764.640 241.920 849.600 322.560} 49 of 65 Thread : 5
VERIFY DRC ..... Sub-Area: {764.640 322.560 849.600 400.000} 62 of 65 Thread : 5
VERIFY DRC ..... Sub-Area: {424.800 241.920 509.760 322.560} 45 of 65 Thread : 4
VERIFY DRC ..... Sub-Area: {169.920 80.640 254.880 161.280} 16 of 65 Thread : 6
VERIFY DRC ..... Sub-Area: {254.880 322.560 339.840 400.000} 56 of 65 Thread : 0
VERIFY DRC ..... Sub-Area: {339.840 80.640 424.800 161.280} 18 of 65 Thread : 2
VERIFY DRC ..... Sub-Area: {509.760 80.640 594.720 161.280} 20 of 65 Thread : 7
VERIFY DRC ..... Sub-Area: {594.720 241.920 679.680 322.560} 47 of 65 Thread : 3
VERIFY DRC ..... Sub-Area: {679.680 80.640 764.640 161.280} 22 of 65 Thread : 1
VERIFY DRC ..... Sub-Area: {424.800 322.560 509.760 400.000} 58 of 65 Thread : 4
VERIFY DRC ..... Sub-Area: {254.880 161.280 339.840 241.920} 30 of 65 Thread : 2
VERIFY DRC ..... Sub-Area: {424.800 161.280 509.760 241.920} 32 of 65 Thread : 7
VERIFY DRC ..... Sub-Area: {679.680 241.920 764.640 322.560} 48 of 65 Thread : 3
VERIFY DRC ..... Thread : 2 finished.
VERIFY DRC ..... Thread : 6 finished.
VERIFY DRC ..... Sub-Area: {594.720 322.560 679.680 400.000} 60 of 65 Thread : 7
VERIFY DRC ..... Thread : 7 finished.
VERIFY DRC ..... Sub-Area: {594.720 161.280 679.680 241.920} 34 of 65 Thread : 3
VERIFY DRC ..... Thread : 3 finished.

Verification Complete : 0 Viols.

*** End Verify DRC (CPU: 0:00:04.5 ELAPSED TIME: 1.00 MEM: 2.0M) ***

innovus 1> win
innovus 2> 
```



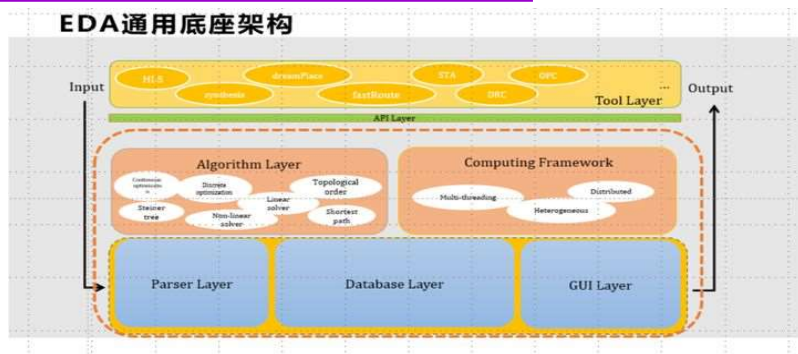
最终版图检查，没有DRC违例

- 能实现从RTL到GDS-II文件
- 部分流程、文件仍依赖传统EDA工具

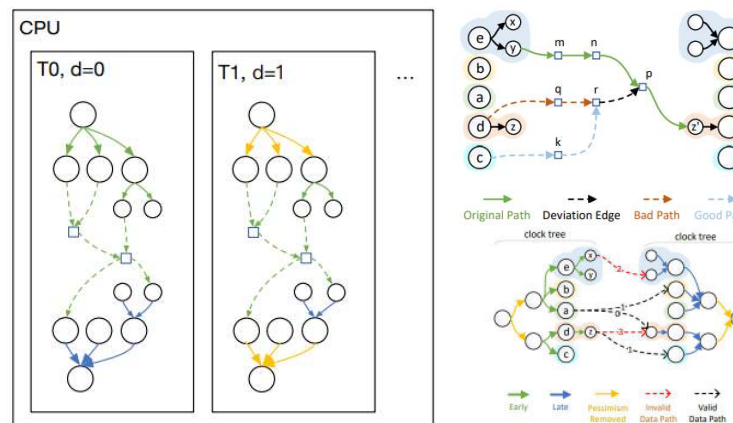


- 值得继续探索
- 短期内难以撼动传统EDA厂商地位

国产开源EDA



南京集创以开源方式打造国产EDA通用平台，有望把点工具连接在一起，支持所有国产EDA工具，聚合成套的EDA工具链



北京大学深度学习辅助布局布线优化项目，利用深度学习来改进传统的布局布线算法，未来有望发布开放EDA数据集^[1]

openDACS开源电路与系统设计自动化
主线版本 V1.0

<https://forgeplus.trustie.net/opendacs>

EDA方向	SIG1: 设计验证 &测试综合	SIG2: 逻辑综合 &高层综合	SIG3: 物理设计 &建模验证	SIG4: 器件模型 &参数提取	SIG6: PCB设计 &验证
开源工具	故障仿真器	工艺映射器	1) Verilog Parser 2) OpenEDI开源数据基础组件	薄膜晶体管紧凑模型OD_TFT	FPC自动布线工具
发布单位	中科院计算所 中科芯公司	北京大学	1) 复旦大学 2) 南京EDA创新中心	中科院微电子所	武汉理工大学

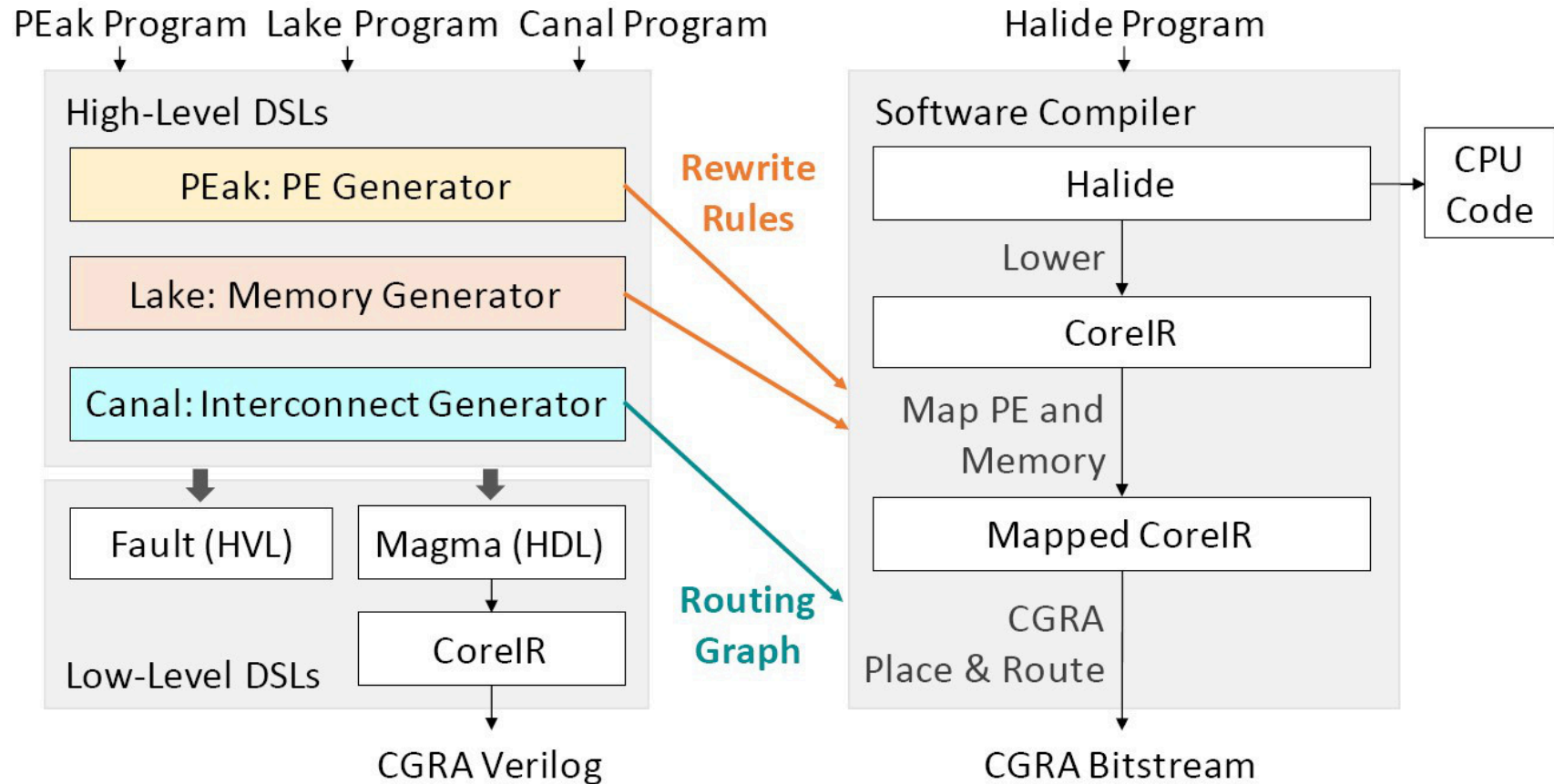
由CCF集成电路设计专业组、中科院计算所、中科院微电子所、北京大学等多家单位通过自发组织发起的“openDACS开源电路与系统设计自动化”开源项目

[1]Z. Guo, M. Yang, T. -W. Huang and Y. Lin, “A Provably Good and Practically Efficient Algorithm for Common Path Pessimism Removal in Large Designs,” in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2021, doi: 10.1109/TCAD.2021.3124758.



- 知识产权的问题仍旧难以绕开，**开源不等于免费，不等于不被“卡脖子”**；
开源的自由与商业的服务价值是两回事；
- 无论是芯片设计代码还是EDA工具开源，都存在难以逾越的质量问题；
- 模拟、验证、仿真调试的环境与生态建立尚需时日。

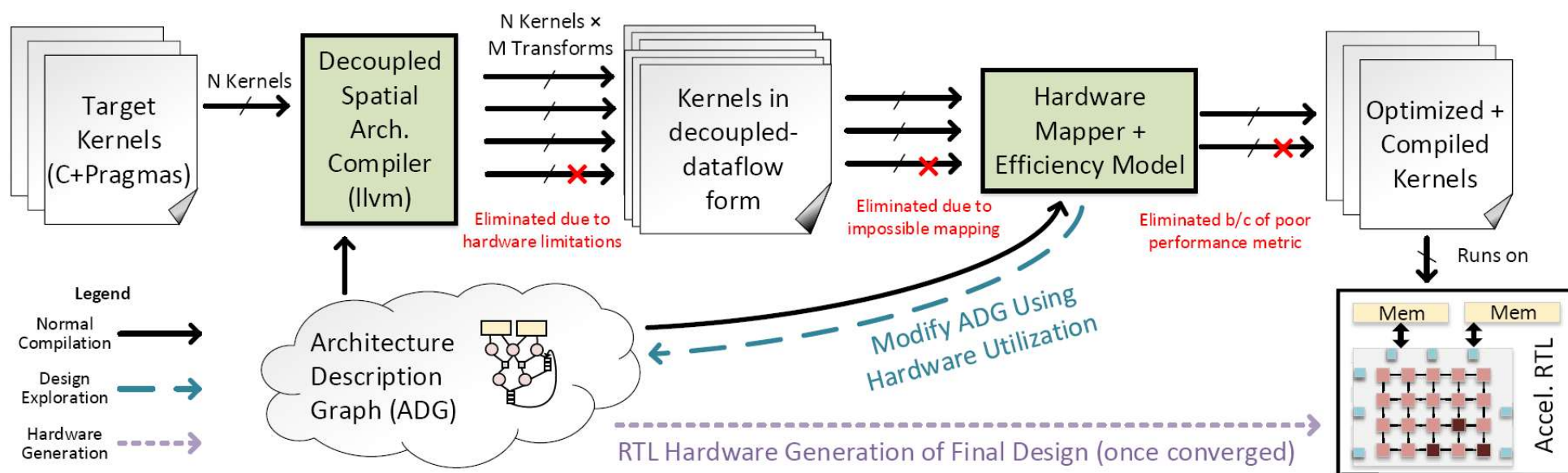
敏捷开发 (Agile Design)



斯坦福大学的AHA (agile hardware) 项目¹

[1] Bahr, Rick, et al. "Creating an agile hardware design flow." 2020 57th ACM/IEEE Design Automation Conference (DAC). IEEE, 2020.

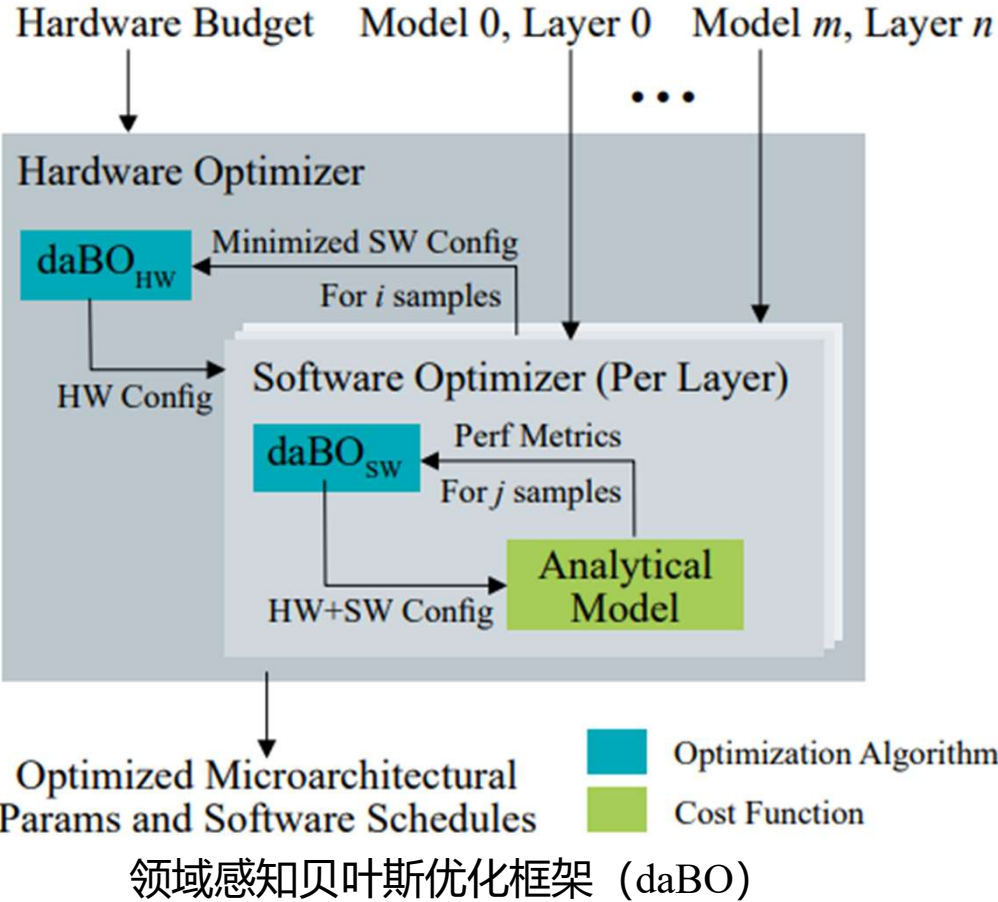
敏捷开发 (Agile Design)



加州大学洛杉矶分校DSAGEN项目¹

[1] Weng, Jian, et al. "Dsagen: Synthesizing programmable spatial accelerators." ISCA, 2020.

敏捷开发 (Agile Design)



Parameter	Range
SIMD Lanes	2 to 16
Bandwidth	64 to 256
PEs	128 to 300

(a) Cardinal parameters.

Parameter	Range	Stride
Scratchpad Size	64 to 256 KB	8
Register File Size	64 to 256 KB	8
PE Aspect Ratio	Divisors of PE Count	N/A
Tiling Factors [†]	Divisors of layer shape	N/A

(b) Ordinal parameters.

Parameter	Values
Loop Order [†]	Permutations of loops
Unroll Dimension [†]	N, K, C, R, S, X, Y

(c) Categorical parameters.

[†]Independent values per scratchpad level.

软硬件设计参数

[1] C. Sakhuja et al, “Leveraging Domain Information for the Efficient Automated Design of Deep Learning Accelerators,” HPCA 2023.

敏捷开发 (Agile Design)



- 从SPEC到GDS-II的流程虽然有所缩短，但比较有限，verification的工作量依然巨大
- 无法缩短芯片制造、封装、测试的时间
- 工程样片validation的时长仍旧没有缩短

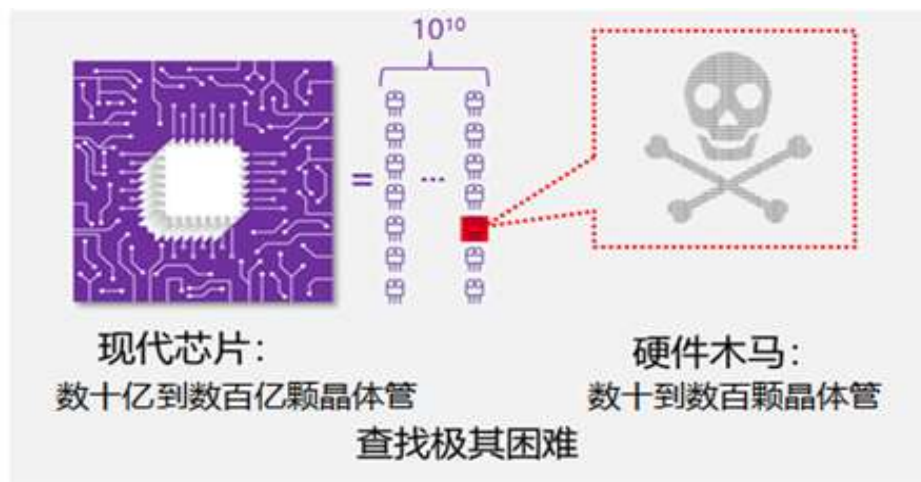


一、提升能量效率

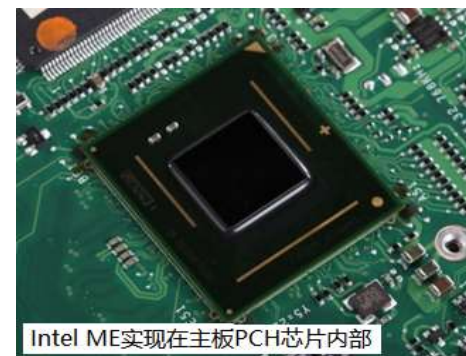
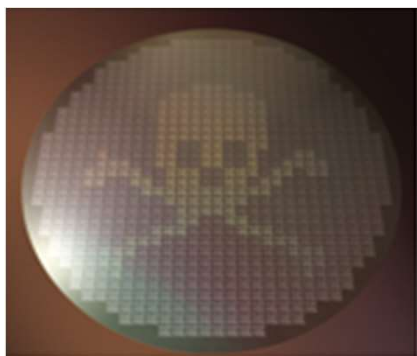
二、降低芯片成本

三、增强硬件安全性、可靠性

芯片硬件安全 (Hardware Security)



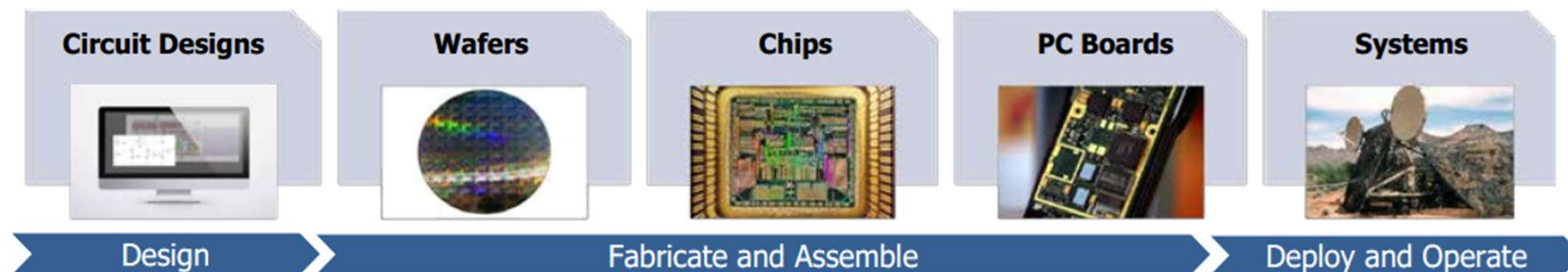
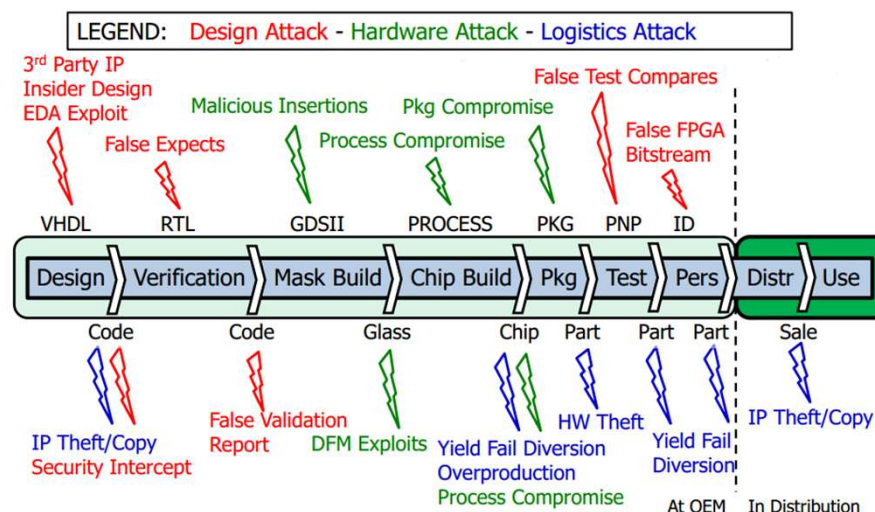
- 恶意硬件（木马/后门）、硬件漏洞、硬件前门等
 - 尚无有效手段能够进行检测和抵御



芯片硬件安全 (Hardware Security)



- ❑ **信息泄漏**
 - 硬件丢失
 - IP泄漏
 - 逆向工程
- ❑ **假冒伪劣产品**
 - 伪造
 - 克隆
 - 未授权生产
- ❑ **恶意电路**
 - 后门
 - 硬件木马
- ❑ **供应风险**
 - 器件过时
 - 供应链风险
- ❑ **设计漏洞**
- ❑ **质量和可靠性失效**



芯片硬件安全 – 类型



- **硬件漏洞**

- 熔断 (Meltdown) ¹、幽灵 (Spectre) ²及其变种
- 其他软件/硬件侧信道攻击, 如Branch Scope³, TLBleed⁴, VolJokey⁵,

- **硬件前门 (被利用后可能成为恶意硬件)**

- 处理器微码, 如Intel, AMD
- 主动管理子系统, 如Intel ME, AMD PSP

- **恶意硬件 (木马/后门)**

- 有相关报道^[6], 尚无我们可复现的实例

[1] Meltdown, <https://meltdownattack.com/>

[3] BranchScope: A New..., ASPLOS'18, March 24–28, 2018

[5] CCS 2019

[2] Spectre Attacks..., <https://meltdownattack.com/>

[4] Translation Leak-aside..., USENIX Security Symposium, 2018

[6] Breakthrough..., CHES 2012.14th international workshop, 2012

芯片硬件安全 –CPU Meltdown硬件漏洞



由CPU技术原理引发的硬件漏洞

- 非程序员的疏忽

	乱序执行	执行结果
1. ; rcx = kernel address 2. ; rbx = probe array 3. retry: 4. mov al, byte [rcx]		指令提交将产生异常
5. shl rax, 0xc 6. jz retry 7. mov rbx, qword [rbx + rax]	异常被执行前, line 7 有机会完成访存, 从而改变Cache布局	以下指令不会被提交 CPU将转入异常处理
	微结构状态改变	体系结构状态不变

熔断攻击 (概念原型代码)

	推测执行	执行结果
- x先赋值为较小数, 反复训练分支预测器 - 使用特定操作, 令条件判断执行时延变长 - 保证probe数组被清除出Cache - 保证目标数据已加载在Cache中 - x赋值为大数, 令array[x] 指向目标数据		
1. if (x < array_size)	预测条件为真	判断语句提交, 条件为假
2. y = probe[array[x] * 256];	有机会执行访存, 从而改变Cache布局	条件为假, 不会提交
	微结构状态改变	体系结构状态不变

幽灵攻击 (概念原型代码)

芯片硬件安全 – CPU Meltdown硬件漏洞



• 详解熔断攻击

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0					DI	RF	ALU1	WB	CO								
1						DI	RF	ALU1	WB	CO							
2							DI	RF	ALU1	WB							
3								DI	RF	ALU1							
4	add \$0x141, %rax		FE	DE	RN				DI	RF							
5	add \$0x141, %rax		FE	DE	RN					DI							
6	add \$0x141, %rax		FE	DE	RN						DI	RF	ALU1	WB	CO		
7	add \$0x141, %rax		FE	DE	RN							DI	RF	ALU1	WB	CO	
8	movb (%rcx), %al			FE	DE	RN	DI	RF	LSU	LSU	WB						CO
9	shl \$12, %rax			FE	DE	RN				DI	RF	ALU1					
10	jz retry			FE	DE	RN					DI	RF	ALU2	WB			
11	add \$target, %rax			FE	DE	RN						DI	RF	ALU2	WB		
12	movq (%rax), %rbx				FE	DE	RN						DI	RF	LSU	LSU	LSU

③ 指令9, 10, 11, 12得以使用保留站中存放的指令8的结果, 继续执行

⑤ 发生异常的指令8提交, 提交后将触发CPU跳入异常处理, 并冲刷流水线使得后续的指令无法提交。

异常延迟提交

FE:取指令
DE:指令解码
RN:寄存器重命名
DI:指令分发
RF:读数据
ALU1/ALU2:算术逻辑单元
LSU:内部写回
WB:物理寄存器写回
CO:写回

① 指令8 (movb (%rcx), %al) 是一条越权读取内核数据的指令

② 权限检查失败, 由于CPU使用精确异常 (CPU需要知道哪条指令发生了异常), 并不立即FLUSH流水线, 而是在reorder buffer对应的指令8位置标记一个异常。由于指令0-7使用和指令8相同的寄存器, 存在写后写依赖, 指令8只能在指令7提交后的周期15提交

④ 权限检查虽然失败, 但在此之前在周期13, 利用内核数据作为索引的访存请求已经发送给内存子系统, 导致这些数据从内存加载到了cache

芯片硬件安全 – CPU Spectre硬件漏洞



• 由CPU技术原理引发的硬件漏洞

- 非程序员的疏忽

• 软件无足够信息进行有效甄别及防范

- 软件无法了解CPU的部分重要硬件行为

	• 乱序执行	• 执行结果
<pre>1. ; rcx = kernel address 2. ; rbx = probe array 3. retry: 4. mov al, byte [rcx]</pre>		指令提交将产生异常
<pre>5. shl rax, 0xc 6. jz retry 7. mov rbx, qword [rbx + rax]</pre>	异常被执行前, line 7 有机会完成访存, 从而 改变Cache布局	以下指令不会被提交 CPU将转入异常处理
	微结构状态改变	体系结构状态不变

熔断攻击 (概念原型代码)

	• 推测执行	• 执行结果
• x先赋值为较小数, 反复训练分支预测器 • 使用特定操作, 令条件判断执行时延变长 • 保证probe数组被清除出Cache • 保证目标数据已加载在Cache中 • x赋值为大数, 令array[x] 指向目标数据		
<pre>1. if (x < array_size)</pre>	预测条件为真	判断语句提交, 条件为假
<pre>2. y = probe[array[x] * 256];</pre>	有机会执行访存, 从而 改变Cache布局	条件为假, 不会提交
	微结构状态改变	体系结构状态不变

幽灵攻击 (概念原型代码)

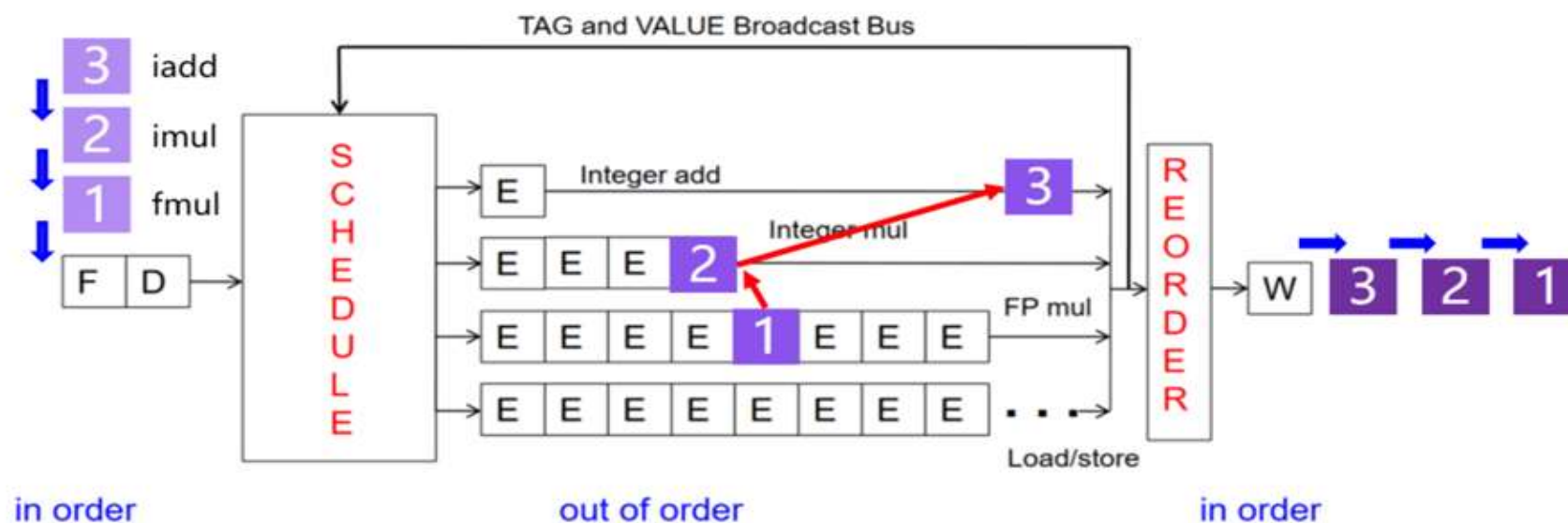
芯片硬件安全 –CPU Spectre硬件漏洞



软件可见

软件不可见

软件可见



处理器乱序执行示意

芯片硬件安全 – CPU Spectre硬件漏洞



• 由CPU技术原理引发的硬件漏洞

- 非程序员的疏忽

• 软件无足够信息进行有效甄别及防范

- 软件无法了解CPU的部分关键硬件行为

• 解决问题需彻底改变现有CPU架构

- CPU性能损失无法接受
 - 禁用推测/乱序等技术
 - 对推测执行/乱序等进行权限检查
- 重新设计Cache依旧无效
 - Cache状态回滚
 - Cache Isolation

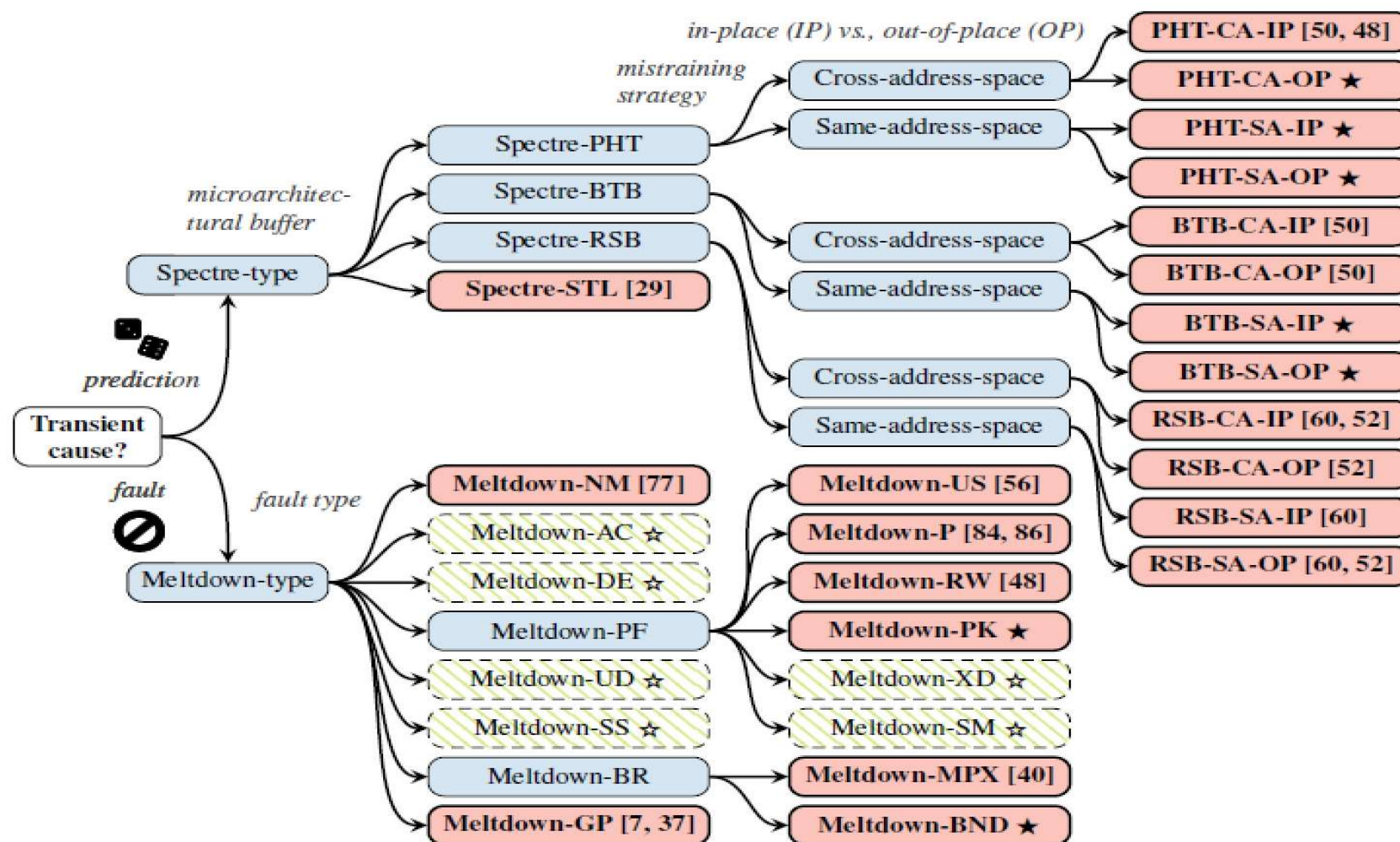
	• 乱序执行	• 执行结果
1. ; rcx = kernel address 2. ; rbx = probe array 3. retry: 4. mov al, byte [rcx]		指令提交将产生异常
5. shl rax, 0xc 6. jz retry 7. mov rbx, qword [rbx + rax]	异常被执行前, line 7 有机会完成访存, 从而 改变Cache布局	以下指令不会被提交 CPU将转入异常处理
	微结构状态改变	体系结构状态不变

熔断攻击 (概念原型代码)

	• 推测执行	• 执行结果
• x先赋值为较小数, 反复训练分支预测器 • 使用特定操作, 令条件判断执行时延变长 • 保证probe数组被清除出Cache • 保证目标数据已加载在Cache中 • x赋值为大数, 令array[x] 指向目标数据		
1. if (x < array_size)	预测条件为真	判断语句提交, 条件为假
2. y = probe[array[x] * 256];	有机会执行访存, 从而 改变Cache布局	条件为假, 不会提交
	微结构状态改变	体系结构状态不变

幽灵攻击 (概念原型代码)

芯片硬件安全 – CPU Meltdown硬件漏洞



[1] Gruss, et al. A Systematic Evaluation of Transient Execution Attacks and Defenses. 28th USENIX Security Symposium 2019.

芯片硬件安全 – CPU Meltdown硬件漏洞



Vulnerability Name(alikes)	CVE
Spectre v1 Bounds Check Bypass	2017-5753
Spectre v2 Branch Target Injection	2017-5715
SpectreRSB/ret2spec Return Mispredict	2018-15572
Meltdown Rogue Data Cache Load	2017-5754
Spectre-NG v3a	2018-3640
Spectre-NG v4 Speculative Store Bypass	2018-3639
Foreshadow L1 Terminal Fault (L1TF)	2018-3615
Spectre-NG Lazy FP State Restore	2018-3665
Spectre-NG v1.1 Bounds Check Bypass Store	2018-3693
Foreshadow-OS L1 Terminal Fault (L1TF)	2018-3620
Foreshadow-VMM L1 Terminal Fault (L1TF)	2018-3646
RIDL/ZombieLoad Microarchitectural Fill Buffer Data Sampling (MFBDS)	2018-12130
RIDL Microarchitectural Load Port Data Sampling (MLPDS)	2018-12127
RIDL Microarchitectural Data Sampling Uncacheable Memory (MDSUM)	2019-11091
Fallout Microarchitectural Store Buffer Data Sampling (MSBDS)	2018-12126
Spectre SWAPGS	2019-1125
RIDL/ZombieLoad v2 Transactional Asynchronous Abort (TAA)	2019-11135
RIDL/CacheOut/ZombieLoad L1D Eviction Sampling (L1DES)	2020-0549
RIDL Vector Register Sampling (VRS)	2020-0548
Load Value Injection (LVI)	2020-0551
CROSSTalk Special Register Buffer Data Sampling (SRBDS)	2020-0543
Branch History Injection (BHI)	2022-0001 2022-0002
Retbleed	2022-29900 2022-29901
Cross-Thread Return Address Predictions	2022-27672
Downfall Gather Data Sampling (GDS)	2022-40982
INCEPTION Speculative Return Stack Overflow (SRSO)	2023-20569
Zenbleed	2023-20593

[1] https://en.wikipedia.org/wiki/Transient_execution_CPU_vulnerability

芯片硬件安全 – 其他CPU硬件漏洞

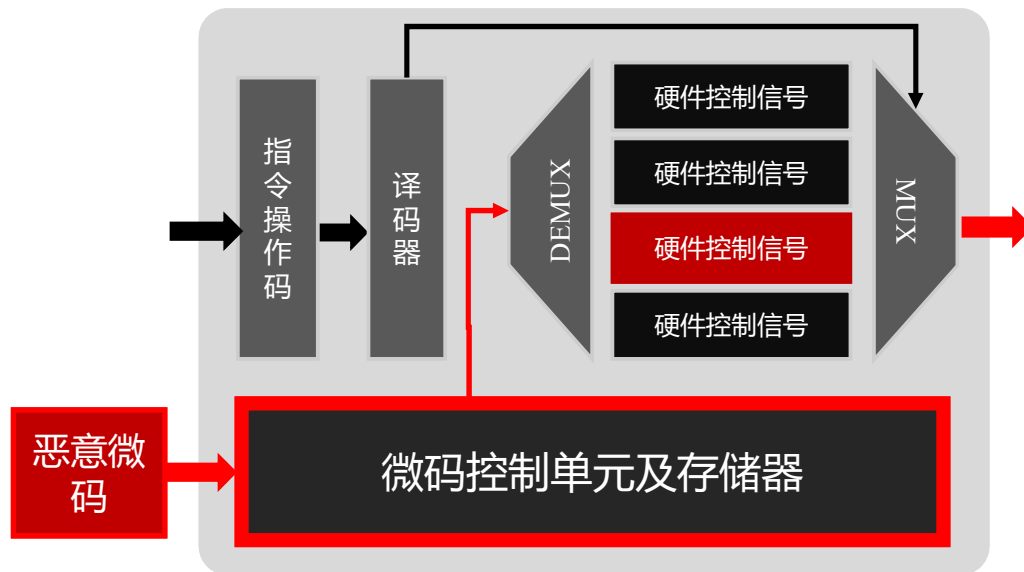


- Branch Scope (ASPLOS 2018)
 - **旁路攻击CPU的分支预测器**
 - 泄露SGX (软件保护扩展) 防护的数据
- TLBleed (USENIX Security 2018)
 - **旁路攻击CPU的TLB(地址翻译缓冲)**
 - 破坏进程间隔离, 造成数据泄露
- VolJokey (CCS 2019)
 - **旁路攻击CPU的驱动电压**
 - 泄露重要数据
- Hertzbleed (USENIX 22)
 - **旁路攻击CPU的DVFS模块**
 - 通过功耗, 频率侧信道泄露重要数据

芯片硬件安全 – CPU硬件前门



- 微码 (Intel、AMD)
 - **厂商预留，不公开**
 - 用于硬件维护与升级
 - 性能提升的重要手段
 - **可改变处理器指令行为**



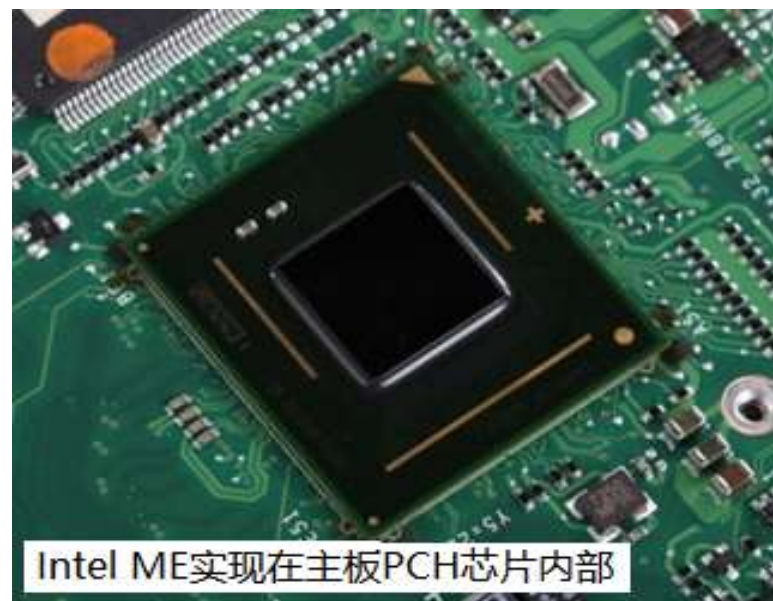
芯片硬件安全 – CPU硬件前门



- 主动管理子系统 (ME、PSP)
 - **厂商预留、不公开**
 - 绕过CPU，直接访问内存资源



可通过ME攻击一台已关闭的电脑：完成启动，远程控制等一系列操作



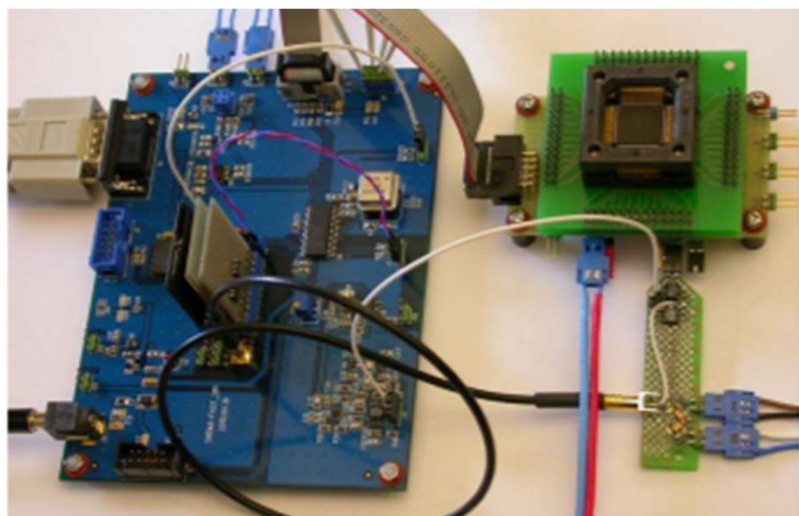
主板PCH芯片 (ME内嵌其中)

- 子系统前门的危害
 - **电脑关闭，仍可被远程控制**
 - **软件手段无法察觉**

芯片硬件安全 – 硬件木马



- 首个公开报道的真实芯片中的木马
 - 军用FPGA芯片
- 与密码运算相关的重要数据遭泄露



恶意硬件原型验证板

Breakthrough silicon scanning discovers backdoor in military chip

Sergei Skorobogatov¹ and Christopher Woods²

¹ University of Cambridge, Computer Laboratory, Cambridge, UK

sps32@cam.ac.uk

² Quo Vadis Labs, London, UK

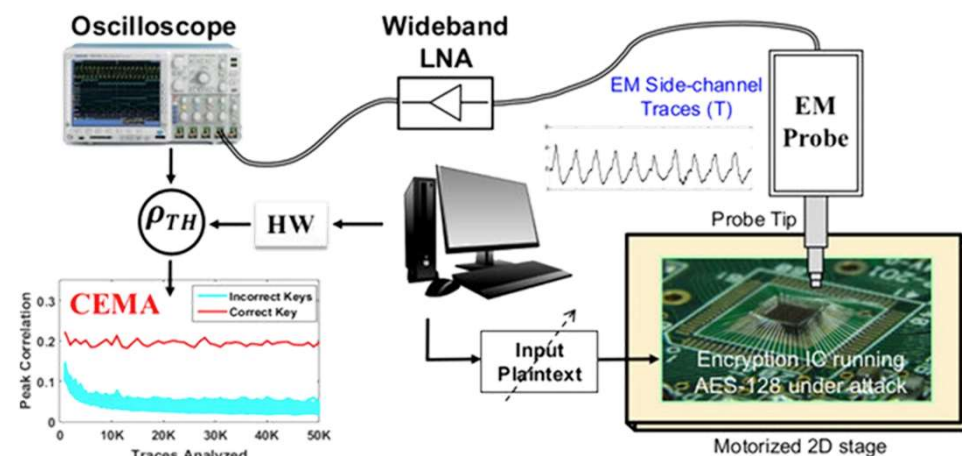
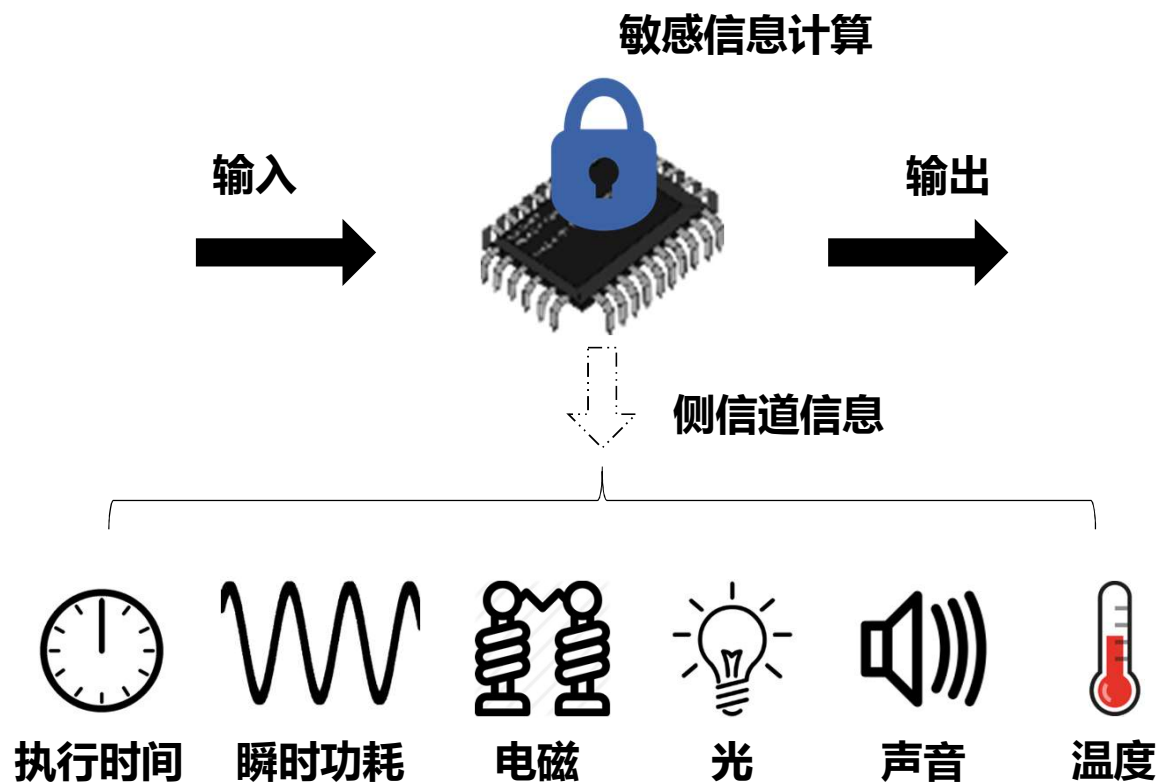
chris@quovadislabs.com

Abstract. This paper is a short summary of the first real world detection of a backdoor in a military grade FPGA. Using an innovative patented technique we were able to detect and analyse in the first documented case of its kind, a backdoor inserted into the Actel/Microsemi ProASIC3 chips for accessing FPGA configuration. The backdoor was found amongst additional JTAG functionality and exists on the silicon itself, it was not present in any firmware loaded onto the chip. Using Pipeline Emission Analysis (PEA), our pioneered technique, we were able to extract the secret key to activate the backdoor, as well as other security keys such as the AES and the Passkey. This way an attacker can extract all the configuration data from the chip, reprogram crypto and access keys, modify low-level silicon features, access unencrypted

2012年, 剑桥大学¹

[1] Breakthrough silicon scanning discovers backdoor in military chip, CHES 2012

芯片硬件安全 – 面向密码芯片的侧信道攻击



旁路攻击示意图（电磁）

[1] Kocher, Paul, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS...", Advances in Cryptology, 1996.

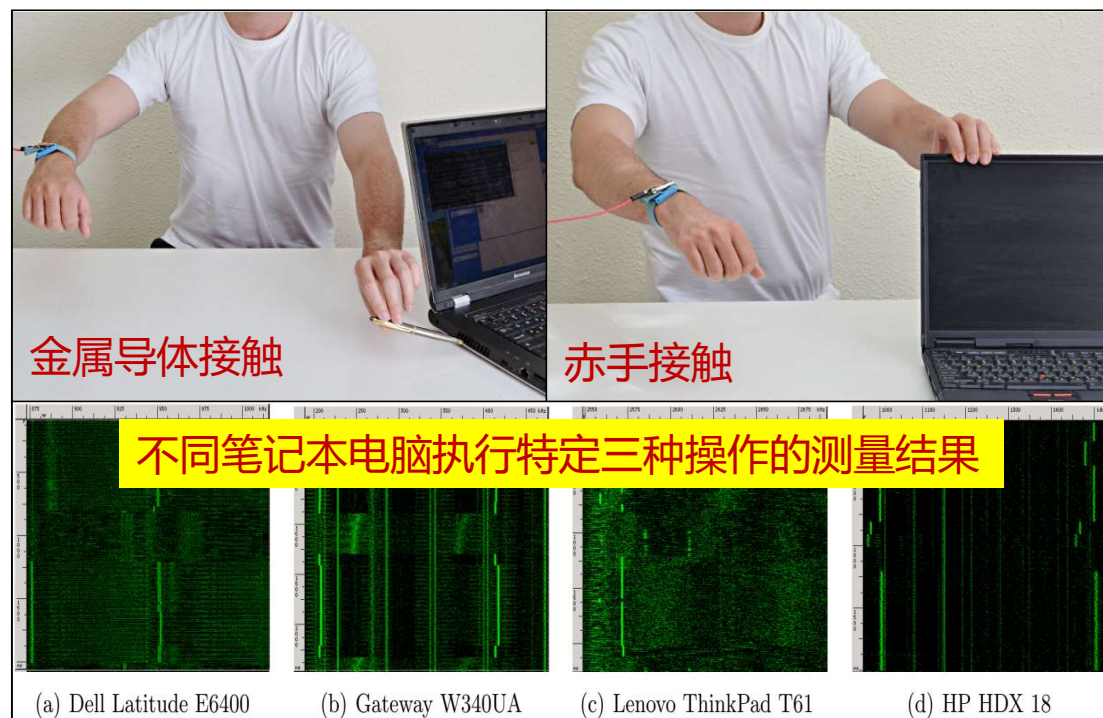
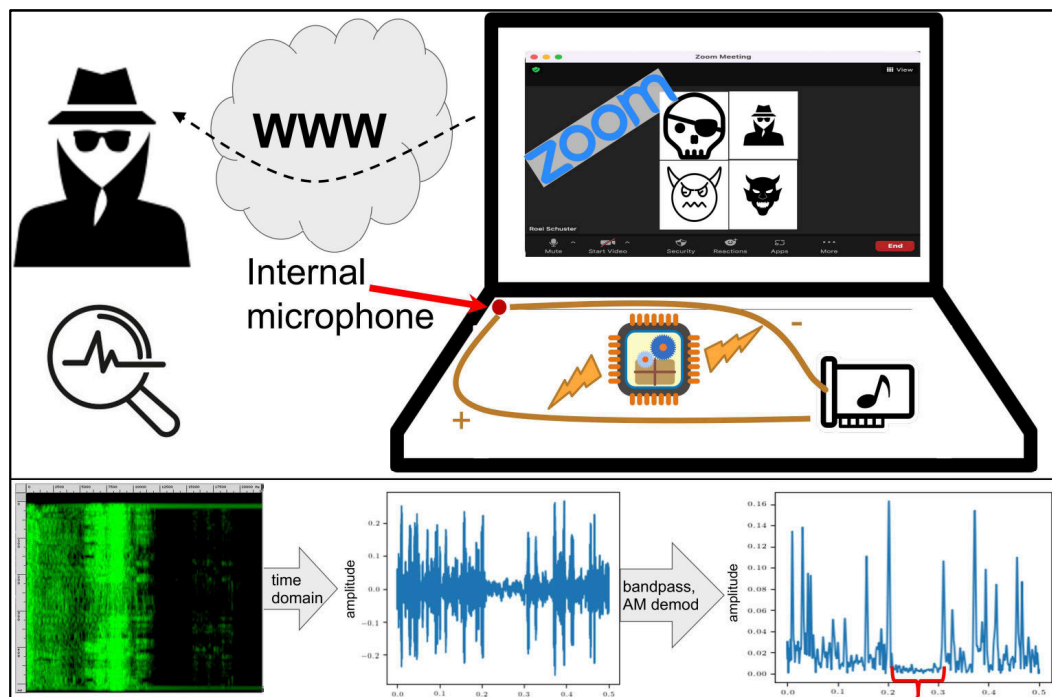
芯片硬件安全 – 面向密码芯片的侧信道攻击



旁路攻击举例

2022年，以色列特拉维夫大学团队利用目标笔记本电脑自己的麦克风获取CPU运行的电磁信息，**破解目标笔记本运行ECDSA加密时的密钥**^[1]

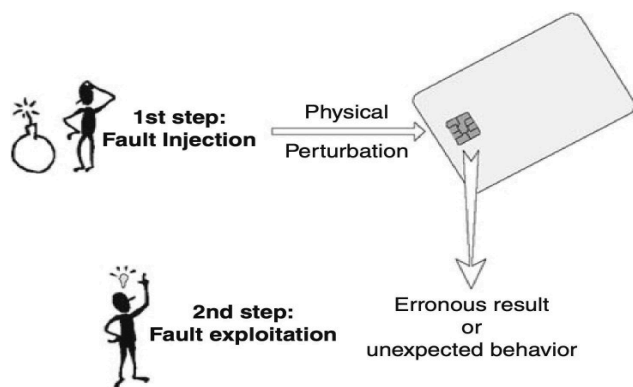
2014年，以色列特拉维夫大学团队通过接触笔记本裸露金属部分测量计算相关的对地电势，**成功提取笔记本上运行的RSA-4096和ElGamal-3072密钥**^[2]



[1] D. Genkin, N. Nissan, R. Schuster, E. Tromer, “Lend me your ear: ...”, USENIX Security Symposium, 2022

[2] D. Genkin, I. Pipman, E. Tromer, “Get your hands off my laptop ...”, CHES, 2014.

芯片硬件安全 – 面向密码芯片的错误注入攻击



错误注入攻击机理^[1]

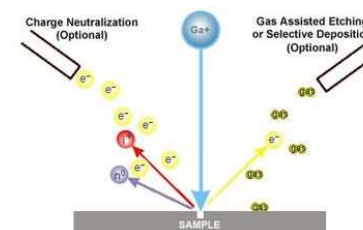
通过分析由物理扰动引入的错误，
获得敏感信息，或者能辅助密码
分析的重要信息。

激光



[Source: Breier et al., 2015]

聚焦离子束



[Source: wikipedia.org]

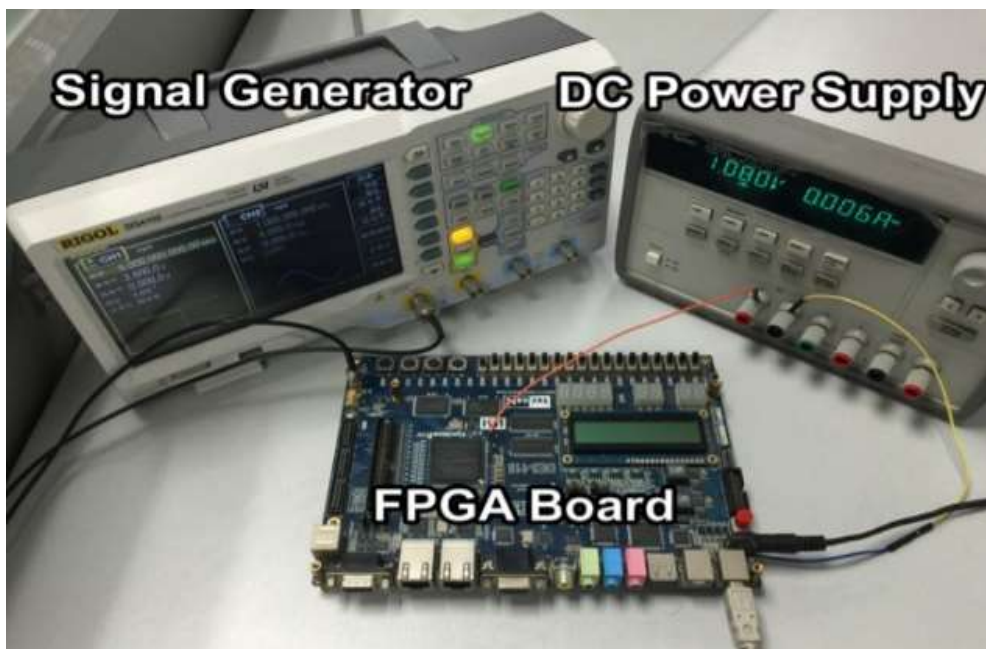
常见的物理扰动

- 电源上注入毛刺
- 改变时钟的占空比和频率
- 全局光束扰动/激光
- 电磁扰动

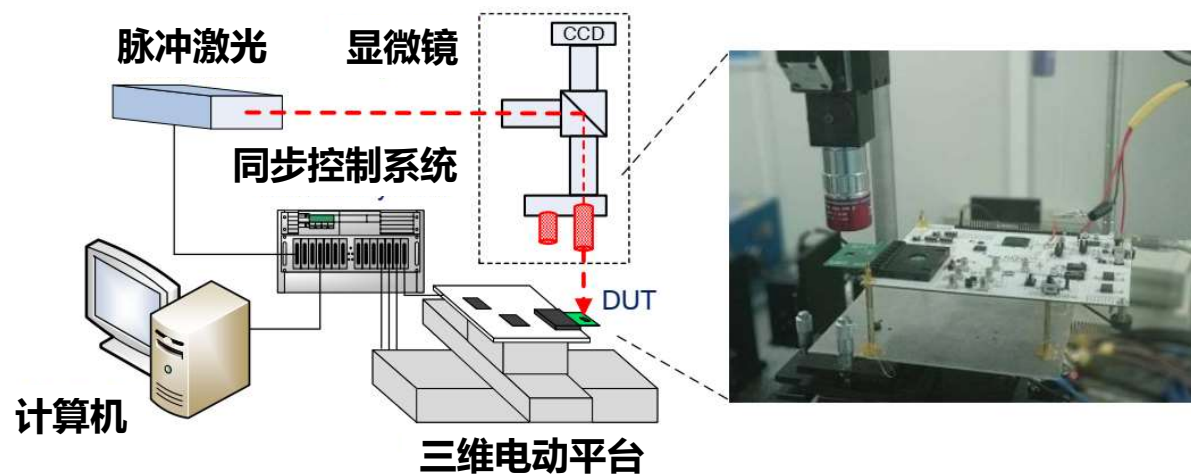
[1] Benot O. (2011) Fault Attack. In: van Tilborg H.C.A., Jajodia S. (eds) Encyclopedia of Cryptography and Security. Springer, Boston, MA.

芯片硬件安全 – 面向密码芯片的错误注入攻击

错误注入攻击举例



对FPGA芯片毛刺注入攻击 (清华 CHES 2015) [1]



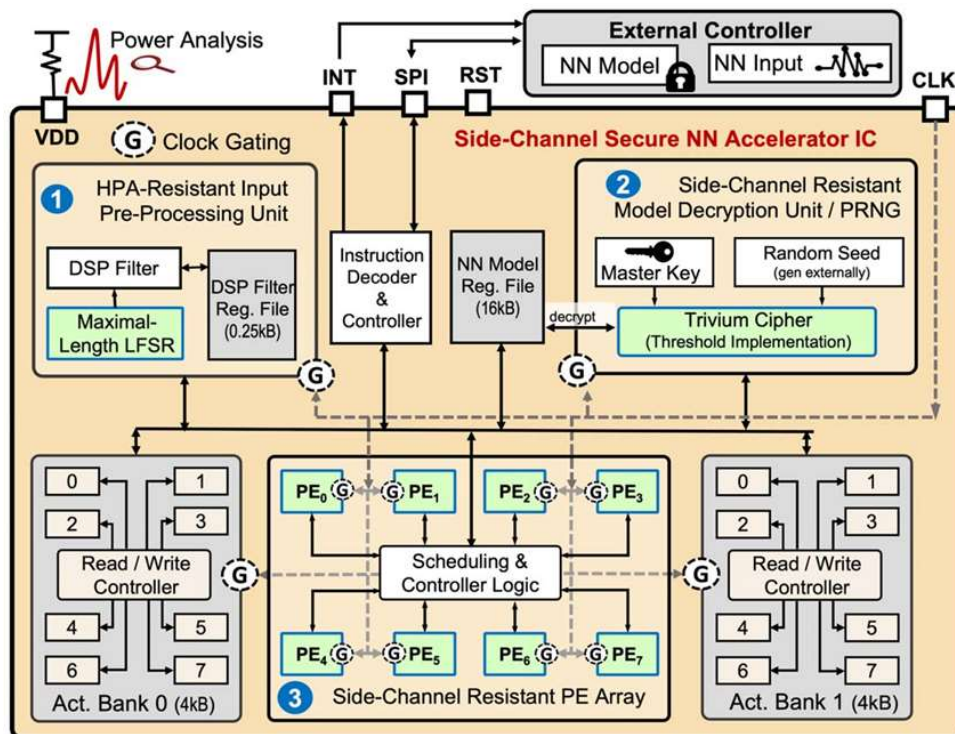
基于脉冲激光的错误注入攻击(浙大 CHES 2020) [2]

[1] Y. Ren, A. Wang, L. Wu, “Transient-Steady Effect Attack on Block Ciphers,” CHES 2015.

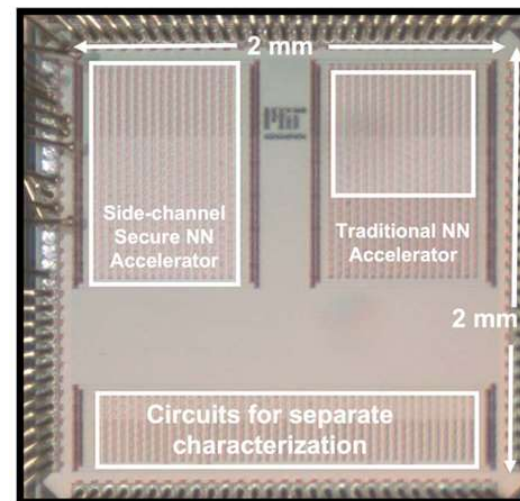
[2] F. Zhang, Y. Zhang, et. al, “Persistent Fault Attack in Practice,” CHES 2020.

芯片硬件安全 – 带有侧信道防护的AI芯片

- 基于阈值实现 (TI) 防护技术的嵌入式神经网络加速器芯片



侧信道安全的神经网络加速器芯片结构



Area of the Secure NN Accelerator: 1.59 μm^2
Area of the Traditional NN Accelerator: 0.97 μm^2

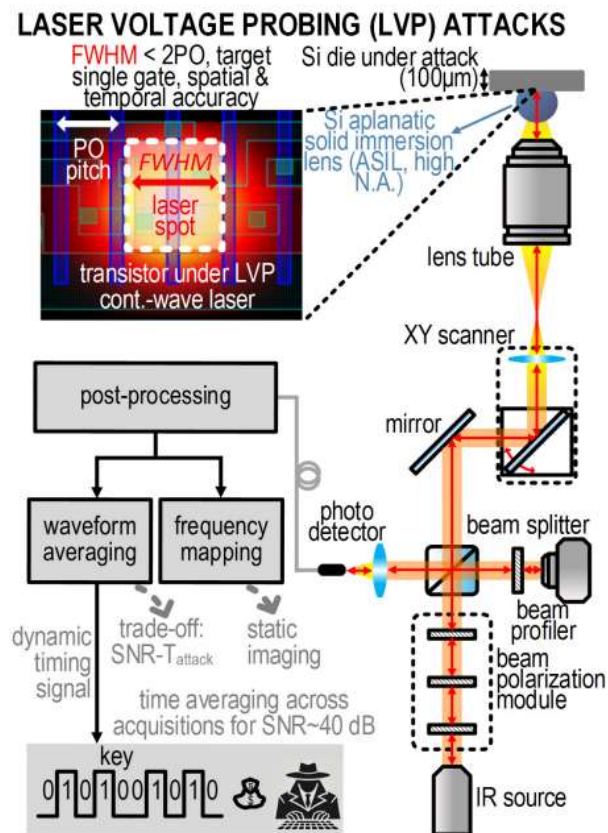
版图和参数列表

Chip Specifications	
Technology	28nm HPC+ CMOS
Supply Voltage	0.6-0.95V
Package	88-pin QFN
Die Size	2mm x 2mm

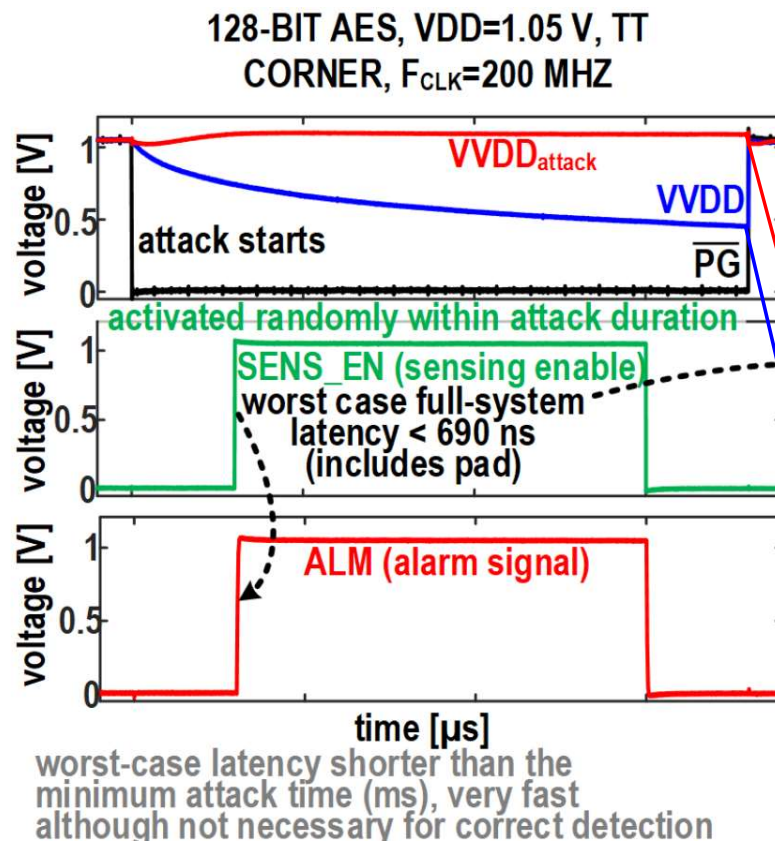
Specifications of the Side-Channel Secure Neural Network Accelerator	
Logic Area	164.4 kGE
Memory	24.5 kB
Operating Freq	10MHz (0.60V) 125MHz (0.95V)
Power	0.66mW (0.60V, 10MHz) 16.15mW (0.95V, 125MHz)
No. of PEs	8
Defense Methodology	Threshold Implementation (TI)
Cipher / PRNG	Trivium (TI-based)
HPA defense	Randomized order of opn.

芯片硬件安全 – 激光电压探测攻击检测

- 通过复用Power Gating晶体管作为隐式光学/温度传感器实现运行时攻击探测



激光电压探测攻击



检测效果

核心思路：控制Power Gating晶体管间歇下电：

受到攻击：VVDD持续拉高

无攻击：VVDD正常降低

芯片硬件安全 – 应对方案



- **芯片设计、制造、封装中的安全**
 - 芯片安全建模技术
 - 设计、制造和封装的流程安全
 - 安全性设计 — Design for Security
 - 侧信道：如何有效抵御
- **芯片部署、使用的硬件安全**
 - 芯片身份认证
 - 芯片安全性静态检查、动态监控
- **基于特定芯片硬件技术的系统安全**
 - 加解密技术
 - 安全子系统
 - 分割与异构提升安全性



可靠性设计 (Reliability) [1]

- 芯片可以正常、准确、稳定地发挥其功能和性能的能力和程度
- 针对由潜在的不可抗力因素引发的失效
- 失效因素
 - 工艺偏差
 - 环境因素：温度、湿度、射线、机械
 - 工作应力：电流、电压、时间、频率
- 物理效应
 - 工艺特点导致的物理效应
 - 电迁移效应、热载流子效应、经时击穿、负偏置温度不稳定性
 - 电路方面
 - 电离辐射效应、静电放电
- 在设计阶段，制定预防失效和失效后的保护措施
 - 电路、版图、工艺、封装

[1] 章晓文, 恩云飞. 半导体集成电路的可靠性及评价方法, 2015.



可靠性增强技术^[1]

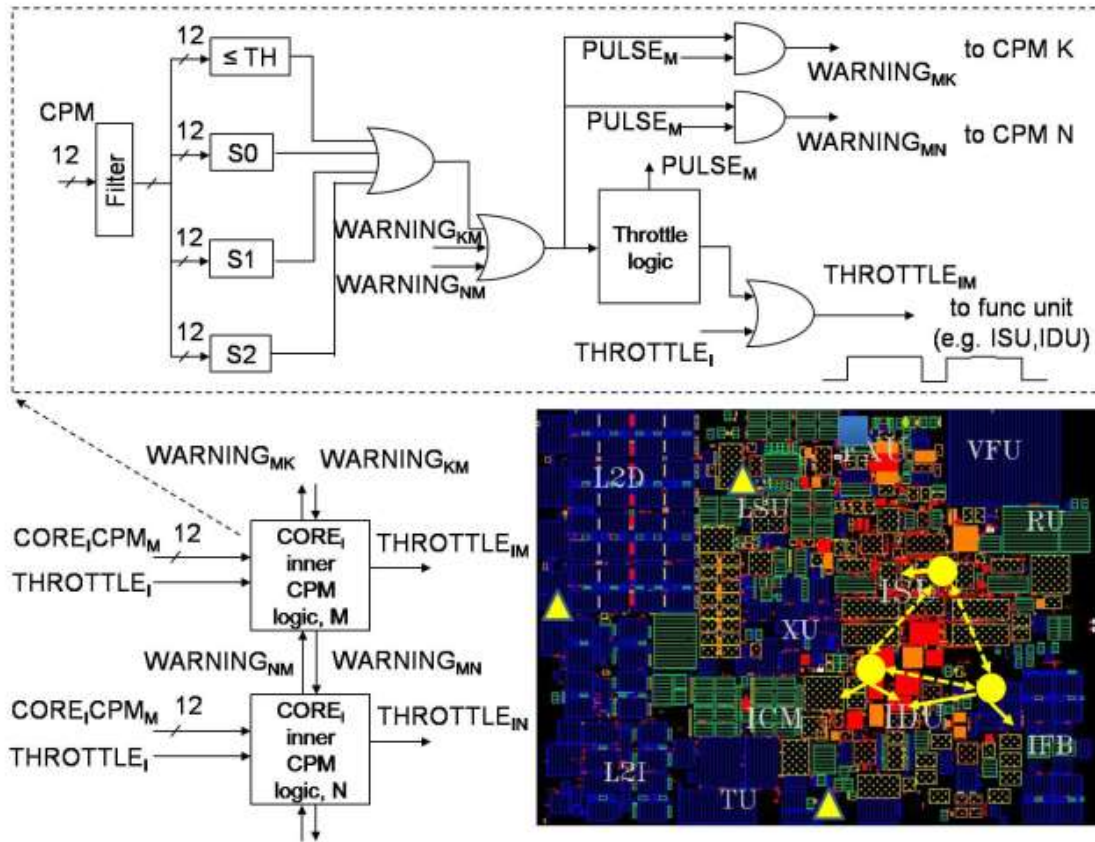
- 电压/频率缩放技术
- 补偿电路
- 片上监测+电压/时钟补偿
 - 环形振荡器、电流传感器、电压传感器
- 容错设计：在出现错误的情况下，仍然能够运行并得到正确的结果
 - 冗余技术
 - 硬件冗余：双/三冗余备份 (DMR, TMR)
 - 信息冗余：增加校验码或纠错码
 - 时间冗余：指令重新执行



[1] 李洋. 抗辐射容错计算机的设计与实现[D]. 哈尔滨工业大学, 2014.

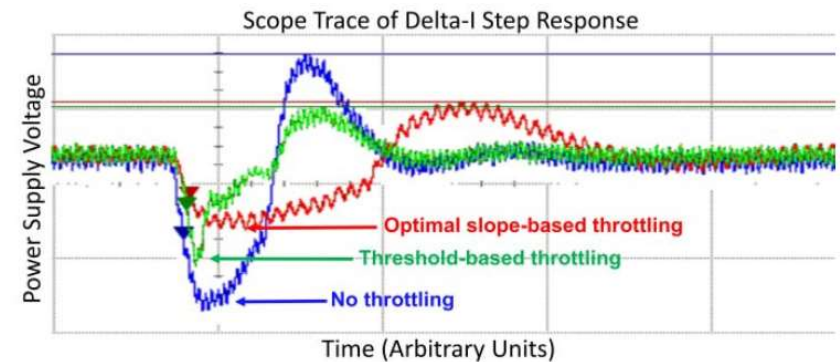
[2]http://www.yescar.cn/kanche/20201121/838267_1.html, 2020

IBM z14:利用电压管理增强可靠性



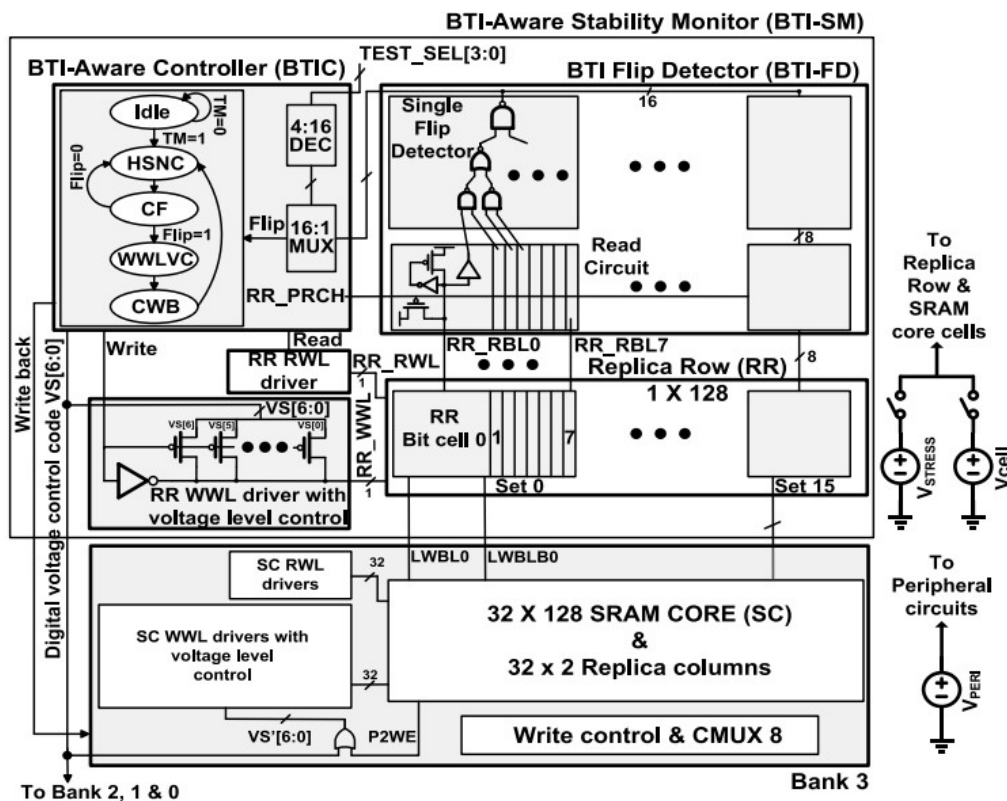
电压管理的电路实现

- 供电电压噪声 (voltage droops)
 - 关键路径监测 (CPM)
 - 压降补偿



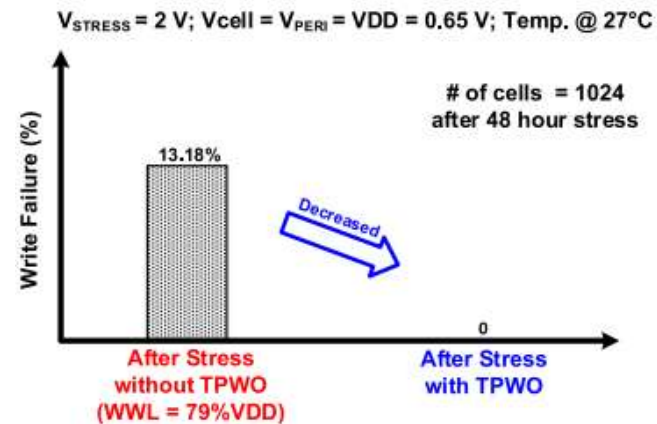
电压噪声及补偿仿真

片上动态可靠性管理技术



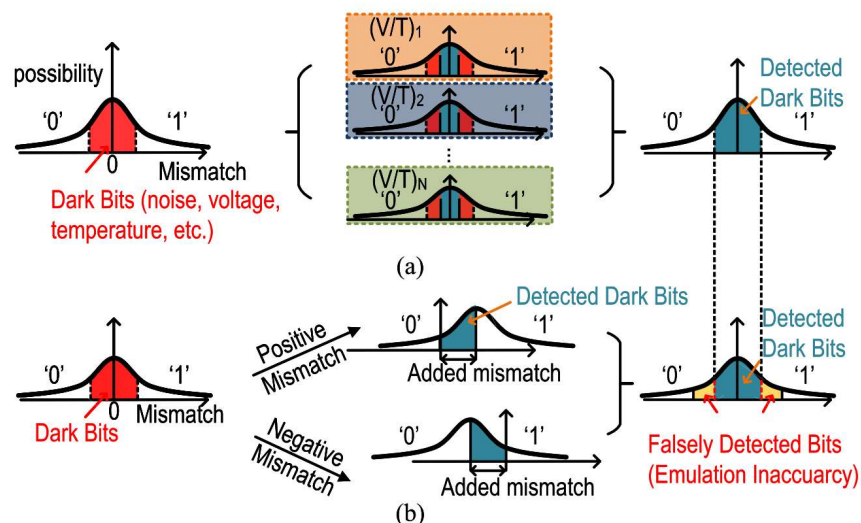
可靠性总体设计结构

- SRAM偏置温度不稳定 (BTI) 退化
 - BTI监测+写字线电压 (WWL) 调节
- 硬件代价
 - 10%面积
 - 3.42%功耗

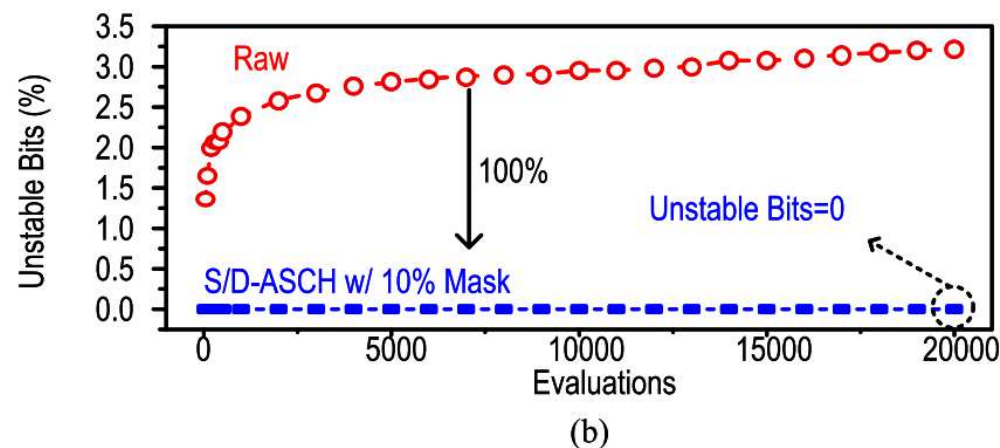
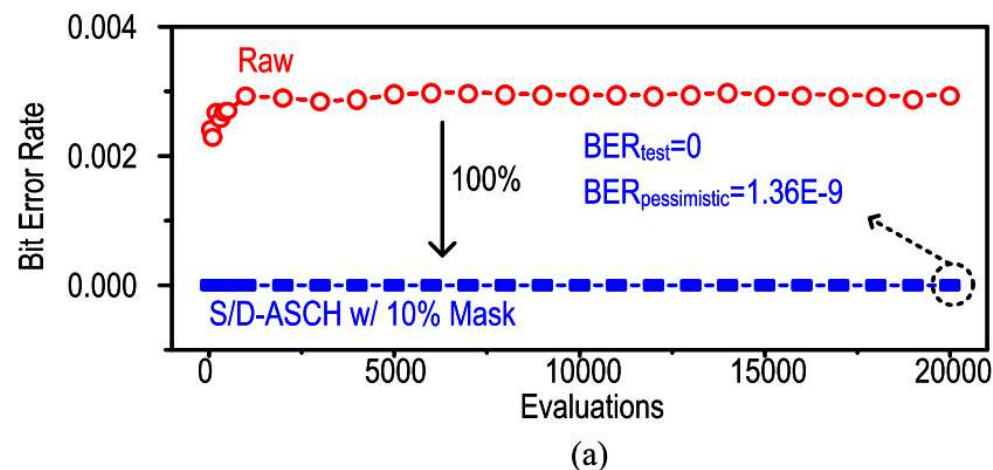


失效率测试

物理不可克隆函数中的Dark-bit 检测与自愈合技术



- 传统方案仅依靠ECC或掩码，开销大
- 在PUF阵列中集成低成本的自愈合机制以降低后续错误校验的开销
- “~0 BER”, $-20^{\circ}\text{C} \sim 125^{\circ}\text{C}$, $0.7\text{V} \sim 1.4\text{V}$



TMR (三模冗余)



- 故障注入

- 故障类型：单比特翻转
- 故障单元：计算单元、流水线寄存器

- 故障分类

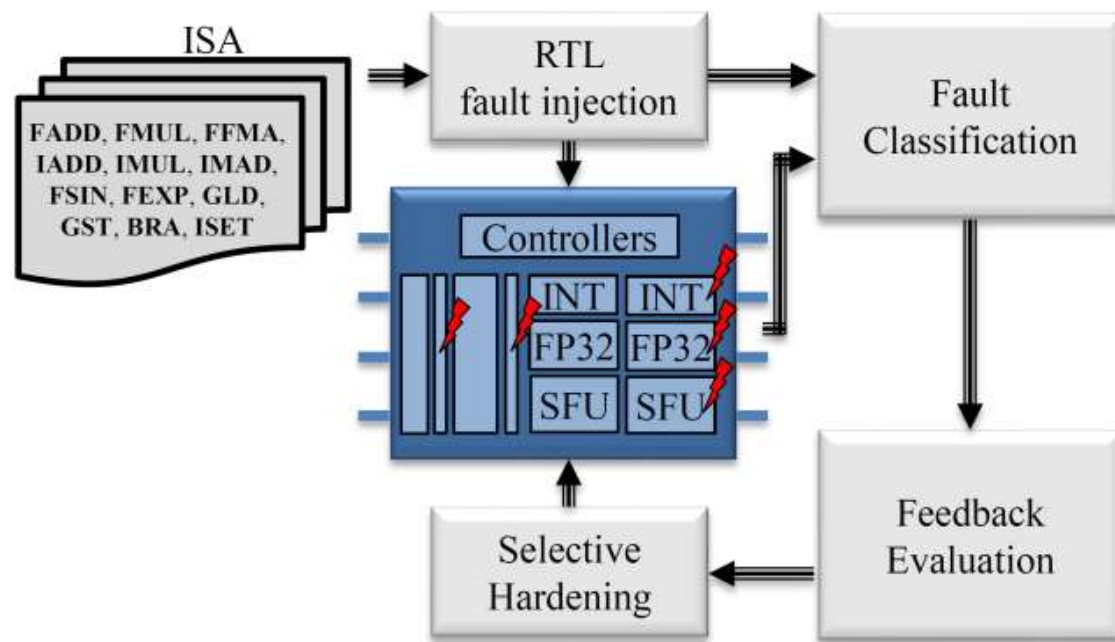
- 单/多线程崩溃、与准确结果的偏差大小

- 反馈评估

- 关键电路、常见错误

- 选择性实施硬件强化

- 三冗余 (TMR) 方法强化关键单元的可靠性

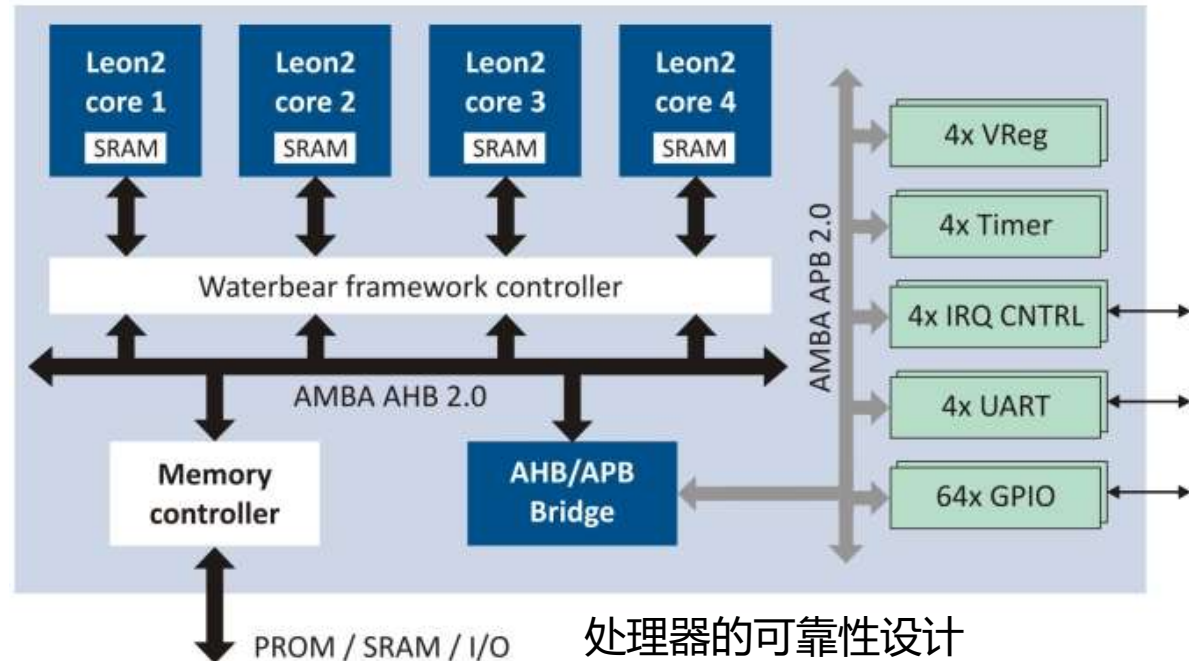


增强GPU微架构的总体方案

- TMR可以修正85%到99%的流水线寄存器故障、50%到100%的计算单元故障

集成电路可靠性

- 四个核心+Waterbear架构
- 三种工作模式
 - 容错模式
 - 双/三/四冗余
 - 高性能模式
 - 低功耗模式



- 在设计阶段增强硬件电路的可靠性, 不可避免地会增加额外的电路开销
- 增加电路面积、降低性能及能效
- 延长了使用寿命



- 利用软件定义芯片技术
 - 与监测电路配合, 动态调整电压、时钟
 - 对硬件进行重新映射, 避开失效部分



课程总结



本课总结

一、提升能量效率

- 提升性能
 - 并行技术→数据并行、线程并行、任务并行
 - 领域定制→针对应用领域对内存、计算、编程方法进行定制优化
- 降低功耗
 - 宽电压、近/亚阈值电压→性能下降
 - 门控电压/时钟→降低动态功耗
 - 可调电压/时钟→面向操作优化供电电压及时钟
- 综合提升能效
 - 异构集成→多种指令集、共享内存、针对某些特定领域进行定制设计
 - 新计算方法
 - 近存/存内计算→减少数据搬移开销、大规模并行计算、领域定制设计
 - 近似计算、概率计算→用精度换取性能/能效、面对容错应用



本课总结

二、降低芯片成本

- 异构集成/芯粒
 - 多芯片集成，良率提升
- 软件定义芯片
 - 一芯多用，功能可变
- 开源化、敏捷开发
 - 授权费降低，缩短开发时间，降低开发门槛

三、增强硬件安全性、可靠性

- 硬件安全
 - 需采用一系列芯片硬件安全增强方法
- 可靠性
 - 不同层面的可靠性增强技术

调研内容建议



- **调研主题：** state-of-the-art 数字集成电路技术设计技术
- **调研内容：** 提升能量效率、降低芯片成本、增强芯片安全性、增强芯片可靠性的数字集成电路设计技术，选其一即可
- **阅读文献：** 不少于 10 篇，其中精读不少于 3 篇
- **要 求：** 报告正文不少于 5000 字